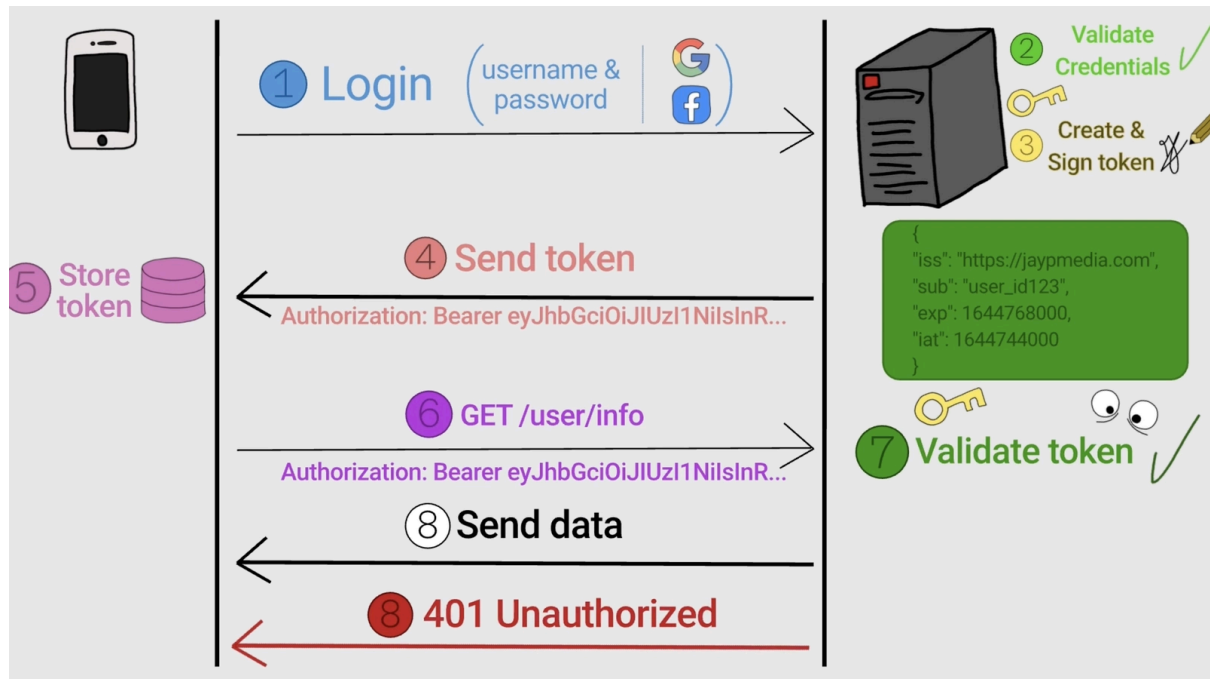


Actividad CI/CD



Tarea realizada por Ernesto Saquete Ríos

Curso de especialización en ciberseguridad

Índice

1.- Configuración inicial	2
2.- Fase 1 - Autenticación con JWT en Laravel	3
2.1.- Configuración de typon/jwt-auth	3
2.2.- Endpoints de autenticación	5
2.3.- Proteger las rutas	6
2.4.- Gestión de la expiración y el refresco	8
2.5.- Pruebas JWT	11
2.5.1.- Prueba login	12
2.5.2.- api/auth/me	13
2.5.3.- Prueba sin token	14
2.5.4.- Refresco de token	14
2.5.5.- Logout	15
2.5.6.- Reutilizar el token	16
2.6.- Test de integración	16
2.6.1.- Tests Auth	16
2.6.2.- Tests Directores	22
2.6.3.- Tests Peliculas	32
2.6.4.- Tests Seguridad transversal	43
3.- Fase 2 - CI/CD con GitHub Actions y Dev Containers	50
3.1.- Dev container	50
3.2.- Pipeline de Integración Continua (CI)	53
3.3.- Pipeline de Despliegue Continuo (CD)	72

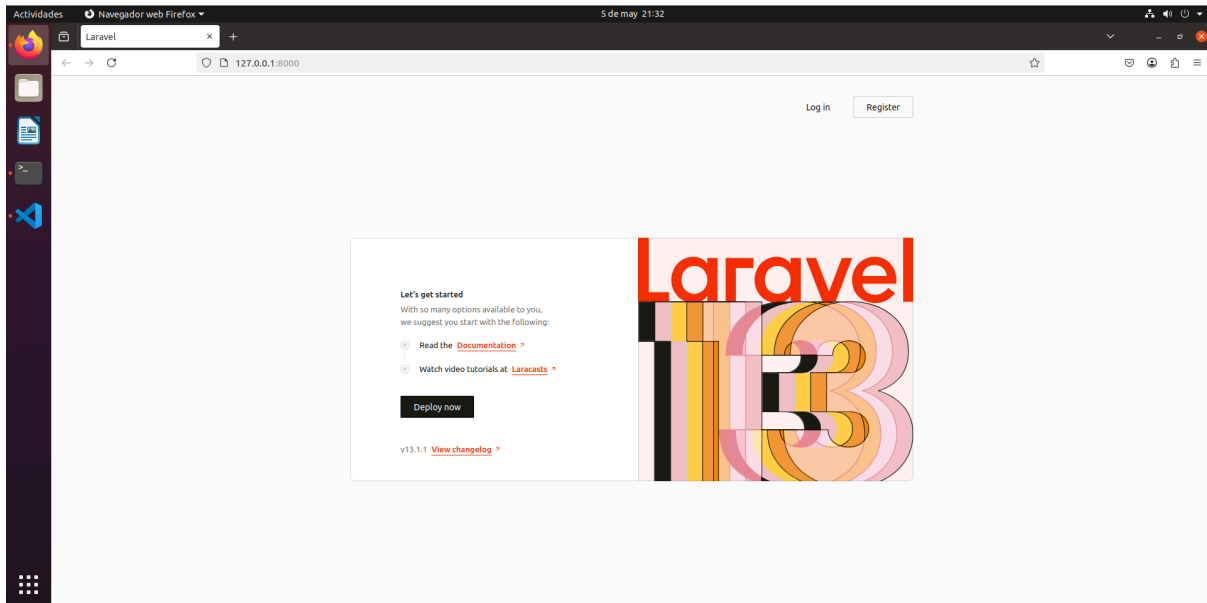
1.- Configuración inicial

Se define un entorno típico corporativo donde este malware podría operar:

Cuando se clona el proyecto peliculapp

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve
```

Con ello se levanta el servidor.



2.- Fase 1 - Autenticación con JWT en Laravel

2.1.- Configuración de tymon/jwt-auth

Lo primero que hay que hacer es instalar y configuración de tymon/jwt-auth

En primer lugar se añade la dependencia al proyecto mediante Composer:
composer require tymon/jwt-auth

```
usuario@MV-Ubuntu:~/peliculapp$ composer require tymon/jwt-auth
./composer.json has been updated
Running composer update tymon/jwt-auth
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking lcobucci/jwt (5.6.0)
  - Locking tymon/jwt-auth (2.3.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Downloading lcobucci/jwt (5.6.0)
  - Downloading tymon/jwt-auth (2.3.0)
  - Installing lcobucci/jwt (5.6.0): Extracting archive
  - Installing tymon/jwt-auth (2.3.0): Extracting archive
1 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

 INFO  Discovering packages.

laravel/breeze ..... DONE
laravel/pail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE
pestphp/pest-plugin-laravel ..... DONE
tymon/jwt-auth ..... DONE

85 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

 INFO  No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^2.3 for tymon/jwt-auth
usuario@MV-Ubuntu:~/peliculapp$
```

Después publicamos el fichero de configuración:

php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"

```
usuario@MV-Ubuntu:~/peliculapp$ php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"

INFO Publishing assets.

Copying file [vendor/tymon/jwt-auth/config/config.php] to [config/jwt.php] ..... DONE

usuario@MV-Ubuntu:~/peliculapp$
```

Generamos la clave secreta de firma JWT: php artisan jwt:secret

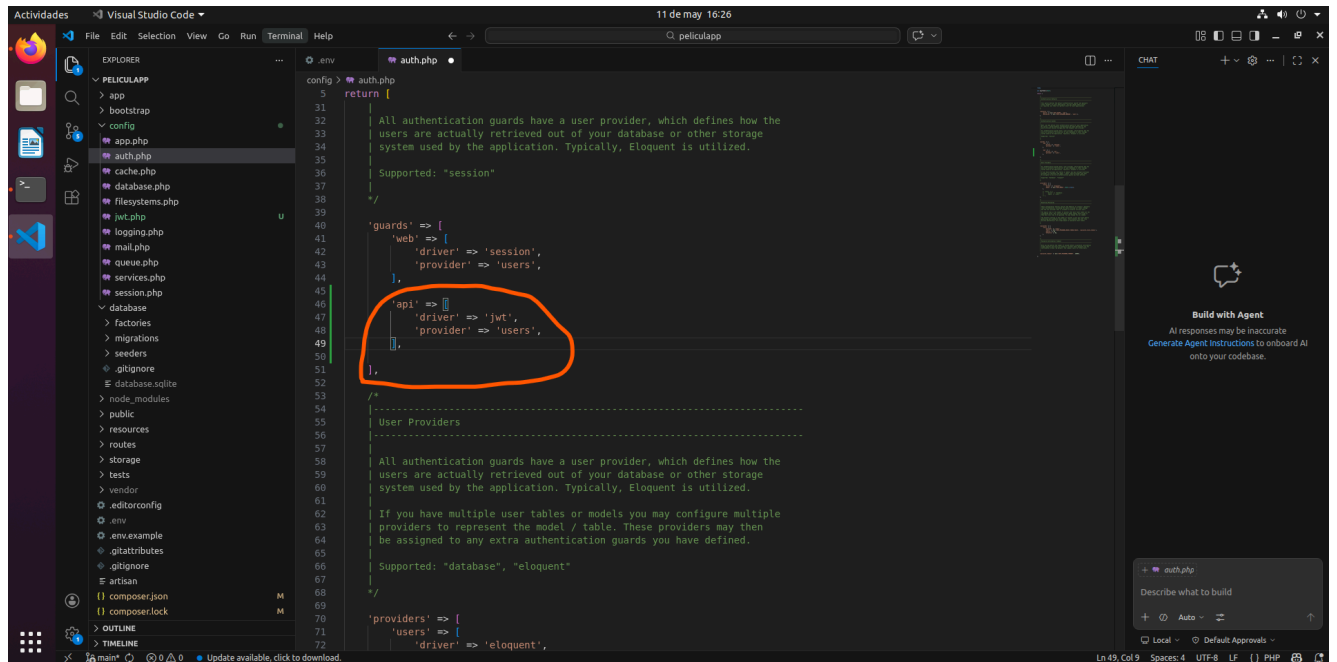
```
usuario@MV-Ubuntu:~/peliculapp$ php artisan jwt:secret
jwt-auth secret [REDACTED] set successfully.
usuario@MV-Ubuntu:~/peliculapp$
```

Configuramos el guard api en config/auth.php para usar el driver jwt. Se busca:

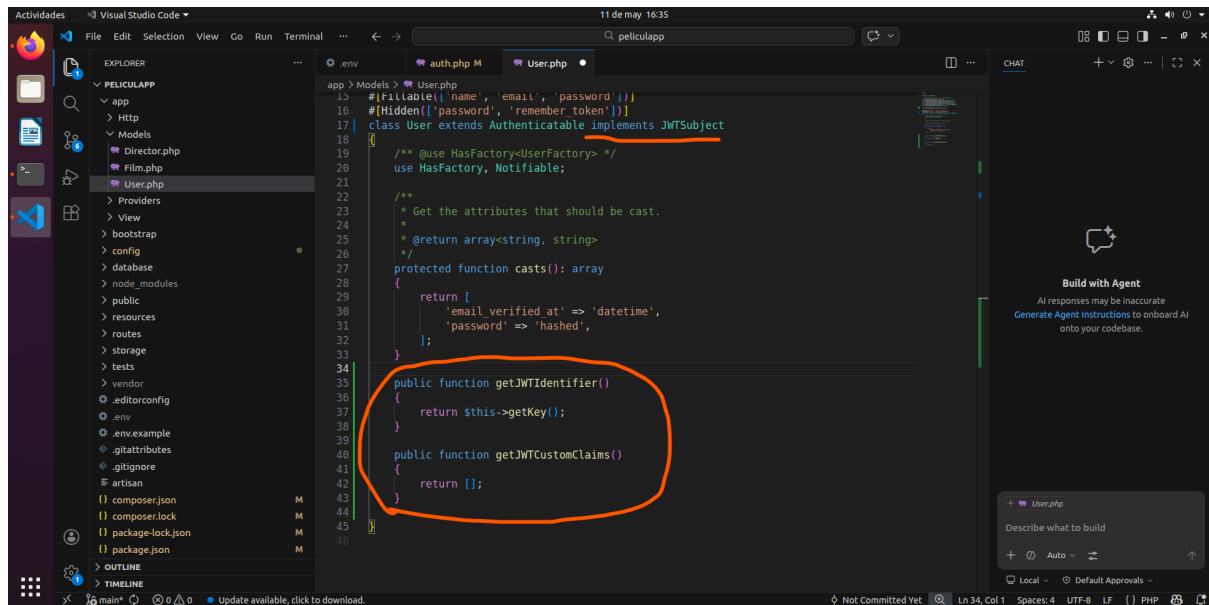
'guards' => [

Se añade:

```
'api' => [
    'driver' => 'jwt',
    'provider' => 'users',
],
```

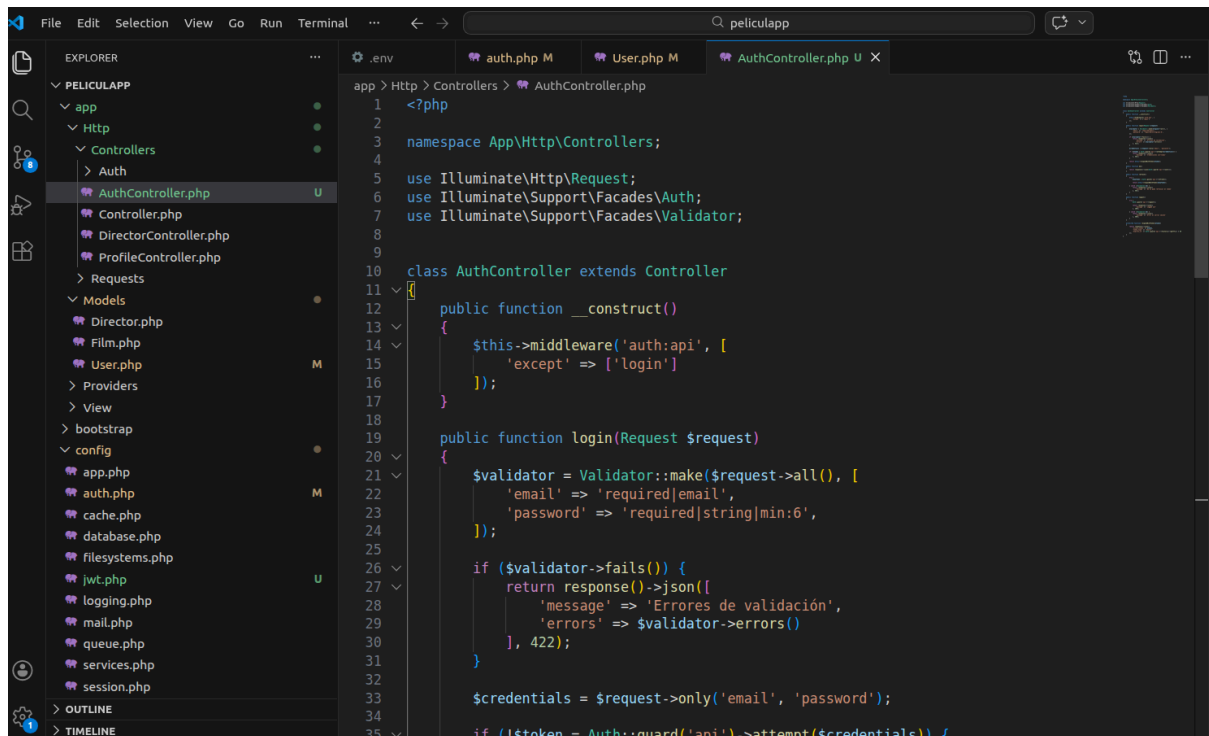


Por último se implementa la interfaz JWTSubject en el modelo User (métodos getJWTIdentifier y getJWTCustomClaims).

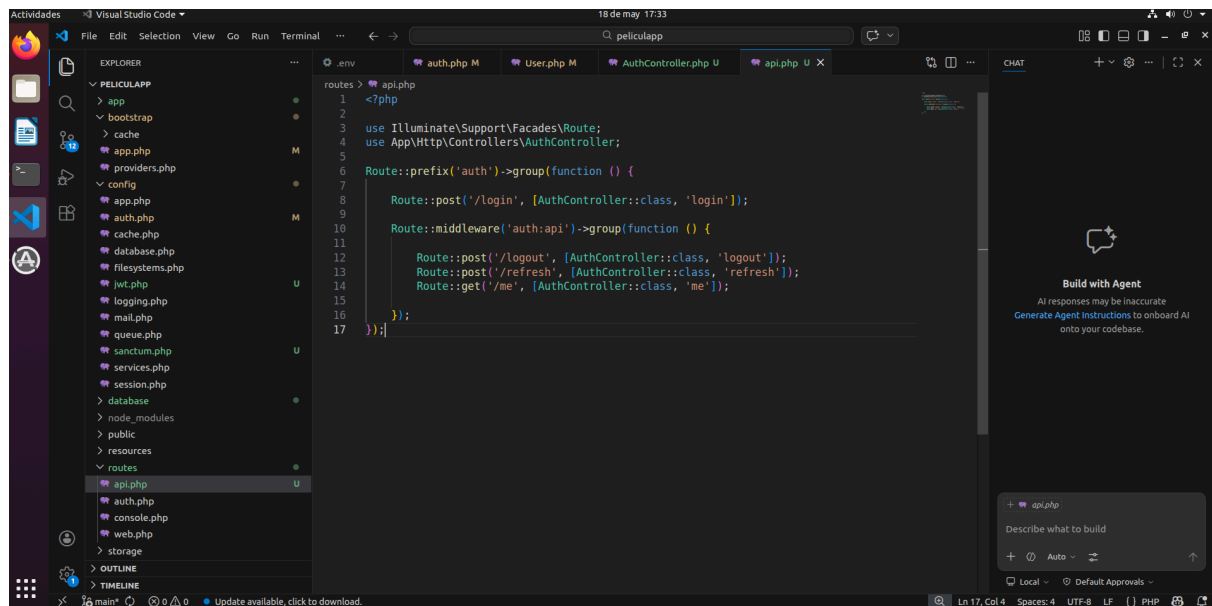


2.2.- Endpoints de autenticación

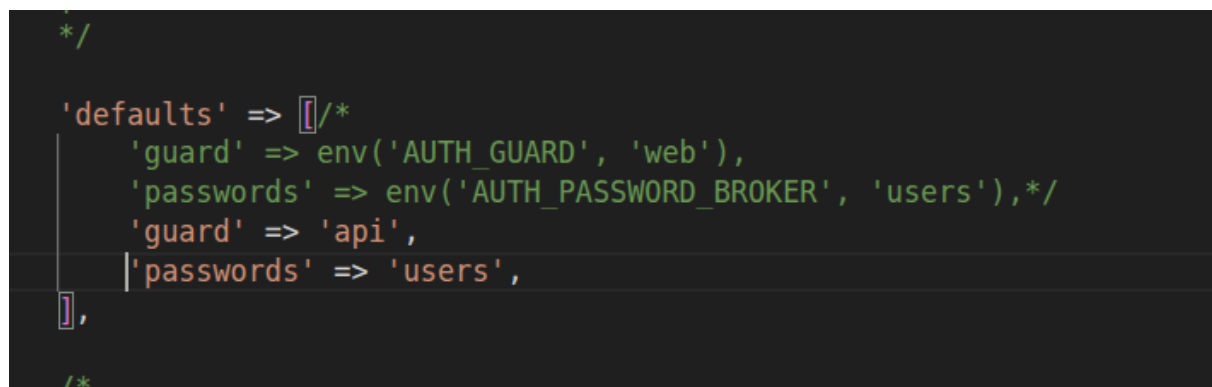
Teniendo en cuenta los endpoints requeridos bajo el prefijo /api/auth, se crea el AuthController:



Aseguramos que las respuestas de error no exponen información sensible del servidor.



Luego en config/auth.php, 'defaults' quedaría así:



Limpiamos caché y verificamos las rutas.

```
marlogMV-Ubuntu:~/peliculapp$ php artisan optimize:clear
INFO: Clearing cached bootstrap files.

config ..... 0.58ms DONE
cache ..... 57.50ms DONE
compiled ..... 0.47ms DONE
events ..... 0.23ms DONE
routes ..... 0.22ms DONE
views ..... 9.65ms DONE

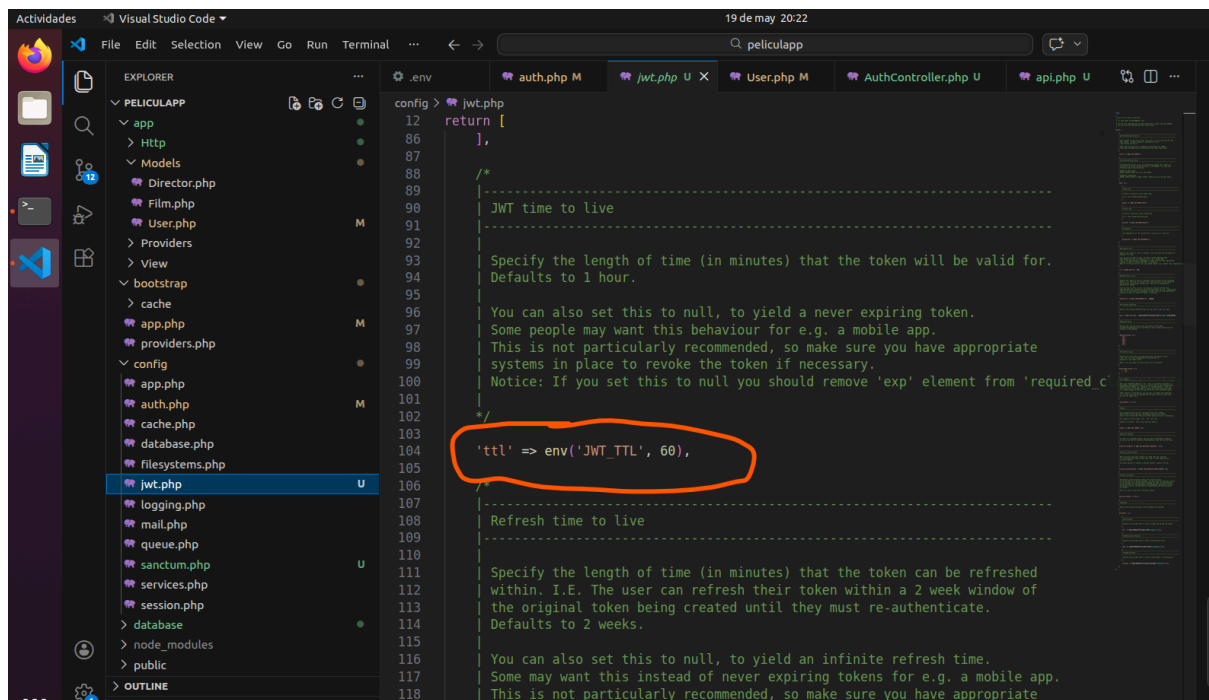
marlogMV-Ubuntu:~/peliculapp$ php artisan route:list

GET|HEAD / ..... routes/web.php:7
POST api/auth/login ..... AuthController@login
POST api/auth/logout ..... AuthController@logout
GET|HEAD api/auth/me ..... AuthController@me
POST api/auth/refresh ..... AuthController@refresh
POST confirm-password ..... password.confirm ..... Auth\ConfirmablePasswordController@show
POST confirm-password ..... password.confirm ..... Auth\ConfirmablePasswordController@store
GET|HEAD dashboard ..... dashboard ..... routes/web.php:11
GET|HEAD directors ..... directors.index ..... DirectorController@index
POST directors ..... directors.create ..... DirectorController@create
GET|HEAD directors/{director} ..... directors.show ..... DirectorController@show
POST email/verification-notification ..... verification.send ..... Auth\EmailVerificationNotificationController@store
POST forgot-password ..... password.request ..... Auth\PasswordResetLinkController@create
POST forgot-password ..... password.email ..... Auth\PasswordResetLinkController@store
POST login ..... login ..... Auth\AuthenticatedSessionController@create
POST login ..... login ..... Auth\AuthenticatedSessionController@store
POST logout ..... logout ..... Auth\AuthenticatedSessionController@destroy
PUT password ..... password.update ..... Auth\PasswordController@update
GET|HEAD profile ..... profile.edit ..... ProfileController@edit
PATCH profile ..... profile.update ..... ProfileController@update
DELETE profile ..... profile.destroy ..... ProfileController@destroy
GET|HEAD register ..... register ..... Auth\RegisteredUserController@create
POST register ..... register ..... Auth\RegisteredUserController@store
POST reset-password/{token} ..... password.store ..... Auth\NewPasswordController@store
GET|HEAD reset-password/{token} ..... password.reset ..... Auth\NewPasswordController@create
GET|HEAD sanctum/csrf-cookie ..... sanctum.csrf-cookie ..... Laravel\Sanctum \CsrfCookieController@show
GET|HEAD storage/{path} ..... storage.local ..... vendor\laravel\framework\src\Illuminate\Filesystem\FilesystemServiceProvider.php:111
PUT storage/{path} ..... storage.local.upload ..... vendor\laravel\framework\src\Illuminate\Filesystem\FilesystemServiceProvider.php:119
GET|HEAD up ..... vendor\laravel\framework\src\Illuminate\Foundation\Configuration\ApplicationBuilder.php:221
GET|HEAD verify-email ..... verification.notice ..... Auth\EmailVerificationPromptController
GET|HEAD verify-email/{id}/{hash} ..... verification.verify ..... Auth\VerifyEmailController

Showing [32] routes
```

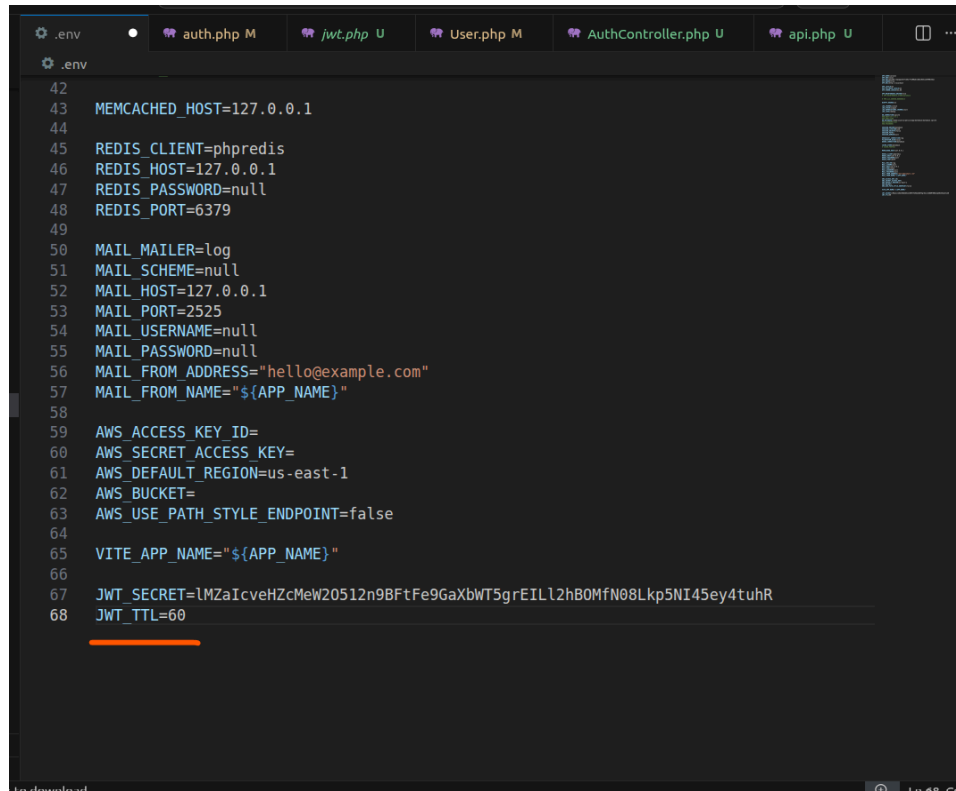
2.4.- Gestión de la expiración y el refresco

Configuramos el tiempo de expiración del token en config/jwt.php estableciendo en 60 minutos:



```
config > jwt.php
12  return [
13      ],
14  ];
15
16  /*
17   * JWT time to live
18   *
19   * Specify the length of time (in minutes) that the token will be valid for.
20   * Defaults to 1 hour.
21   *
22   * You can also set this to null, to yield a never expiring token.
23   * Some people may want this behaviour for e.g. a mobile app.
24   * This is not particularly recommended, so make sure you have appropriate
25   * systems in place to revoke the token if necessary.
26   * Notice: If you set this to null you should remove 'exp' element from 'required c
27   */
28  'ttl' => env('JWT_TTL', 60),
29
30  /*
31   * Refresh time to live
32   *
33   * Specify the length of time (in minutes) that the token can be refreshed
34   * within. I.E. The user can refresh their token within a 2 week window of
35   * the original token being created until they must re-authenticate.
36   * Defaults to 2 weeks.
37   *
38   * You can also set this to null, to yield an infinite refresh time.
39   * Some may want this instead of never expiring tokens for e.g. a mobile app.
40   * This is not particularly recommended, so make sure you have appropriate
```

En el archivo .env ponemos JWT_TTL=60

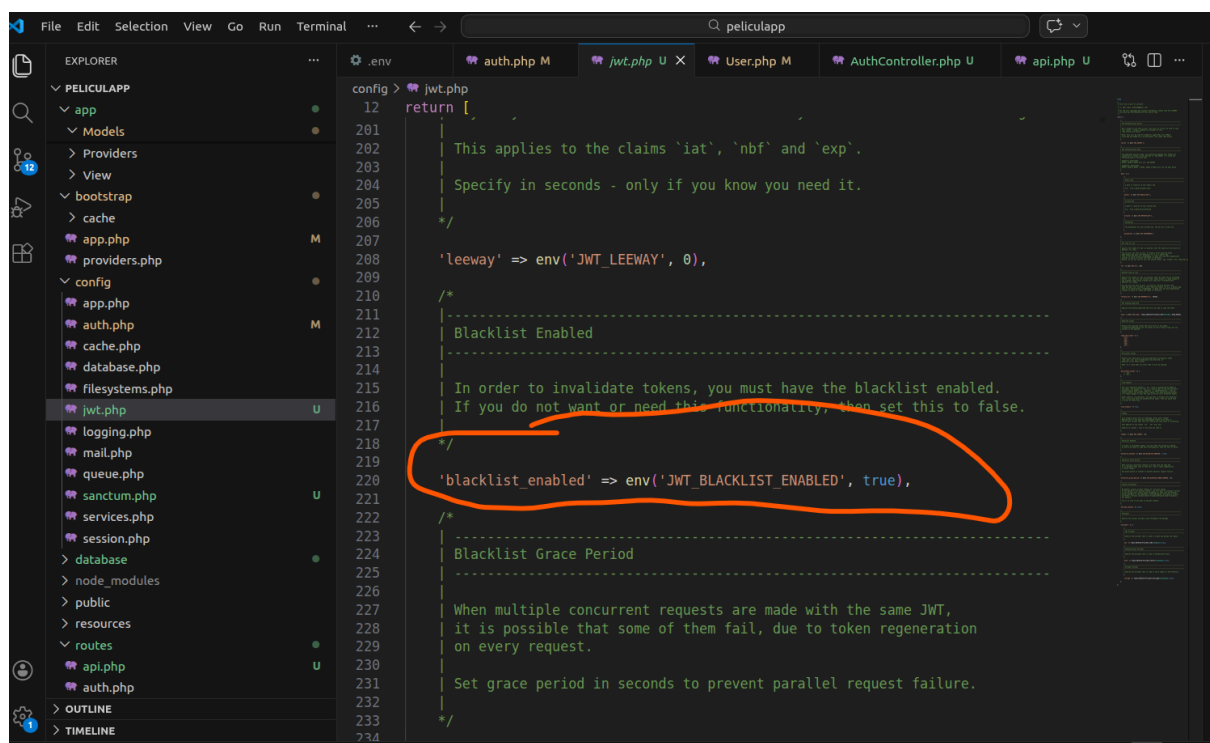


```
42
43 MEMCACHED_HOST=127.0.0.1
44
45 REDIS_CLIENT=phpredis
46 REDIS_HOST=127.0.0.1
47 REDIS_PASSWORD=null
48 REDIS_PORT=6379
49
50 MAIL_MAILER=log
51 MAIL_SCHEME=null
52 MAIL_HOST=127.0.0.1
53 MAIL_PORT=2525
54 MAIL_USERNAME=null
55 MAIL_PASSWORD=null
56 MAIL_FROM_ADDRESS="hello@example.com"
57 MAIL_FROM_NAME="{APP_NAME}"
58
59 AWS_ACCESS_KEY_ID=
60 AWS_SECRET_ACCESS_KEY=
61 AWS_DEFAULT_REGION=us-east-1
62 AWS_BUCKET=
63 AWS_USE_PATH_STYLE_ENDPOINT=false
64
65 VITE_APP_NAME="{APP_NAME}"
66
67 JWT_SECRET=LMZaIcveHZcMeW20512n9BftFe9GaXbWT5grEIL2hBOMfN08Lkp5NI45ey4tuhR
68 JWT_TTL=60
```

Esto significa que tras 60 minutos, el token expira y se tendrá que usar /refresh.

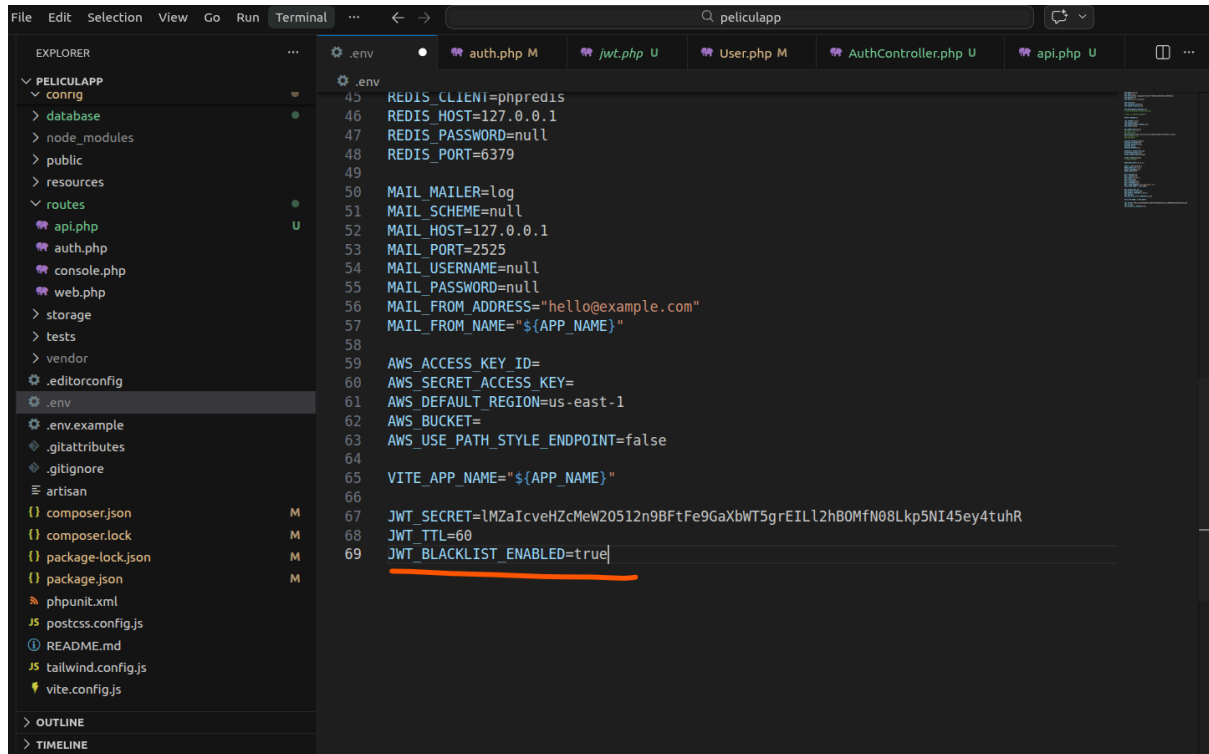
También se comprueba en config/jwt.php que esté:

'blacklist_enabled' => env('JWT_BLACKLIST_ENABLED', true),



```
12 return [
13     | This applies to the claims 'iat', 'nbf' and 'exp'.
14     | Specify in seconds - only if you know you need it.
15     | */
16     'leeway' => env('JWT_LEEWAY', 0),
17     |
18     | -----
19     | Blacklist Enabled
20     | -----
21     |
22     | In order to invalidate tokens, you must have the blacklist enabled.
23     | If you do not want or need this functionality, then set this to false.
24     |
25     | */
26     'blacklist_enabled' => env('JWT_BLACKLIST_ENABLED', true),
27     |
28     | -----
29     | Blacklist Grace Period
30     | -----
31     |
32     | When multiple concurrent requests are made with the same JWT,
33     | it is possible that some of them fail, due to token regeneration
34     | on every request.
35     |
36     | Set grace period in seconds to prevent parallel request failure.
37     |
38     | */
39 ]
```

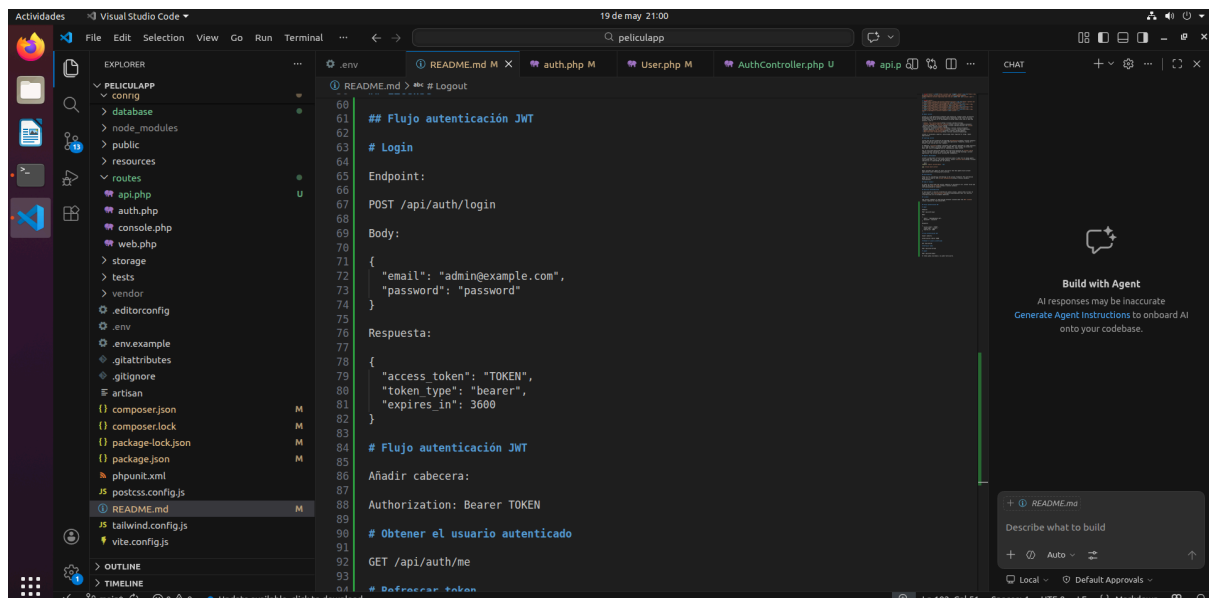
Es necesario porque el test de logout necesita invalidar tokens. Posteriormente en .env indicamos: **JWT_BLACKLIST_ENABLED=true**



The screenshot shows the Visual Studio Code editor with the `.env` file open. The file contains various environment variables for a Laravel application. The line `JWT_BLACKLIST_ENABLED=true` is highlighted with an orange underline. The Explorer sidebar on the left shows the project structure, including `config`, `database`, `node_modules`, `public`, `resources`, `routes`, `api.php`, `auth.php`, `console.php`, `web.php`, `storage`, `tests`, `vendor`, `.editorconfig`, `.env`, `.env.example`, `.gitattributes`, `.gitignore`, `artisan`, `composer.json`, `composer.lock`, `package-lock.json`, `package.json`, `phpunit.xml`, `postcss.config.js`, `README.md`, `tailwind.config.js`, and `vite.config.js`. The Terminal panel at the bottom is empty.

```
.env
45 REDIS_CLIENT=phpredis
46 REDIS_HOST=127.0.0.1
47 REDIS_PASSWORD=null
48 REDIS_PORT=6379
49
50 MAIL_MAILER=log
51 MAIL_SCHEME=null
52 MAIL_HOST=127.0.0.1
53 MAIL_PORT=2525
54 MAIL_USERNAME=null
55 MAIL_PASSWORD=null
56 MAIL_FROM_ADDRESS="hello@example.com"
57 MAIL_FROM_NAME="${APP_NAME}"
58
59 AWS_ACCESS_KEY_ID=
60 AWS_SECRET_ACCESS_KEY=
61 AWS_DEFAULT_REGION=us-east-1
62 AWS_BUCKET=
63 AWS_USE_PATH_STYLE_ENDPOINT=false
64
65 VITE_APP_NAME="${APP_NAME}"
66
67 JWT_SECRET=lmZaIcveHZcMeW20512n9BftFe9GaXbWT5grEILl2hB0MfN08Lkp5NI45ey4tuhR
68 JWT_TTL=60
69 JWT_BLACKLIST_ENABLED=true
```

Se documenta en el README.md el flujo completo: obtención del token → uso en cabeceras → refresco → logout.



The screenshot shows the Visual Studio Code editor with the `README.md` file open. The file contains documentation for the JWT authentication flow. The documentation is written in Spanish and includes sections for Login, Logout, and Refresh Token. The Explorer sidebar on the left shows the project structure, including `config`, `database`, `node_modules`, `public`, `resources`, `routes`, `api.php`, `auth.php`, `console.php`, `web.php`, `storage`, `tests`, `vendor`, `.editorconfig`, `.env`, `.env.example`, `.gitattributes`, `.gitignore`, `artisan`, `composer.json`, `composer.lock`, `package-lock.json`, `package.json`, `phpunit.xml`, `postcss.config.js`, `README.md`, `tailwind.config.js`, and `vite.config.js`. The Terminal panel at the bottom is empty.

```
60 ## Flujo autenticación JWT
61 # Login
62
63 Endpoint:
64 POST /api/auth/login
65
66 Body:
67 {
68   "email": "admin@example.com",
69   "password": "password"
70 }
71
72 Respuesta:
73 {
74   "access token": "TOKEN",
75   "token_type": "bearer",
76   "expires_in": 3600
77 }
78
79 # Flujo autenticación JWT
80
81 Añadir cabecera:
82 Authorization: Bearer TOKEN
83
84 # Obtener el usuario autenticado
85
86 GET /api/auth/me
87
88 # Refrescar token
```

A continuación agregamos unos ejemplos con curl en el [readme.md](#)

Login:

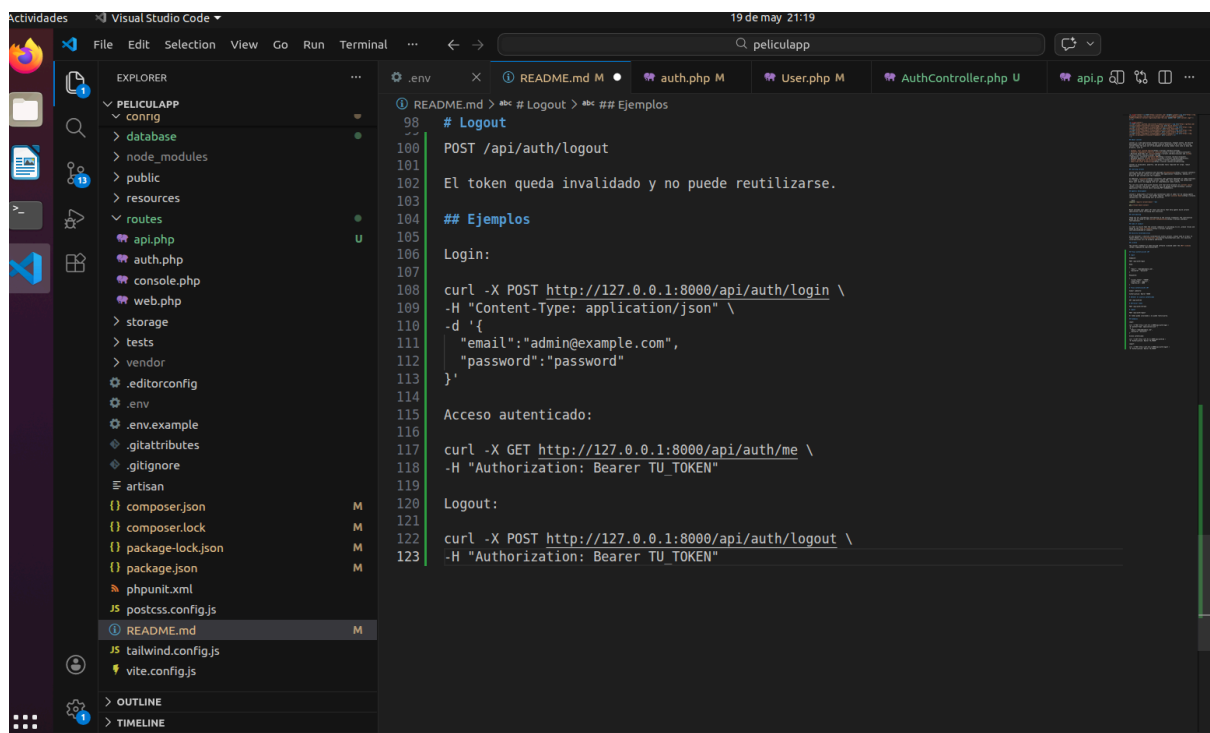
```
curl -X POST http://127.0.0.1:8000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email":"admin@example.com",
  "password":"password"
}'
```

Acceso autenticado:

```
curl -X GET http://127.0.0.1:8000/api/auth/me \
-H "Authorization: Bearer TU_TOKEN"
```

Logout:

```
curl -X POST http://127.0.0.1:8000/api/auth/logout \
-H "Authorization: Bearer TU_TOKEN"
```



2.5.- Pruebas JWT

Creamos un usuario para las pruebas. Para ello abrimos un terminal ejecutamos:
php artisan tinker

```

usuario@MV-Ubuntu:~/peliculapp$ php artisan tinker
Psy Shell v0.12.22 (PHP 8.4.1 - cli) by Justin Hileman
New PHP manual is available (latest: 3.0.6). Update with `doc --update-manual`

>

```

Después escribimos:

```

\App\Models\User::create([
    'name' => 'Admin',
    'email' => 'admin@example.com',
    'password' => bcrypt('password')
]);

```

Luego salimos.

```

usuario@MV-Ubuntu:~/peliculapp$ php artisan tinker
Psy Shell v0.12.22 (PHP 8.4.1 - cli) by Justin Hileman
New PHP manual is available (latest: 3.0.6). Update with `doc --update-manual`

> \App\Models\User::create([
.   'name' => 'Admin',
.   'email' => 'admin@example.com',
.   'password' => bcrypt('password')
. ]);

= App\Models\User {#7994
  name: "Admin",
  email: "admin@example.com",
  #password: "\$2y\$12\$td2xKdix8kUzSI01L0e6we2p4nD6uFFl/HMgeYPN3ZUp7SO/0scmm",
  updated_at: "2026-05-19 19:47:16",
  created_at: "2026-05-19 19:47:16",
  id: 1,
}

> exit

[INFO] Goodbye.
usuario@MV-Ubuntu:~/peliculapp$

```

2.5.1.- Prueba login

Usamos thunder client para las pruebas JWT. Empezamos primero con el login.

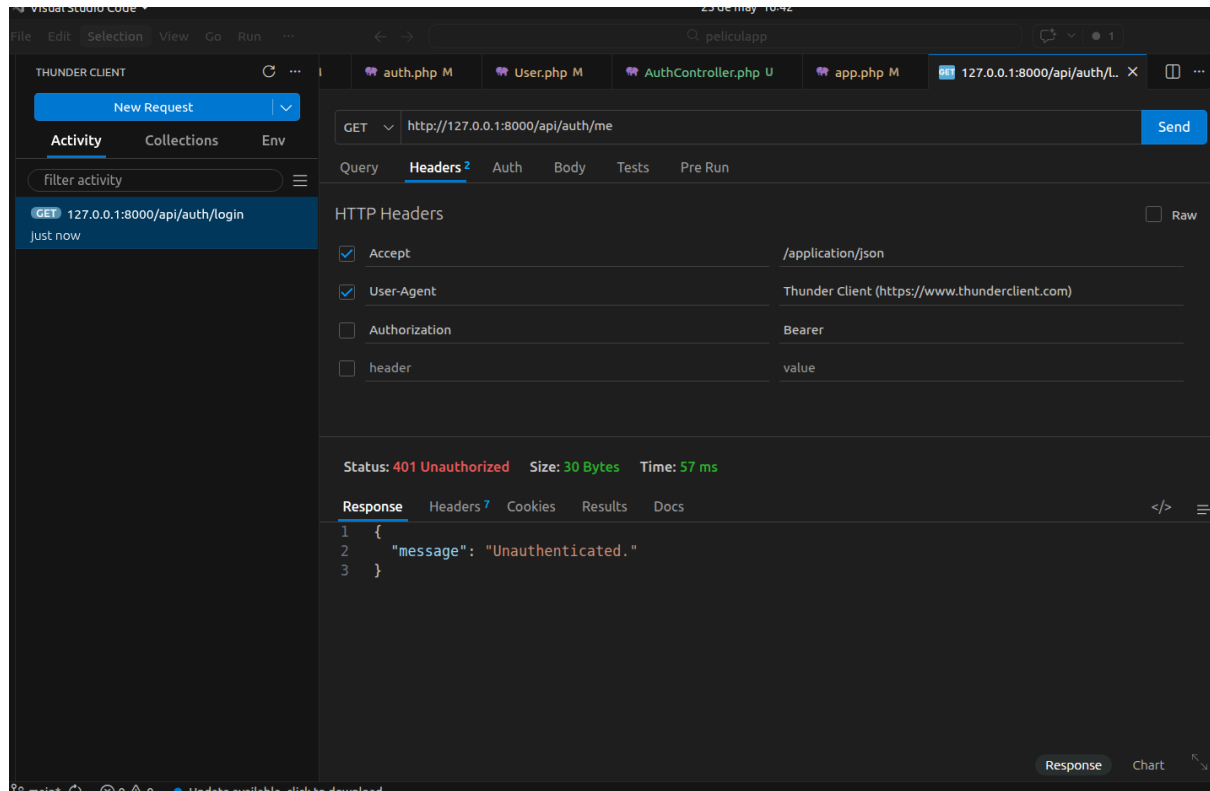
Se realiza un método post a <http://127.0.0.1:8000/api/auth/login>, en body seleccionamos json y ponemos {
 "email": "admin@example.com",
 "password": "password"
}

Al hacer la petición nos da un token jwt que lo usaremos para hacer peticiones a la api a través de las rutas protegidas.

2.5.3.- Prueba sin token

Realizamos una petición a <http://127.0.0.1:8000/api/auth/me> y quitamos dicho header.

Tras ejecutar la petición tiene que arrojar un error.



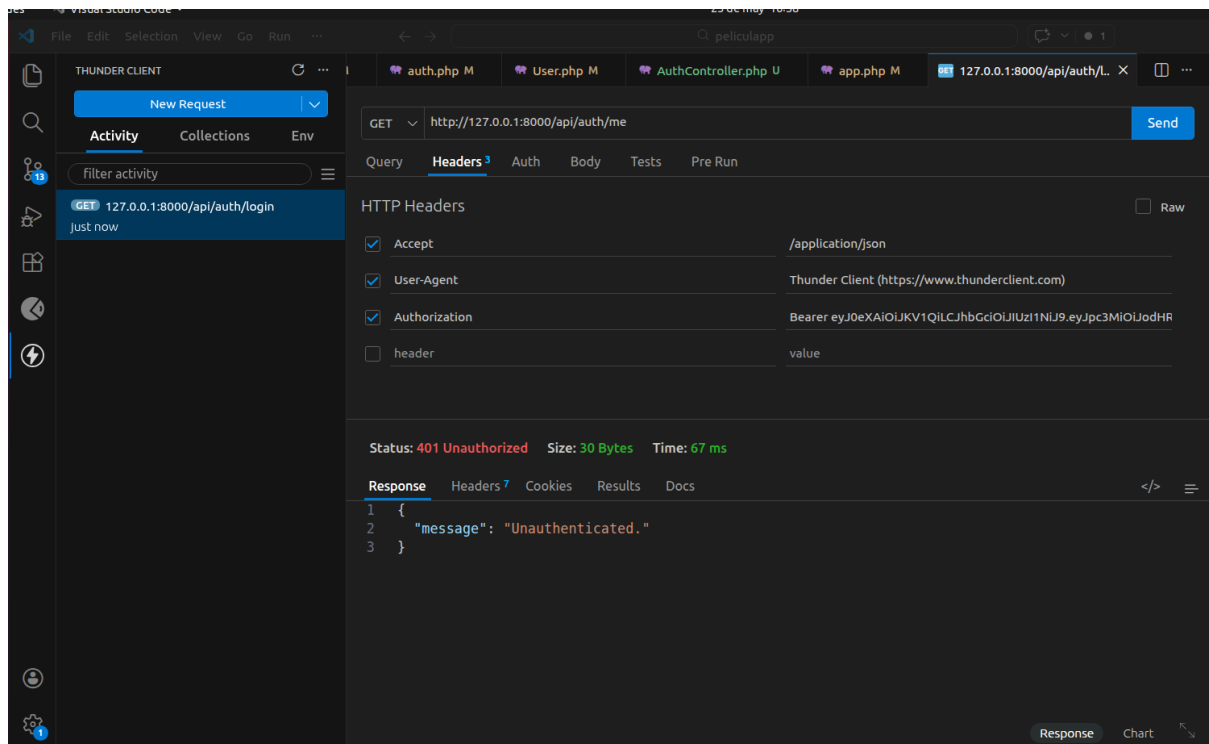
2.5.4.- Refresco de token

Realizamos una petición post a <http://127.0.0.1:8000/api/auth/refresh> y en headers establecemos Key: Authorization y Value: Bearer TU_TOKEN

Tras ejecutar la petición esta se completa devolviendo el nuevo token solicitado.

2.5.6.- Reutilizar el token

Si realizamos la petición a `/api/auth/me` con el token esto arroja un error.

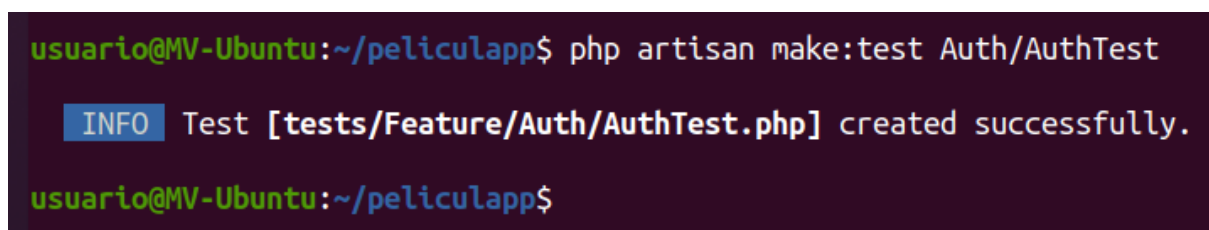


2.6.- Test de integración

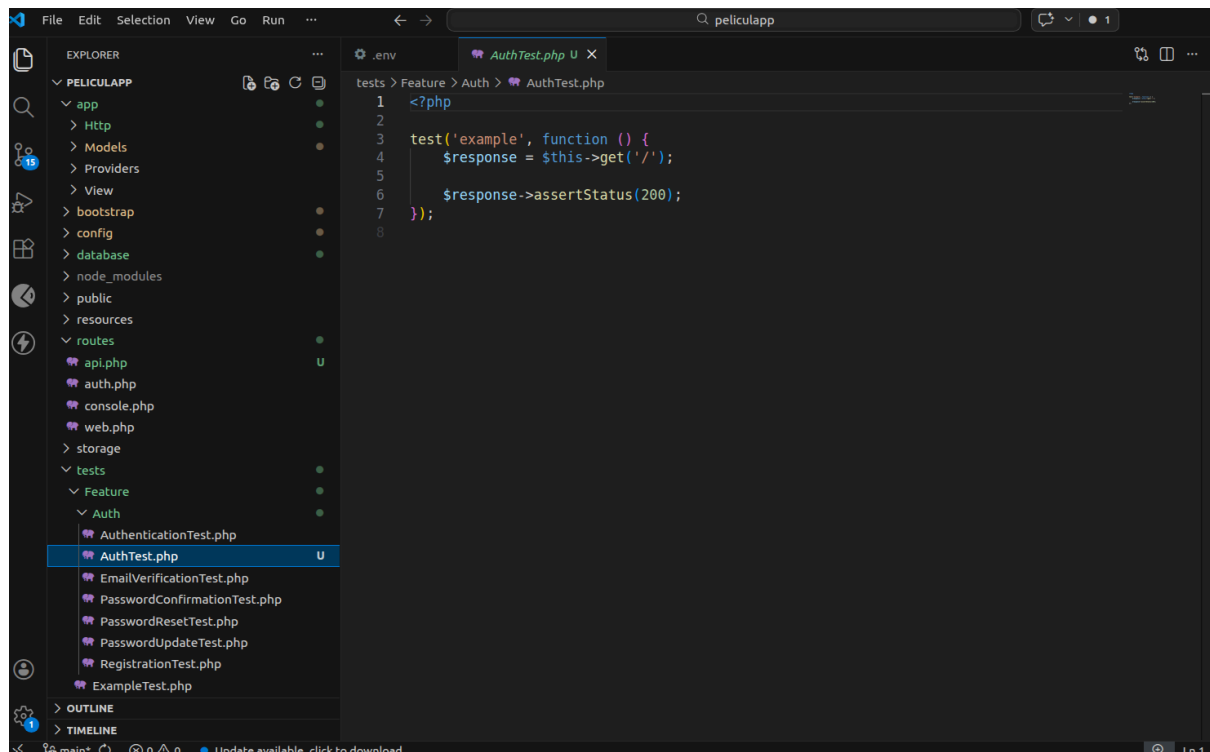
Los test unitarios se distribuyen en 3 archivos: AuthJWT, Películas CRUD y Directores CRUD. Estos archivos se van probando, para verificar que los test funcionen y su depuración sea simple.

2.6.1.- Tests Auth

Para crear el archivo, se ejecuta el comando **php artisan make:test Auth/AuthTest**



Se genera un archivo `AuthTest.php` donde se implementan las pruebas unitarias de autenticación.



Una vez obtenido el archivo para los test, los implementamos y quedarían de esta forma:

```
<?php
```

```
namespace Tests\Feature\Auth;
```

```
use App\Models\User;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Tests\TestCase;
```

```
class AuthTest extends TestCase
```

```
{
```

```
    use RefreshDatabase;
```

```
    /**
```

```
     * Login correcto
```

```
     */
```

```
    public function test_user_can_login_with_valid_credentials(): void
```

```
    {
```

```
        User::factory()->create([
            'email' => 'admin@example.com',
            'password' => bcrypt('password')
        ]);
```

```

    $response = $this->postJson('/api/auth/login', [
        'email' => 'admin@example.com',
        'password' => 'password'
    ]);

    $response
        ->assertStatus(200)
        ->assertJsonStructure([
            'access_token',
            'token_type',
            'expires_in'
        ]);
}

/**
 * Login incorrecto
 */
public function test_user_cannot_login_with_invalid_credentials(): void
{
    User::factory()->create([
        'email' => 'admin@example.com',
        'password' => bcrypt('password')
    ]);

    $response = $this->postJson('/api/auth/login', [
        'email' => 'admin@example.com',
        'password' => 'incorrecta'
    ]);

    $response->assertStatus(401);
}

/**
 * Login sin email
 */
public function test_login_requires_email(): void
{
    $response = $this->postJson('/api/auth/login', [
        'password' => 'password'
    ]);

    $response
        ->assertStatus(422)
        ->assertJsonValidationErrors('email');
}

```

```

}

/**
 * Login sin password
 */
public function test_login_requires_password(): void
{
    $response = $this->postJson('/api/auth/login', [
        'email' => 'admin@example.com'
    ]);

    $response
        ->assertStatus(422)
        ->assertJsonValidationErrors('password');
}

/**
 * /me con token válido
 */
public function test_authenticated_user_can_access_me_endpoint(): void
{
    $user = User::factory()->create();

    $token = auth('api')->login($user);

    $response = $this->withHeader(
        'Authorization',
        'Bearer ' . $token
    )->getJson('/api/auth/me');

    $response
        ->assertStatus(200)
        ->assertJsonFragment([
            'email' => $user->email
        ]);
}

/**
 * /me sin token
 */
public function test_me_endpoint_requires_token(): void
{
    $response = $this->getJson('/api/auth/me');
}

```

```

    $response->assertStatus(401);
}

/**
 * Refresh válido
 */
public function test_user_can_refresh_token(): void
{
    $user = User::factory()->create();

    $token = auth('api')->login($user);

    $response = $this->withHeader(
        'Authorization',
        'Bearer ' . $token
    )->postJson('/api/auth/refresh');

    $response
        ->assertStatus(200)
        ->assertJsonStructure([
            'access_token',
            'token_type',
            'expires_in'
        ]);
}

/**
 * Logout invalida token
 */
public function test_logout_invalidates_token(): void
{
    $user = User::factory()->create();

    $token = auth('api')->login($user);

    // Logout
    $this->withHeader(
        'Authorization',
        'Bearer ' . $token
    )->postJson('/api/auth/logout');

    // Reutilizar token
    $response = $this->withHeader(
        'Authorization',

```



```

        'Bearer ' . $token
    )->getJson('/api/auth/me');

    $response->assertStatus(401);
}
}

```

Este código contiene los test unitarios de Auth.

```

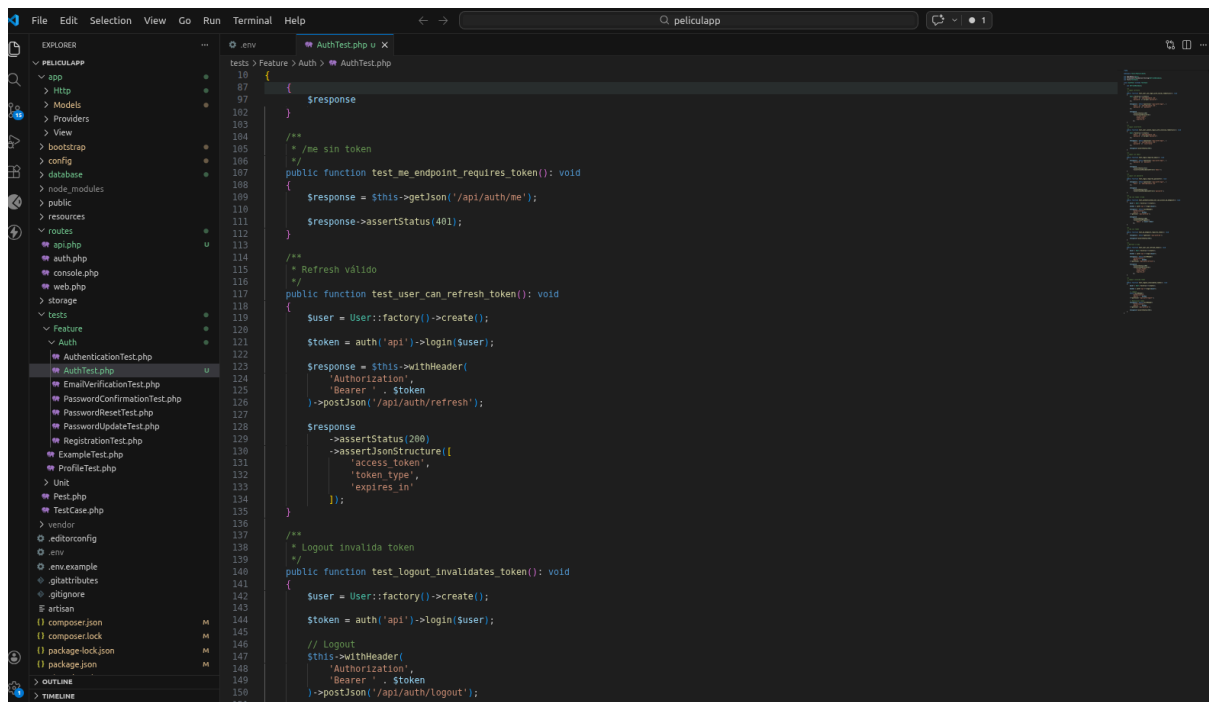
tests/Feature/Auth/AuthTest.php
1 <?php
2
3 namespace Tests\Feature\Auth;
4
5 use App\Models\User;
6 use Illuminate\Foundation\Testing\RefreshDatabase;
7 use Tests\TestCase;
8
9 class AuthTest extends TestCase
10 {
11     use RefreshDatabase;
12
13     /**
14      * Login correcto
15      */
16     public function test_user_can_login_with_valid_credentials(): void
17     {
18         User::factory()->create([
19             'email' => 'admin@example.com',
20             'password' => bcrypt('password')
21         ]);
22
23         $response = $this->postJson('/api/auth/login', [
24             'email' => 'admin@example.com',
25             'password' => 'password'
26         ]);
27
28         $response
29             ->assertStatus(200)
30             ->assertJsonStructure([
31                 'access_token',
32                 'token_type',
33                 'expires_in'
34             ]);
35     }
36
37     /**
38      * Login incorrecto
39      */
40     public function test_user_cannot_login_with_invalid_credentials(): void
41     {
42         User::factory()->create([
43             'email' => 'admin@example.com',
44             'password' => bcrypt('password')
45         ]);
46
47         $response = $this->postJson('/api/auth/login', [
48             'email' => 'admin@example.com',
49             'password' => 'incorrecta'
50         ]);
51
52         $response->assertStatus(401);
53     }
54 }

```

```

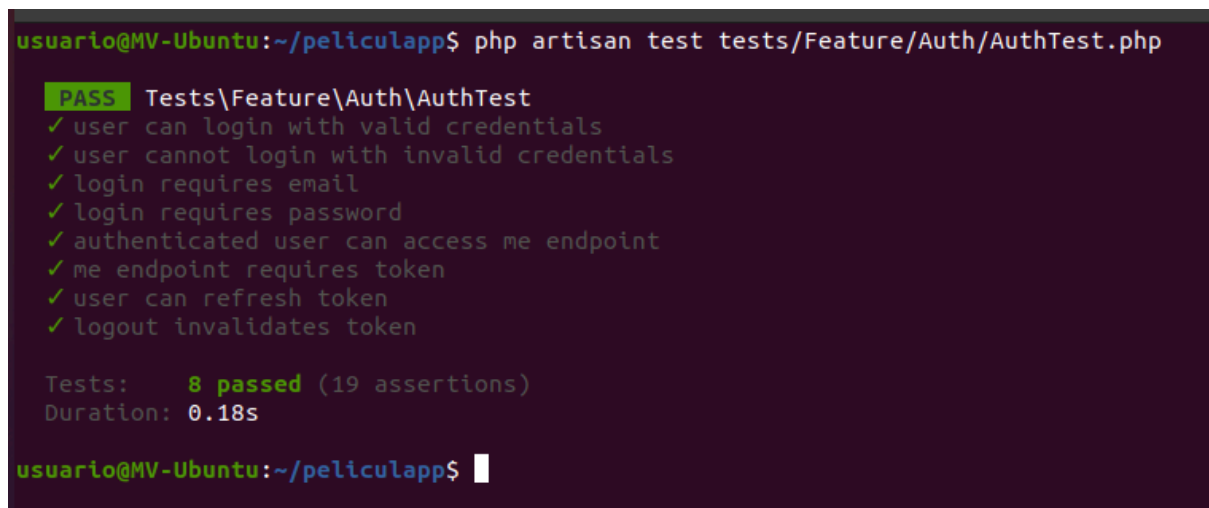
tests/Feature/Auth/AuthTest.php
54 {
55 }
56
57 /**
58  * Login sin email
59  */
60 public function test_login_requires_email(): void
61 {
62     $response = $this->postJson('/api/auth/login', [
63         'password' => 'password'
64     ]);
65
66     $response
67         ->assertStatus(422)
68         ->assertJsonValidationErrors('email');
69 }
70
71 /**
72  * Login sin password
73  */
74 public function test_login_requires_password(): void
75 {
76     $response = $this->postJson('/api/auth/login', [
77         'email' => 'admin@example.com'
78     ]);
79
80     $response
81         ->assertStatus(422)
82         ->assertJsonValidationErrors('password');
83 }
84
85 /**
86  * Me con token válido
87  */
88 public function test_authenticated_user_can_access_me_endpoint(): void
89 {
90     $user = User::factory()->create();
91
92     $token = auth('api')->login($user);
93
94     $response = $this->withHeader(
95         'Authorization',
96         'Bearer ' . $token
97     )->getJson('/api/auth/me');
98
99     $response
100         ->assertStatus(200)
101         ->assertJsonFragment([
102             'email' => $user->email
103         ]);
104 }
105 }

```



Una vez implementados los tests, los ejecutamos con el comando **php artisan test tests/Feature/Auth/AuthTest.php**

En la captura se muestra que los test han pasado con éxito.



2.6.2.- Tests Directores

Creamos Director Factory ya que es necesario para los test. Para crearlo, usamos el comando **php artisan make:factory DirectorFactory --model=Director**

```

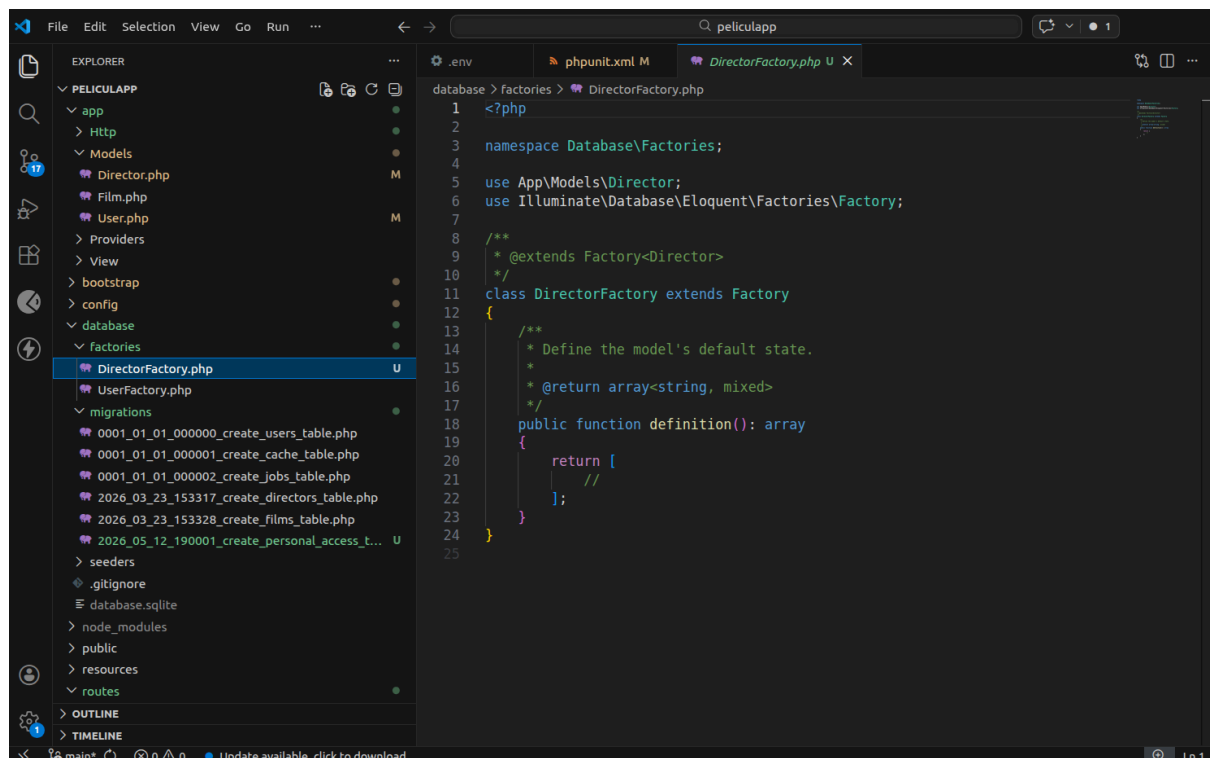
usuario@MV-Ubuntu:~/peliculapp$ php artisan make:factory DirectorFactory --model=Director

INFO Factory [database/factories/DirectorFactory.php] created successfully.

usuario@MV-Ubuntu:~/peliculapp$

```

Y se nos genera DirectorFactory.php en database/factories



En el archivo se pone:

```
<?php
```

```
namespace Database\Factories;
```

```
use Illuminate\Database\Eloquent\Factories\Factory;
```

```
class DirectorFactory extends Factory
```

```
{
```

```
    public function definition(): array
```

```
    {
```

```
        return [
```

```
            'name' => fake()->firstName(),
```

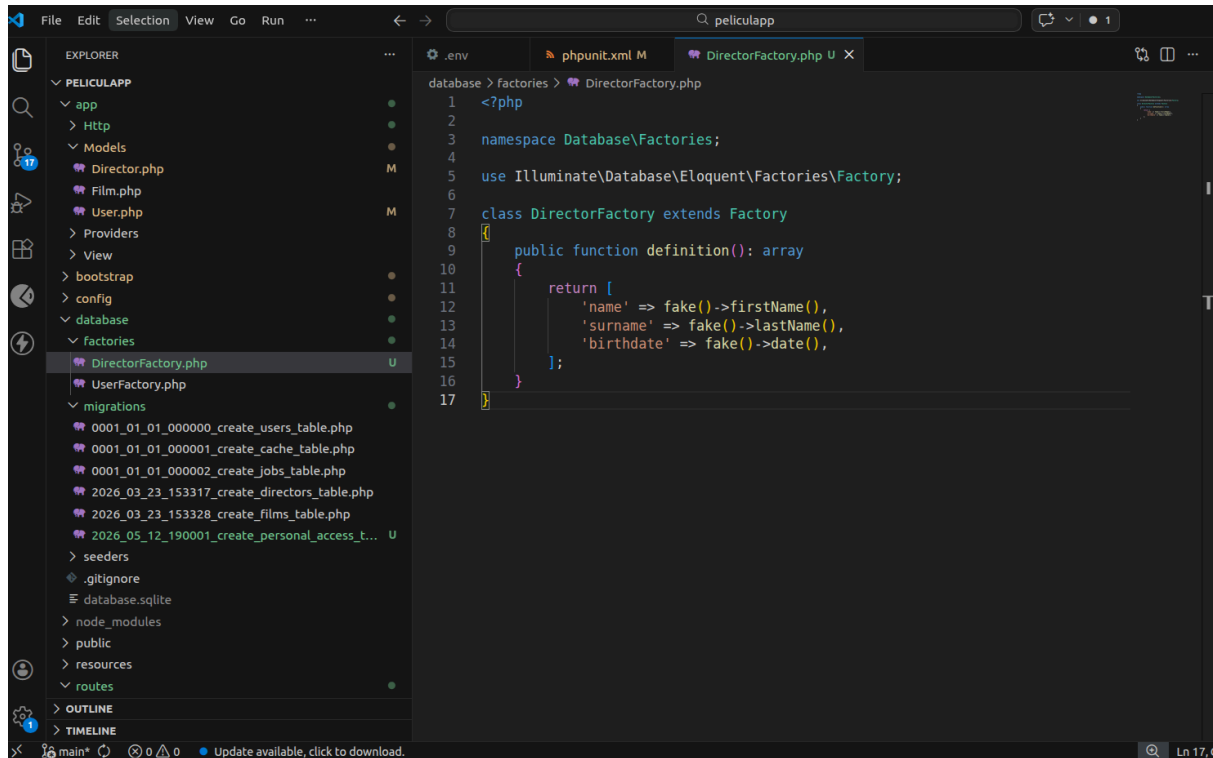
```
            'surname' => fake()->lastName(),
```

```
            'birthdate' => fake()->date(),
```

```
        ];
```

```
}  
}
```

Este quedaría de la siguiente manera:

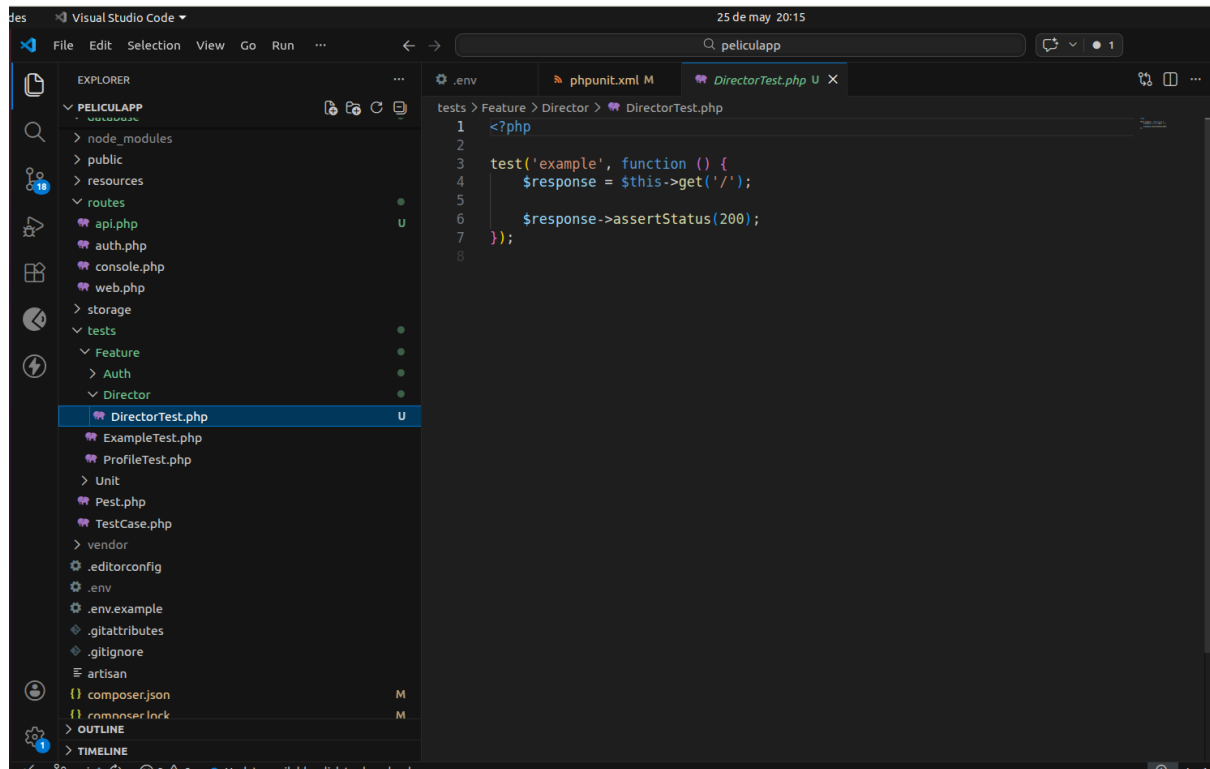


Tras implementar el factory, podemos crear los tests.

Para crear el archivo, se ejecuta el comando **php artisan make:test Director/DirectorTest**

```
usuario@MV-Ubuntu:~/peliculapp$ php artisan make:test Director/DirectorTest  
  
INFO Test [tests/Feature/Director/DirectorTest.php] created successfully  
  
usuario@MV-Ubuntu:~/peliculapp$
```

Se genera un archivo DirectorTest.php donde se implementan las pruebas unitarias de los directores.



Una vez obtenido el archivo para los test, los implementamos y quedarían de esta forma:

```
<?php
```

```
namespace Tests\Feature\Director;
```

```
use App\Models\User;
```

```
use App\Models\Director;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Tests\TestCase;
```

```
class DirectorTest extends TestCase
```

```
{
```

```
    use RefreshDatabase;
```

```
    protected function authenticate()
```

```
    {
```

```
        $user = User::factory()->create();
```

```
        return auth('api')->login($user);
```

```
    }
```

```
/**
```

```

* Listar directores
*/
public function test_authenticated_user_can_list_directors(): void
{
    Director::factory()->count(3)->create();

    $token = $this->authenticate();

    $response = $this->withHeader(
        'Authorization',
        'Bearer '.$token
    )->getJson('/api/auth/directors');

    $response->assertStatus(200);
}

/**
 * Crear director
 */
public function test_authenticated_user_can_create_director(): void
{
    $token = $this->authenticate();

    $data = [
        'name' => 'Christopher',
        'surname' => 'Nolan',
        'birthdate' => '1970-07-30'
    ];

    $response = $this->withHeader(
        'Authorization',
        'Bearer '.$token
    )->postJson('/api/auth/directors', $data);

    $response->assertStatus(201);

    $this->assertDatabaseHas('directors', [
        'name' => 'Christopher'
    ]);
}

/**
 * Crear director inválido
 */

```

```
public function test_create_director_requires_valid_data(): void
{
    $token = $this->authenticate();

    $response = $this->withHeader(
        'Authorization',
        'Bearer '.$token
    )->postJson('/api/auth/directors', []);

    $response->assertStatus(422);
}
```

```
/**
 * Ver director
 */
public function test_authenticated_user_can_view_director(): void
{
    $director = Director::factory()->create();

    $token = $this->authenticate();

    $response = $this->withHeader(
        'Authorization',
        'Bearer '.$token
    )->getJson('/api/auth/directors/'.$director->id);

    $response
        ->assertStatus(200)
        ->assertJsonFragment([
            'id' => $director->id
        ]);
}
```

```
/**
 * Actualizar director
 */
public function test_authenticated_user_can_update_director(): void
{
    $director = Director::factory()->create();

    $token = $this->authenticate();

    $response = $this->withHeader(
        'Authorization',
```

```

        'Bearer '.$token
    )->putJson('/api/auth/directors/'.$director->id, [
        'name' => 'Updated'
    ]);

    $response->assertStatus(200);

    $this->assertDatabaseHas('directors', [
        'id' => $director->id,
        'name' => 'Updated'
    ]);
}

/**
 * Borrar director
 */
public function test_authenticated_user_can_delete_director(): void
{
    $director = Director::factory()->create();

    $token = $this->authenticate();

    $response = $this->withHeader(
        'Authorization',
        'Bearer '.$token
    )->deleteJson('/api/auth/directors/'.$director->id);

    $response->assertStatus(200);

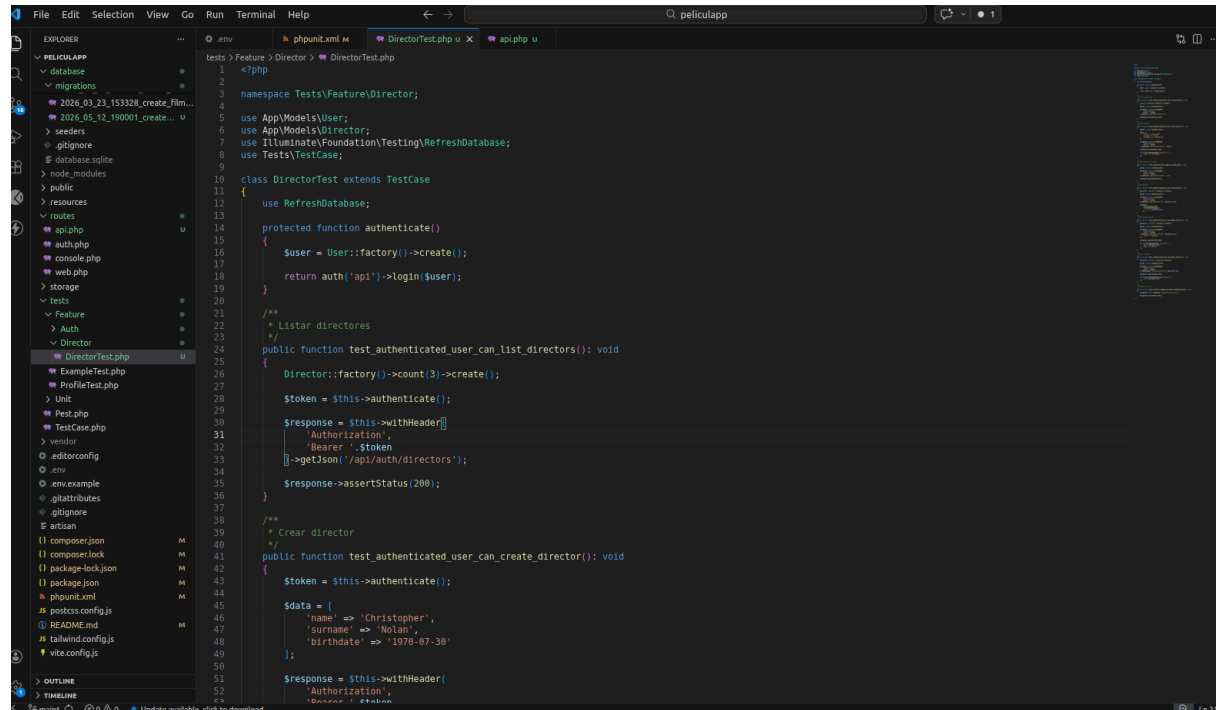
    $this->assertDatabaseMissing('directors', [
        'id' => $director->id
    ]);
}

/**
 * Acceso sin token
 */
public function test_directors_endpoints_require_authentication(): void
{
    $response = $this->getJson('/api/auth/directors');

    $response->assertStatus(401);
}
}

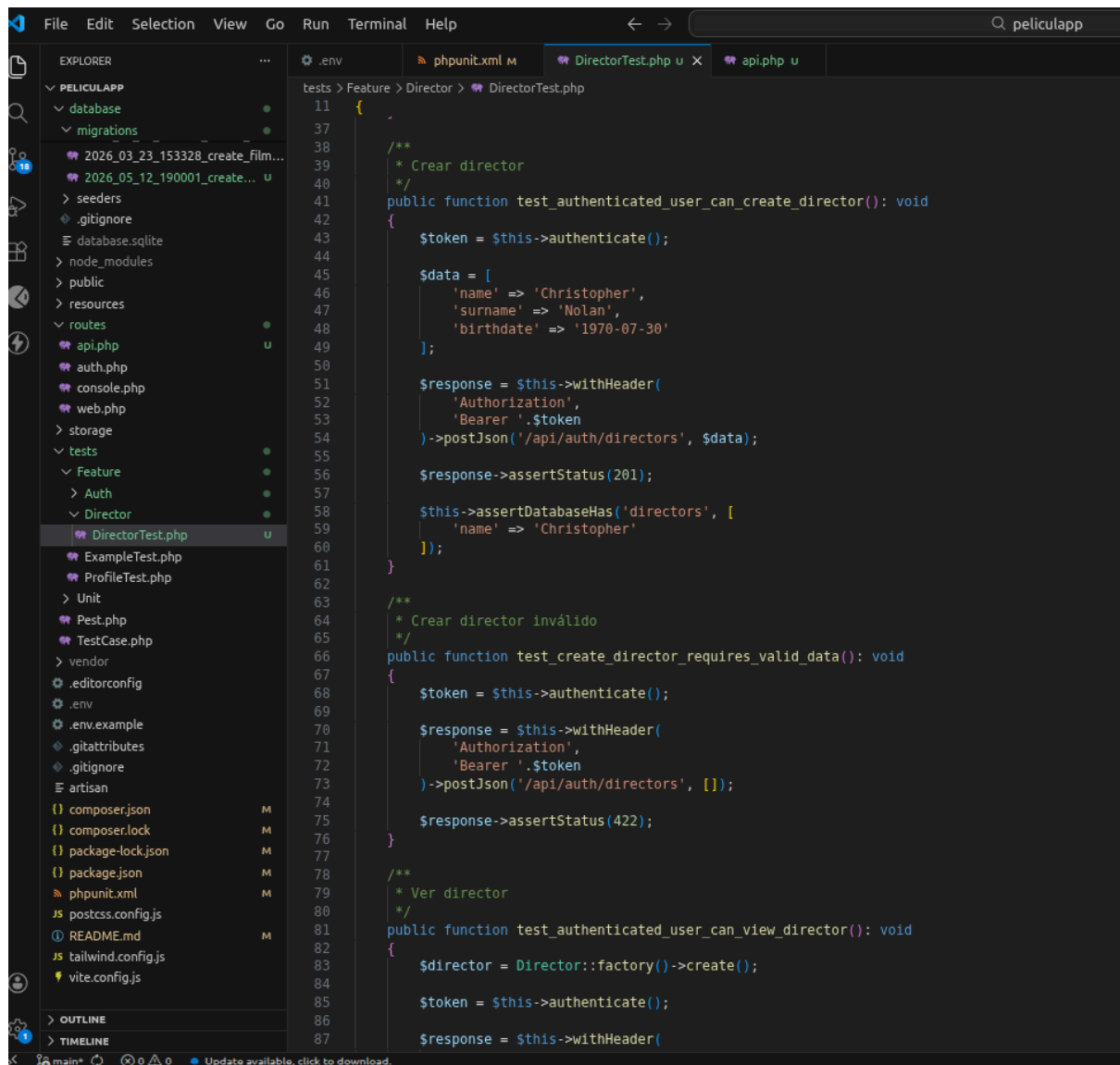
```

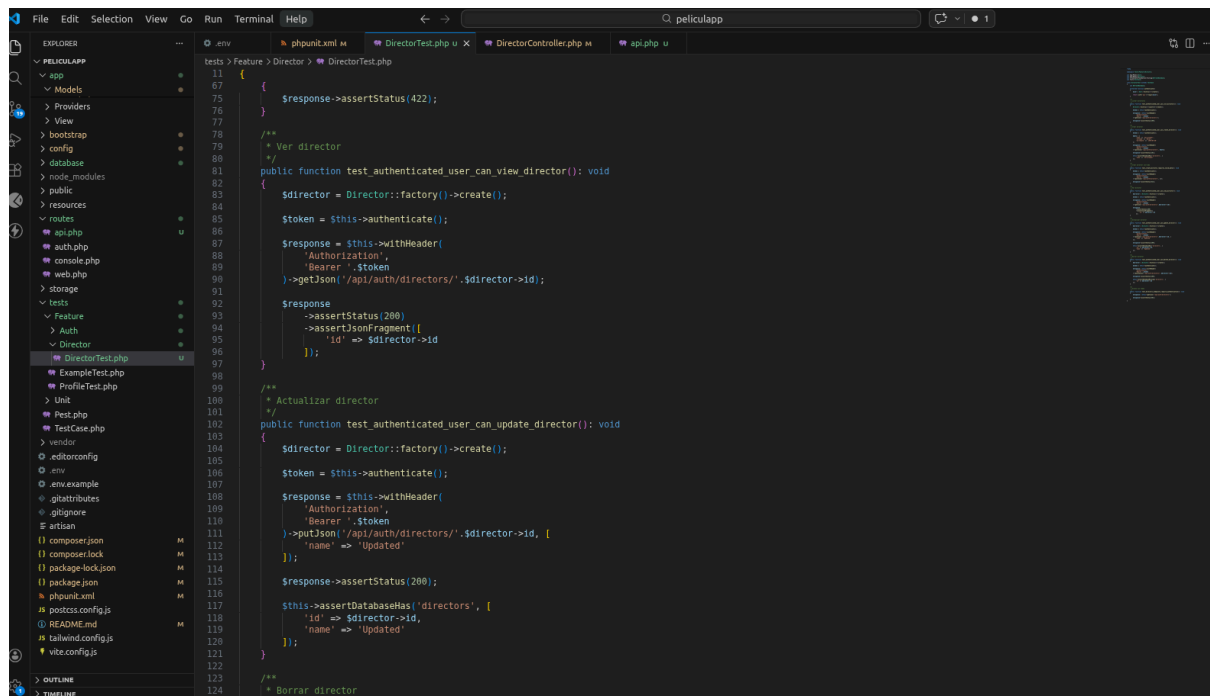

Este código contiene los test unitarios de Directores.



The image shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'database', 'migrations', 'seeds', 'resources', 'routes', 'storage', 'tests', and 'vendor'. The 'tests' folder is expanded, showing files like 'DirectorTest.php', 'ExampleTest.php', 'ProfileTest.php', 'Unit', 'Pest.php', and 'TestCase.php'. The 'DirectorTest.php' file is selected, and its content is displayed in the code editor. The code is a PHP unit test for the 'Director' feature, using PHPUnit and Laravel's testing framework. It includes a namespace declaration, use statements for 'App\Models\User', 'App\Models\Director', 'Illuminate\Foundation\Testing\RefreshDatabase', and 'Tests\TestCase'. The test class 'DirectorTest' extends 'TestCase' and uses 'RefreshDatabase'. It contains two test methods: 'test_authenticated_user_can_list_directors()' and 'test_authenticated_user_can_create_director()'. The first test method creates three directors, authenticates a user, and asserts that the user can list the directors. The second test method authenticates a user and asserts that the user can create a director with specific data.

```
1 <?php
2
3 namespace Tests\Feature\Director;
4
5 use App\Models\User;
6 use App\Models\Director;
7 use Illuminate\Foundation\Testing\RefreshDatabase;
8 use Tests\TestCase;
9
10
11 class DirectorTest extends TestCase
12 {
13     use RefreshDatabase;
14
15     protected function authenticate()
16     {
17         $user = User::factory()->create();
18         return auth('api')->login($user);
19     }
20
21     /**
22      * Listar directores
23      */
24     public function test_authenticated_user_can_list_directors(): void
25     {
26         Director::factory()->count(3)->create();
27
28         $token = $this->authenticate();
29
30         $response = $this->withHeader(
31             'Authorization',
32             'Bearer ' . $token
33         )->getJson('/api/auth/directors');
34
35         $response->assertStatus(200);
36     }
37
38     /**
39      * Crear director
40      */
41     public function test_authenticated_user_can_create_director(): void
42     {
43         $token = $this->authenticate();
44
45         $data = [
46             'name' => 'Christopher',
47             'suriname' => 'Nolan',
48             'birthdate' => '1978-07-30'
49         ];
50
51         $response = $this->withHeader(
52             'Authorization',
53             'Bearer ' . $token
54         )->postJson('/api/auth/directors', $data);
55
56         $response->assertStatus(201);
57     }
58 }
```





Una vez implementados los tests, los ejecutamos con el comando
php artisan test tests/Feature/Director/DirectorTest.php

En la captura se muestra que los test han pasado con éxito.

```

usuario@MV-Ubuntu:~/peliculapp$ php artisan test tests/Feature/Director/DirectorTest.php

PASS Tests\Feature\Director\DirectorTest
✓ authenticated user can list directors
✓ authenticated user can create director
✓ create director requires valid data
✓ authenticated user can view director
✓ authenticated user can update director
✓ authenticated user can delete director
✓ directors endpoints require authentication

Tests: 7 passed (11 assertions)
Duration: 0.17s

usuario@MV-Ubuntu:~/peliculapp$

```

2.6.3.- Tests Peliculas

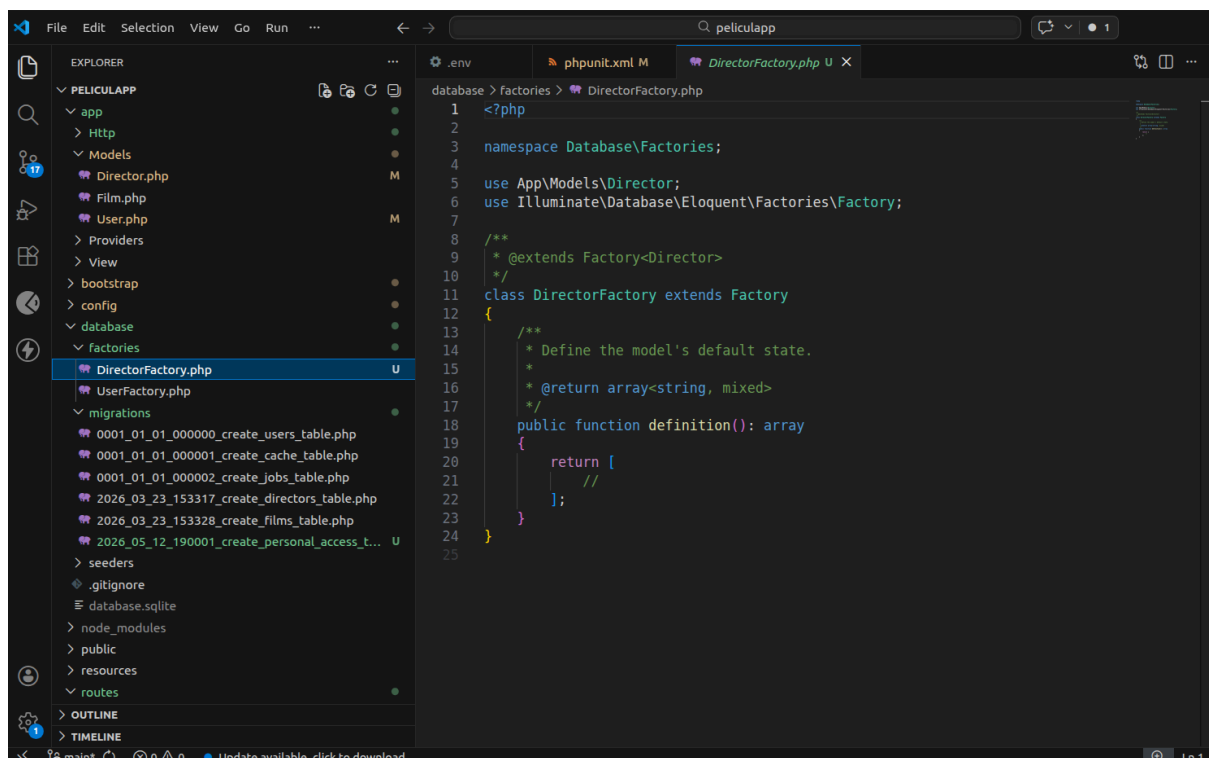
Creamos Movie Factory ya que es necesario para los test. Para crearlo, usamos el comando **php artisan make:factory DirectorFactory --model=Movie**

```

INFO Factory [database/factories/MovieFactory.php] created successfully.

```

Y se nos genera MovieFactory.php en database/factories



En el archivo se pone:

```
<?php
```

```
namespace Database\Factories;
```

```
use App\Models\Movie;
```

```
use Illuminate\Database\Eloquent\Factories\Factory;
```

```
/**
```

```
 * @extends Factory<Movie>
```

```
 */
```

```
class MovieFactory extends Factory
```

```
{
```

```
    /**
```

```
     * Define the model's default state.
```

```
     *
```

```
     * @return array<string, mixed>
```

```
     */
```

```
    public function definition(): array
```

```
    {
```

```
        return [
```

```
            'title' => fake()->sentence(2),
```

```
            'year' => fake()->year(),
```

```
            'duration' => fake()->numberBetween(80, 180),
```

```
            'genre' => fake()->randomElement([
```

```
                'Action',
```

```
                'Drama',
```

```
                'Comedy',
```

```
                'Sci-Fi'
```

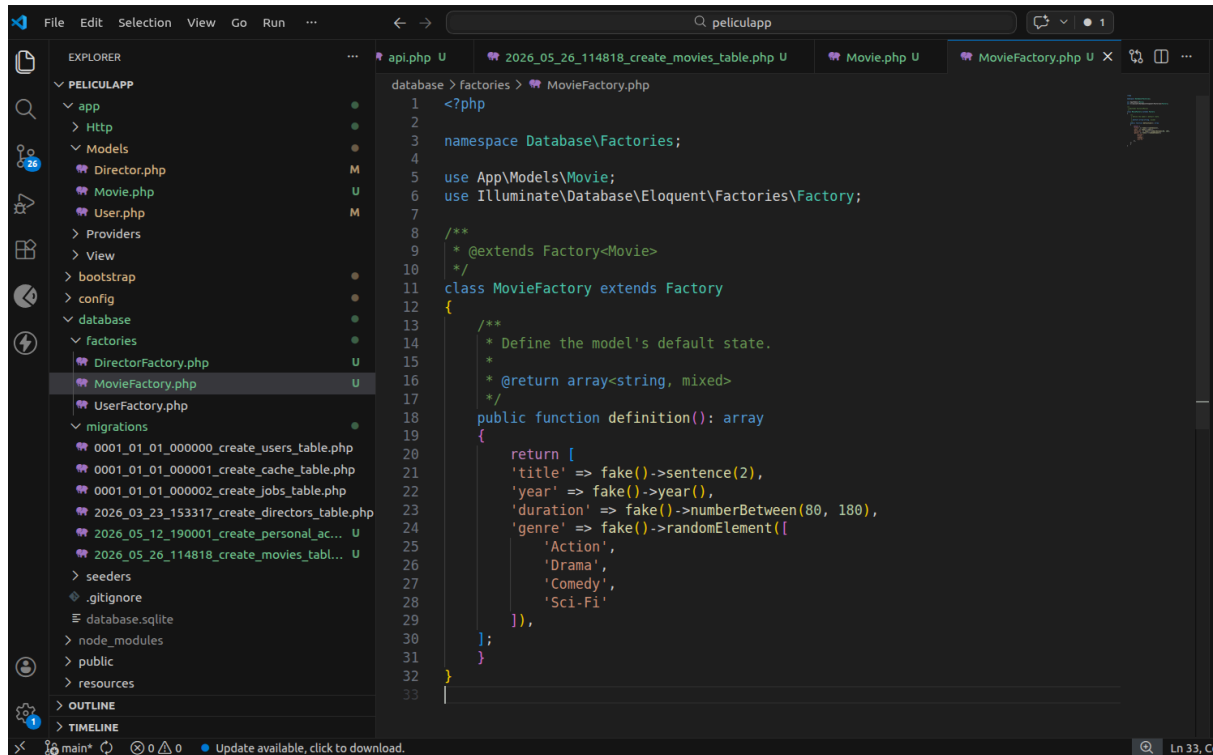
```
            ]),
```

```
        ];
```

```
    }
```

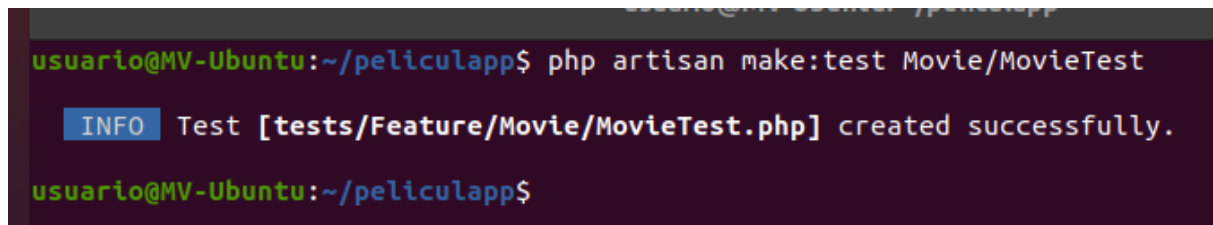
```
}
```

Este quedaría de la siguiente manera:

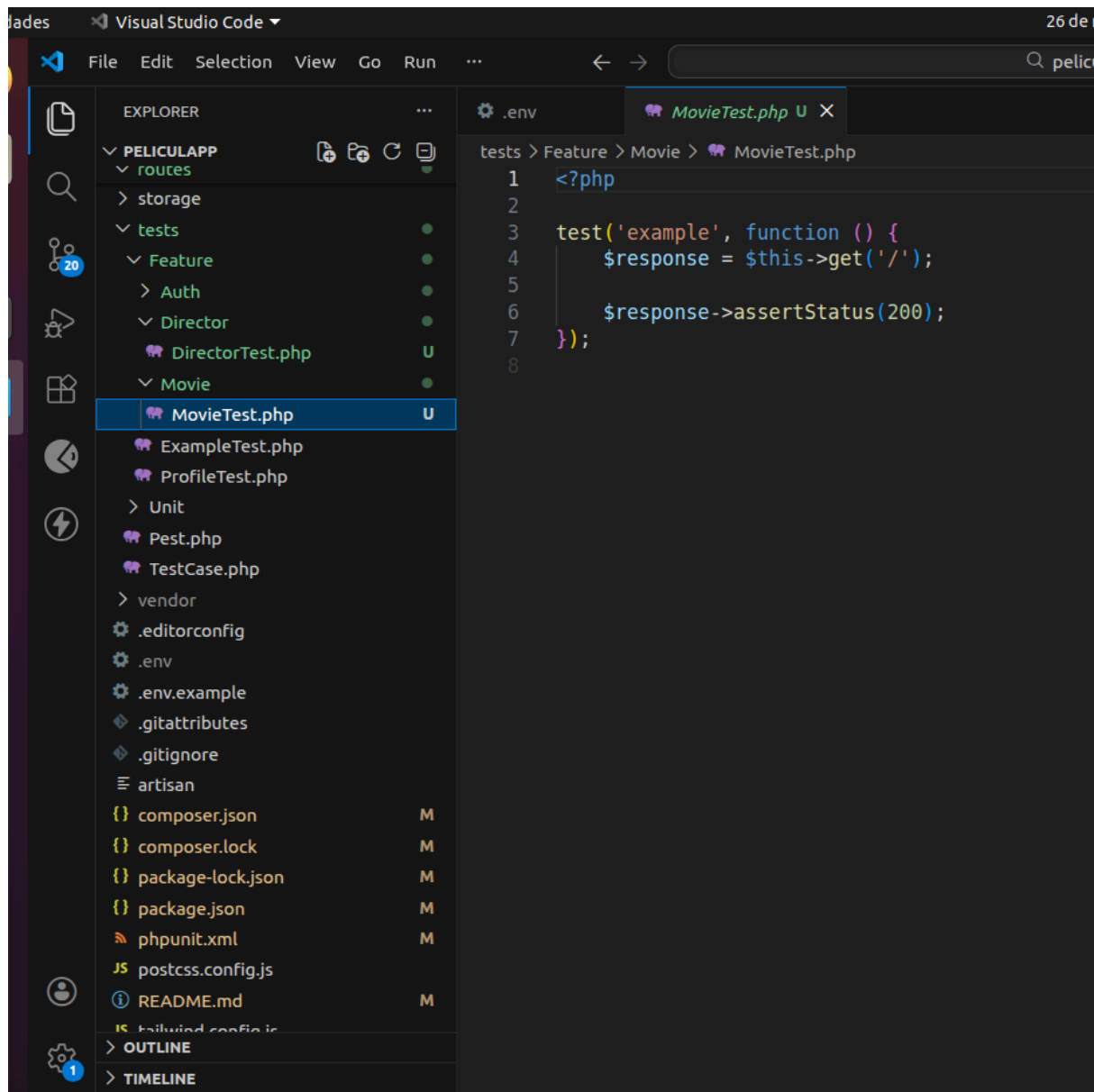


Tras implementar el factory, podemos crear los tests.

Para crear el archivo, se ejecuta el comando **php artisan make:test Movie/MovieTest**



Se genera un archivo `MovieTest.php` donde se implementan las pruebas unitarias de las películas.



Una vez obtenido el archivo para los test, los implementamos y quedarían de esta forma:

```
<?php
```

```
namespace Tests\Feature\Movie;
```

```
use App\Models\User;
```

```
use App\Models\Movie;
```

```
use App\Models\Director;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Tests\TestCase;
```

```
class MovieTest extends TestCase
```

```

{
    use RefreshDatabase;

    /**
     * Obtener token JWT
     */
    private function getToken(): string
    {
        $user = User::factory()->create();

        return auth('api')->login($user);
    }

    /**
     * Listar películas
     */
    public function test_authenticated_user_can_list_movies(): void
    {
        Movie::factory()->count(3)->create();

        $token = $this->getToken();

        $response = $this
            ->withHeader(
                'Authorization',
                'Bearer '.$token
            )->getJson('/api/auth/auth/movies');

        $response->assertStatus(200);
    }

    /**
     * Crear película
     */
    public function test_authenticated_user_can_create_movie(): void
    {
        $token = $this->getToken();

        $director = Director::factory()->create();

        $data = [
            'title' => 'Interstellar',
            'release_date' => '2014-11-07',
            'sinopsis' => 'Viaje espacial épico',
        ];
    }
}

```



```

        'duration' => 169,
        'genre' => 'Sci-Fi',
        'director_id' => $director->id
    ];

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->postJson('/api/auth/auth/movies', $data);

    $response->assertStatus(201);

    $this->assertDatabaseHas('movies', [
        'title' => 'Interstellar'
    ]);
}

/**
 * Validación al crear película
 */
public function test_create_movie_requires_valid_data(): void
{
    $token = $this->getToken();

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->postJson('/api/auth/auth/movies', []);

    $response->assertStatus(422);
}

/**
 * Ver película
 */
public function test_authenticated_user_can_view_movie(): void
{
    $movie = Movie::factory()->create();

    $token = $this->getToken();

    $response = $this

```

```

        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->getJson('/api/auth/auth/movies/'.$movie->id);

$response
    ->assertStatus(200)
    ->assertJsonFragment([
        'id' => $movie->id
    ]);
}

/**
 * Actualizar película
 */
public function test_authenticated_user_can_update_movie(): void
{
    $movie = Movie::factory()->create();

    $token = $this->getToken();

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->putJson('/api/auth/auth/movies/'.$movie->id, [
            'title' => 'Updated Movie'
        ]);

    $response->assertStatus(200);

    $this->assertDatabaseHas('movies', [
        'id' => $movie->id,
        'title' => 'Updated Movie'
    ]);
}

/**
 * Eliminar película
 */
public function test_authenticated_user_can_delete_movie(): void
{
    $movie = Movie::factory()->create();

```

```

$token = $this->getToken();

$response = $this
    ->withHeader(
        'Authorization',
        'Bearer '.$token
    )->deleteJson('/api/auth/auth/movies/'.$movie->id);

$response->assertStatus(200);

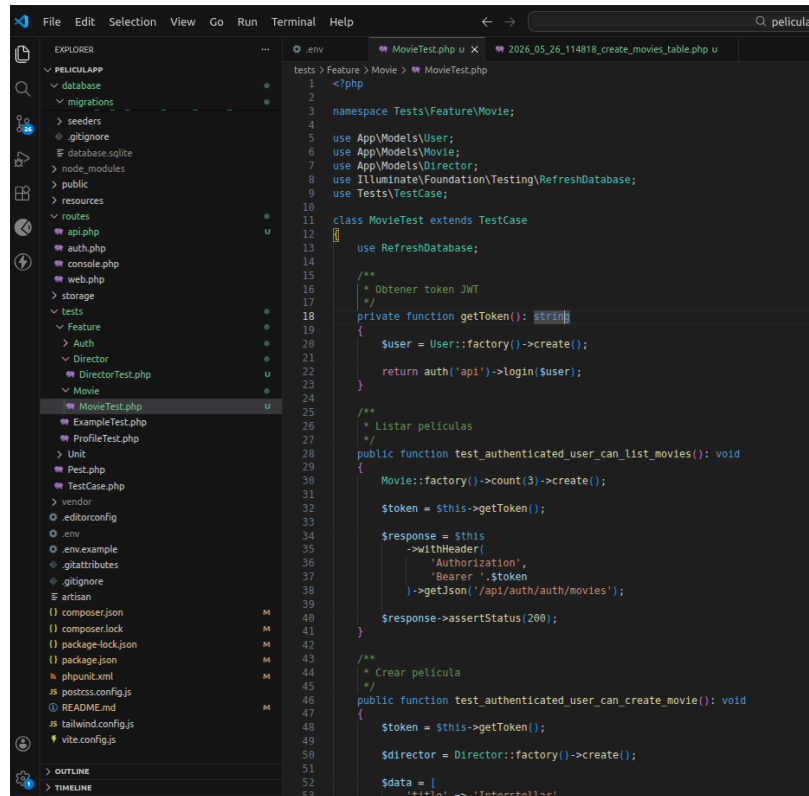
$this->assertDatabaseMissing('movies', [
    'id' => $movie->id
]);
}

/**
 * Rutas protegidas
 */
public function test_movies_endpoints_require_authentication(): void
{
    $response = $this->getJson('/api/auth/auth/movies');

    $response->assertStatus(401);
}
}

```

Este código contiene los test unitarios de las películas.



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'database', 'migrations', 'seeds', 'public', 'resources', 'routes', 'storage', 'tests', and 'vendor'. The 'tests' folder is expanded, showing 'Feature', 'Unit', and 'Movie' subfolders. The 'Movie' folder is selected, and the file 'MovieTest.php' is open in the editor. The code in the editor is as follows:

```
1 <?php
2
3 namespace Tests\Feature\Movie;
4
5 use App\Models\User;
6 use App\Models\Movie;
7 use App\Models\Director;
8 use Illuminate\Foundation\Testing\RefreshDatabase;
9 use Tests\TestCase;
10
11 class MovieTest extends TestCase
12 {
13     use RefreshDatabase;
14
15     /**
16      * Obtener token JWT
17      */
18     private function getToken(): string
19     {
20         $user = User::factory()->create();
21         return auth('api')->login($user);
22     }
23
24     /**
25      * Listar películas
26      */
27     public function test_authenticated_user_can_list_movies(): void
28     {
29         Movie::factory()->count(3)->create();
30
31         $token = $this->getToken();
32
33         $response = $this
34             ->withHeader(
35                 'Authorization',
36                 'Bearer ' . $token
37             )
38             ->getJson('/api/auth/auth/movies');
39
40         $response->assertStatus(200);
41     }
42
43     /**
44      * Crear película
45      */
46     public function test_authenticated_user_can_create_movie(): void
47     {
48         $token = $this->getToken();
49
50         $director = Director::factory()->create();
51
52         $data = [
53             'title' => 'Interstellar',
54             'release_date' => '2014-11-07',
55             'sinopsis' => 'Viaje espacial épico',
56             'duration' => 169,
57             'gendre' => 'Sci-Fi',
58             'director_id' => $director->id
59         ];
60
61         $response = $this
62             ->withHeader(
63                 'Authorization',
64                 'Bearer ' . $token
65             )
66             ->postJson('/api/auth/auth/movies', $data);
67
68         $response->assertStatus(201);
69
70         $this->assertDatabaseHas('movies', [
71             'title' => 'Interstellar'
72         ]);
73     }
74
75     /**
76      * Validación al crear película
77      */
78     public function test_create_movie_requires_valid_data(): void
79     {
80         $token = $this->getToken();
81
82         $response = $this
83             ->withHeader(
84                 'Authorization',
85                 'Bearer ' . $token
86             )
87             ->postJson('/api/auth/auth/movies', []);
88
89         $response->assertStatus(422);
90     }
91 }
```

```
    }
}

/**
 * Crear película
 */
public function test_authenticated_user_can_create_movie(): void
{
    $token = $this->getToken();

    $director = Director::factory()->create();

    $data = [
        'title' => 'Interstellar',
        'release_date' => '2014-11-07',
        'sinopsis' => 'Viaje espacial épico',
        'duration' => 169,
        'gendre' => 'Sci-Fi',
        'director_id' => $director->id
    ];

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer ' . $token
        )
        ->postJson('/api/auth/auth/movies', $data);

    $response->assertStatus(201);

    $this->assertDatabaseHas('movies', [
        'title' => 'Interstellar'
    ]);
}

/**
 * Validación al crear película
 */
public function test_create_movie_requires_valid_data(): void
{
    $token = $this->getToken();

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer ' . $token
        )
        ->postJson('/api/auth/auth/movies', []);

    $response->assertStatus(422);
}
}
```

```

/**
 * Ver película
 */
public function test_authenticated_user_can_view_movie(): void
{
    $movie = Movie::factory()->create();

    $token = $this->getToken();

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->getJson('/api/auth/movies/'.$movie->id);

    $response
        ->assertStatus(200)
        ->assertJsonFragment([
            'id' => $movie->id
        ]);
}

/**
 * Actualizar película
 */
public function test_authenticated_user_can_update_movie(): void
{
    $movie = Movie::factory()->create();

    $token = $this->getToken();

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->putJson('/api/auth/movies/'.$movie->id, [
            'title' => 'Updated Movie'
        ]);

    $response->assertStatus(200);
}

```

unload.

```

/**
 * Eliminar película
 */
public function test_authenticated_user_can_delete_movie(): void
{
    $movie = Movie::factory()->create();

    $token = $this->getToken();

    $response = $this
        ->withHeader(
            'Authorization',
            'Bearer '.$token
        )->deleteJson('/api/auth/auth/movies/'.$movie->id);

    $response->assertStatus(200);

    $this->assertDatabaseMissing('movies', [
        'id' => $movie->id
    ]);
}

/**
 * Rutas protegidas
 */
public function test_movies_endpoints_require_authentication(): void
{
    $response = $this->getJson('/api/auth/auth/movies');

    $response->assertStatus(401);
}
}

```

Una vez implementados los tests, los ejecutamos con el comando
php artisan test tests/Feature/Movie/MovieTest.php

En la captura se muestra que los test han pasado con éxito.

```

usuario@MV-Ubuntu:~/peliculapp$ php artisan test tests/Feature/Movie/MovieTest.php

PASS Tests\Feature\Movie\MovieTest
✓ authenticated user can list movies
✓ authenticated user can create movie
✓ create movie requires valid data
✓ authenticated user can view movie
✓ authenticated user can update movie
✓ authenticated user can delete movie
✓ movies endpoints require authentication

Tests: 7 passed (11 assertions)
Duration: 0.19s

usuario@MV-Ubuntu:~/peliculapp$

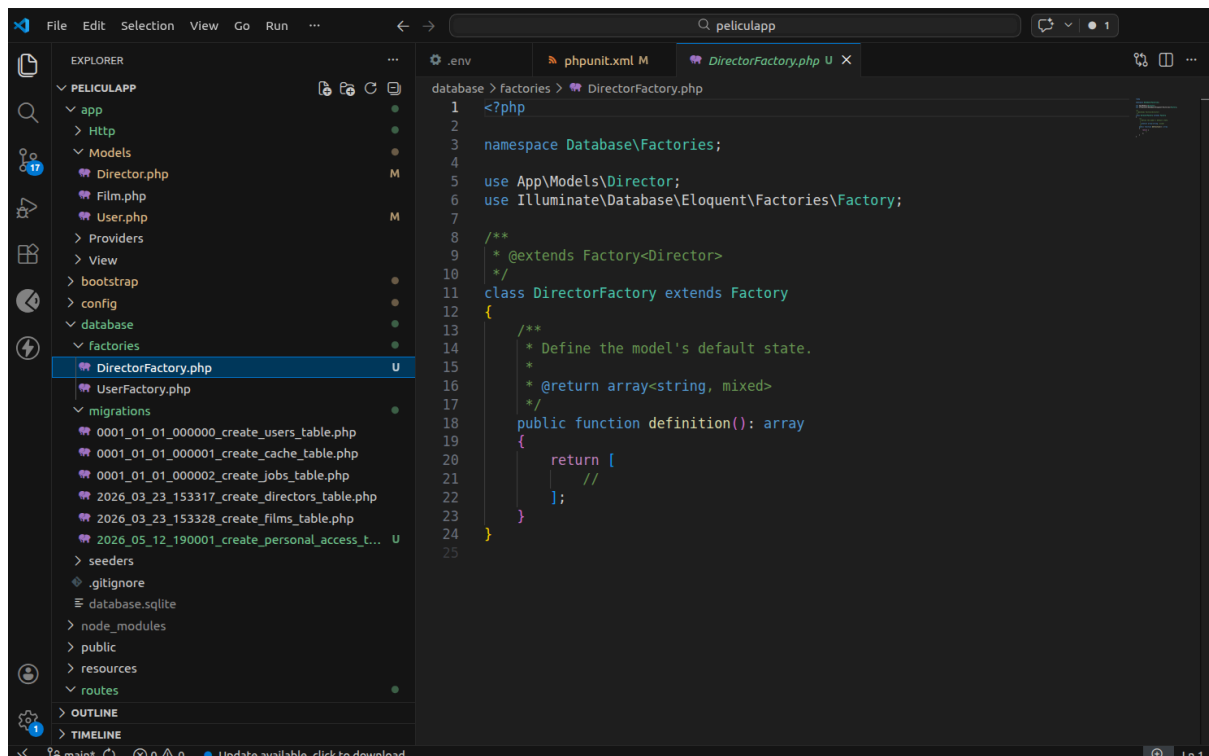
```

2.6.4.- Tests Seguridad transversal

Creamos Movie Factory ya que es necesario para los test. Para crearlo, usamos el comando **php artisan make:factory DirectorFactory --model=Movie**

INFO Factory [database/factories/MovieFactory.php] created successfully.

Y se nos genera MovieFactory.php en database/factories



En el archivo se pone:

```
<?php
```

```
namespace Database\Factories;
```

```
use App\Models\Movie;
```

```
use Illuminate\Database\Eloquent\Factories\Factory;
```

```
/**
```

```
 * @extends Factory<Movie>
```

```
 */
```

```
class MovieFactory extends Factory
```

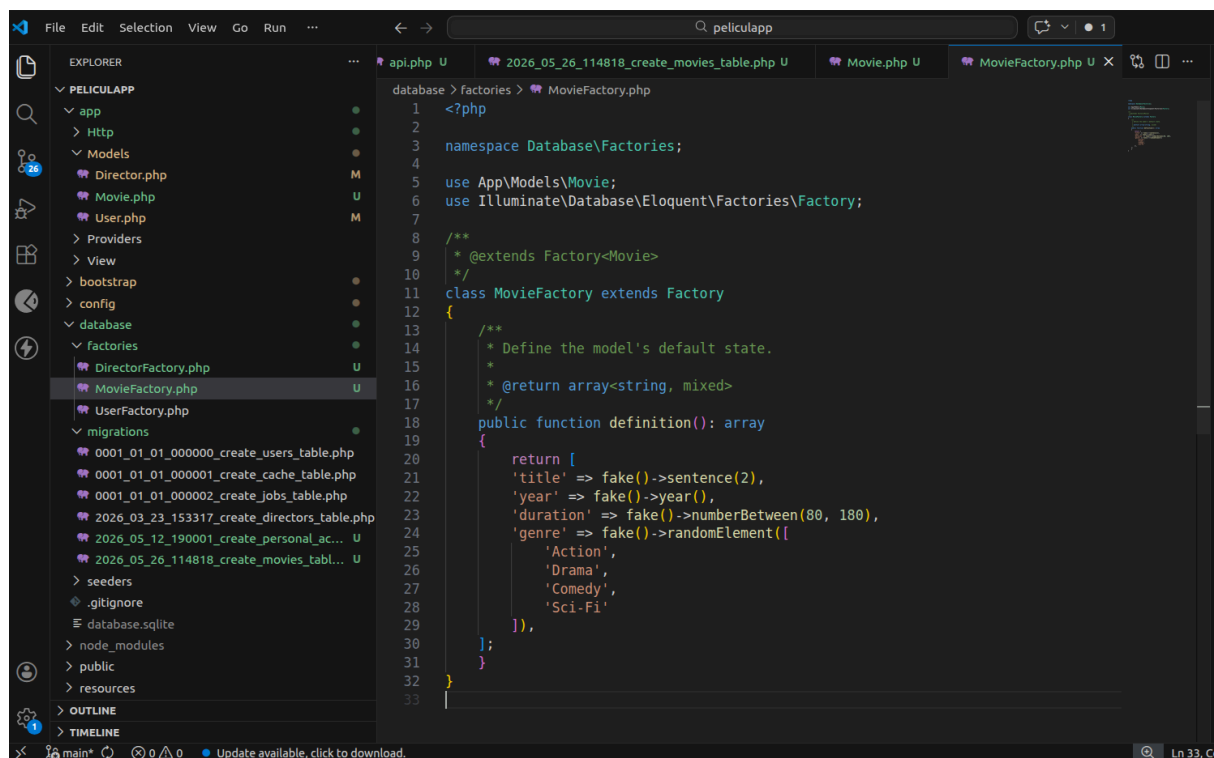
```
{
```

```

/**
 * Define the model's default state.
 *
 * @return array<string, mixed>
 */
public function definition(): array
{
    return [
        'title' => fake()->sentence(2),
        'year' => fake()->year(),
        'duration' => fake()->numberBetween(80, 180),
        'genre' => fake()->randomElement([
            'Action',
            'Drama',
            'Comedy',
            'Sci-Fi'
        ]),
    ];
}
}

```

Este quedaría de la siguiente manera:

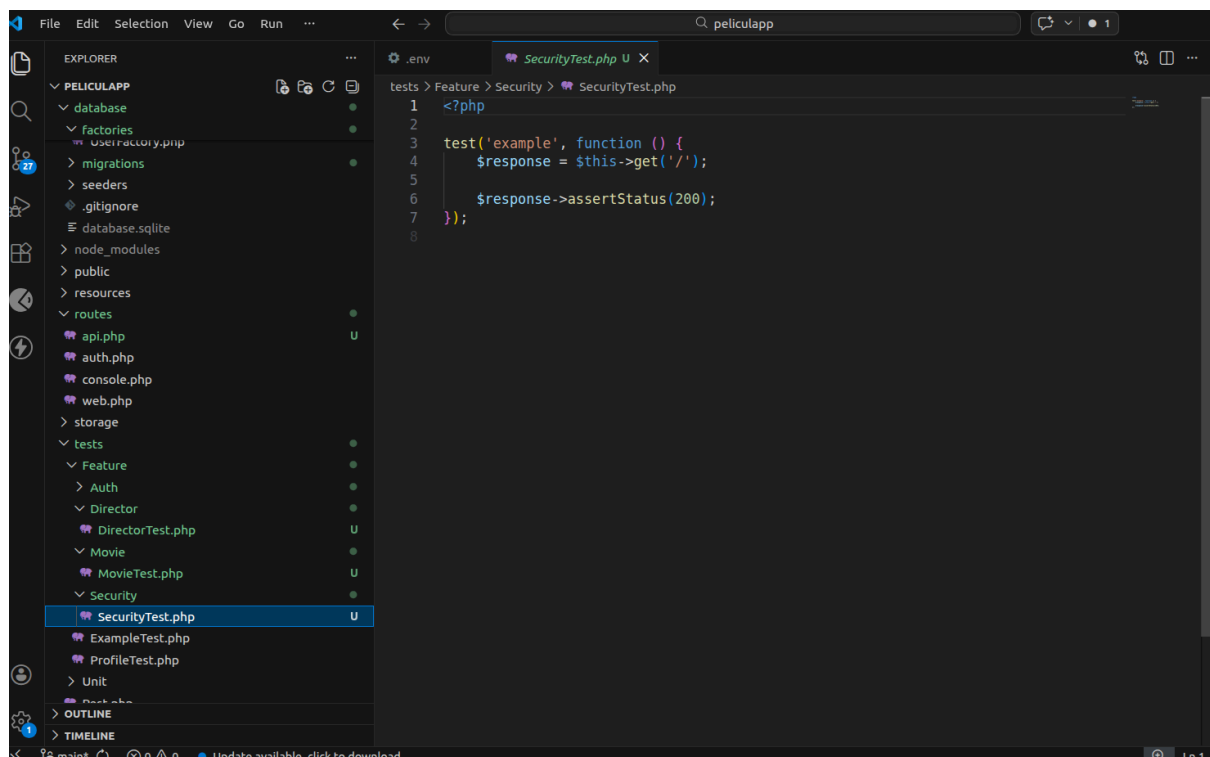


Tras implementar el factory, podemos crear los tests.

Para crear el archivo, se ejecuta el comando **php artisan make:test Security/SecurityTest**

```
usuario@MV-Ubuntu:~/peliculapp$ php artisan make:test Security/SecurityTest
INFO Test [tests/Feature/Security/SecurityTest.php] created successfully.
usuario@MV-Ubuntu:~/peliculapp$
```

Se genera un archivo SecurityTest.php donde se implementan las pruebas unitarias de seguridad transversal.



Una vez obtenido el archivo para los test, los implementamos y quedarían de esta forma:

```
<?php
```

```
namespace Tests\Feature\Security;
```

```
use App\Models\User;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Illuminate\Support\Facades\Hash;
```

```
use Carbon\Carbon;
```

```
use Tests\TestCase;
```

```

use Tymon\JWTAuth\Facades\JWTAuth;

class SecurityTest extends TestCase
{
    use RefreshDatabase;

    /**
     * TOKEN EXPIRADO → 401 con mensaje de expiración
     */
    public function test_token_expirado_devuelve_401()
    {
        $user = User::factory()->create([
            'password' => Hash::make('password123')
        ]);

        // Generamos token
        $token = JWTAuth::fromUser($user);

        // Simulamos que el tiempo ha pasado (token expirado)
        Carbon::setTestNow(Carbon::now()->addDays(10));

        $response = $this->withHeaders([
            'Authorization' => "Bearer $token"
        ])->getJson('/api/auth/me');

        $response->assertStatus(401)
            ->assertJson([
                'message' => 'Token has expired'
            ]);

        Carbon::setTestNow(); // limpiar mock de tiempo
    }

    /**
     * ERRORES NO EXPONEN STACK TRACE EN PRODUCCIÓN
     */
    public function test_respuestas_de_error_no_exponen_stack_trace()
    {
        config(['app.env' => 'production']);
        config(['app.debug' => false]);

        // Forzamos error llamando a endpoint inexistente protegido o inválido
        $response = $this->getJson('/api/endpoint-que-no-existe');
    }
}

```

```

    $response->assertStatus(404);

    $json = $response->json();

    $this->assertArrayNotHasKey('exception', $json);
    $this->assertArrayNotHasKey('file', $json);
    $this->assertArrayNotHasKey('line', $json);
}

/**
 * PASSWORD NUNCA SE EXPONE EN /me
 */
public function test_password_no_aparece_en_respuesta_me()
{
    $user = User::factory()->create([
        'password' => Hash::make('secret123')
    ]);

    $token = JWTAuth::fromUser($user);

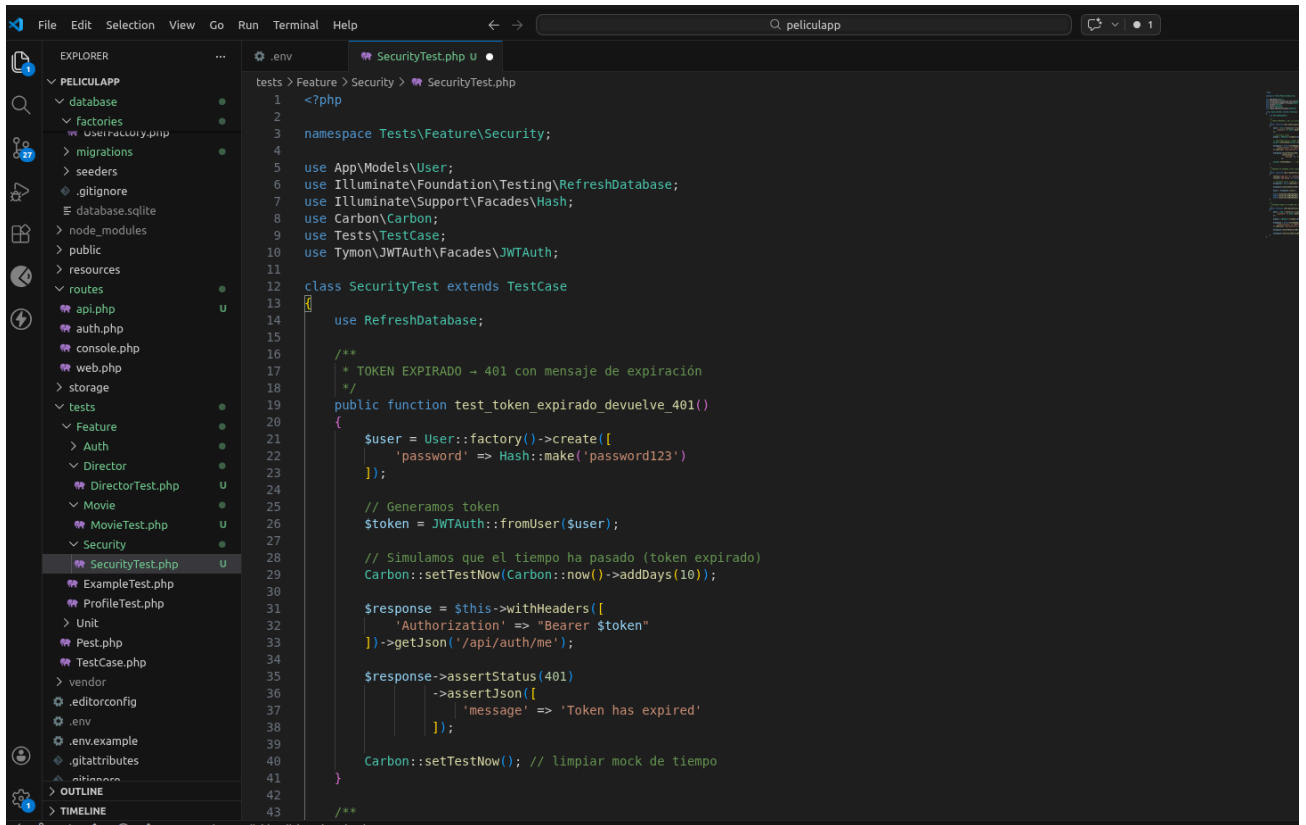
    $response = $this->withHeaders([
        'Authorization' => "Bearer $token"
    ]->getJson('/api/auth/me');

    $response->assertStatus(200);

    $response->assertJsonMissingPath('password');
}
}

```

Este código contiene los test unitarios de seguridad transversal.



```
    */
    public function test_respuestas_de_error_no_exponen_stack_trace()
    {
        config(['app.env' => 'production']);
        config(['app.debug' => false]);

        // Forzamos error llamando a endpoint inexistente protegido o inválido
        $response = $this->getJson('/api/endpoint-que-no-existe');

        $response->assertStatus(404);

        $json = $response->json();

        $this->assertArrayNotHasKey('exception', $json);
        $this->assertArrayNotHasKey('file', $json);
        $this->assertArrayNotHasKey('line', $json);
    }

    /**
     * PASSWORD NUNCA SE EXPONE EN /me
     */
    public function test_password_no_aparece_en_respuesta_me()
    {
        $user = User::factory()->create([
            'password' => Hash::make('secret123')
        ]);

        $token = JWTAuth::fromUser($user);

        $response = $this->withHeaders([
            'Authorization' => "Bearer $token"
        ])->getJson('/api/auth/me');

        $response->assertStatus(200);

        $response->assertJsonMissingPath('password');
    }
}
```

Una vez implementados los tests, los ejecutamos con el comando **php artisan test tests/Feature/Security/SecurityTest.php**

En la captura se muestra que los test han pasado con éxito.

```
usuario@MV-Ubuntu:~/peliculapp$ php artisan test tests/Feature/Security/SecurityTest.php

PASS Tests\Feature\Security\SecurityTest
✓ token expirado devuelve 401
✓ respuestas de error no exponen stack trace
✓ password no aparece en respuesta me

Tests: 3 passed (7 assertions)
Duration: 0.16s

usuario@MV-Ubuntu:~/peliculapp$
```

Si ejecutamos el comando **php artisan test**, este correrá todo los test que hemos implementado. En la captura se aprecia que han pasado todos.

```
usuario@MV-Ubuntu:~/peliculapp$ php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\Auth\AuthTest
✓ user can login with valid credentials
✓ user cannot login with invalid credentials
✓ login requires email
✓ login requires password
✓ authenticated user can access me endpoint
✓ me endpoint requires token
✓ user can refresh token
✓ logout invalidates token

PASS Tests\Feature\Director\DirectorTest
✓ authenticated user can list directors
✓ authenticated user can create director
✓ create director requires valid data
✓ authenticated user can view director
✓ authenticated user can update director
✓ authenticated user can delete director
✓ directors endpoints require authentication

PASS Tests\Feature\Movie\MovieTest
✓ authenticated user can list movies
✓ authenticated user can create movie
✓ create movie requires valid data
✓ authenticated user can view movie
✓ authenticated user can update movie
✓ authenticated user can delete movie
✓ movies endpoints require authentication

PASS Tests\Feature\Security\SecurityTest
✓ token expirado devuelve 401
✓ respuestas de error no exponen stack trace
✓ password no aparece en respuesta me

Tests: 26 passed (49 assertions)
Duration: 0.36s

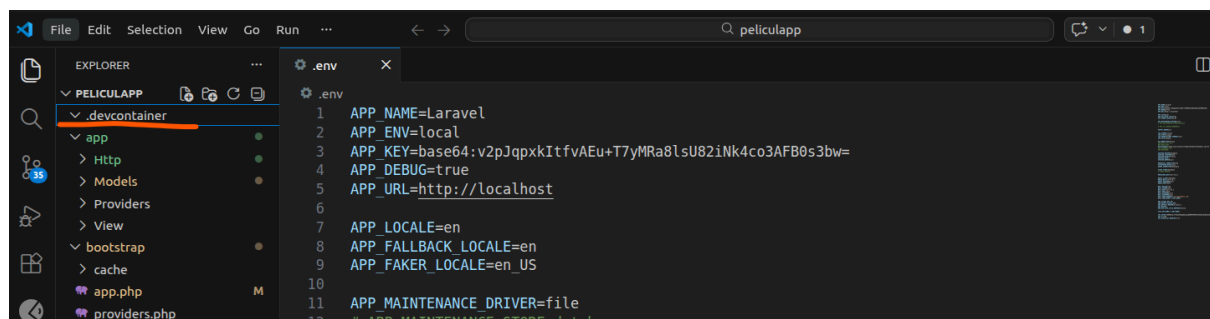
usuario@MV-Ubuntu:~/peliculapp$
```

3.- Fase 2 - CI/CD con GitHub Actions y Dev Containers

3.1.- Dev container

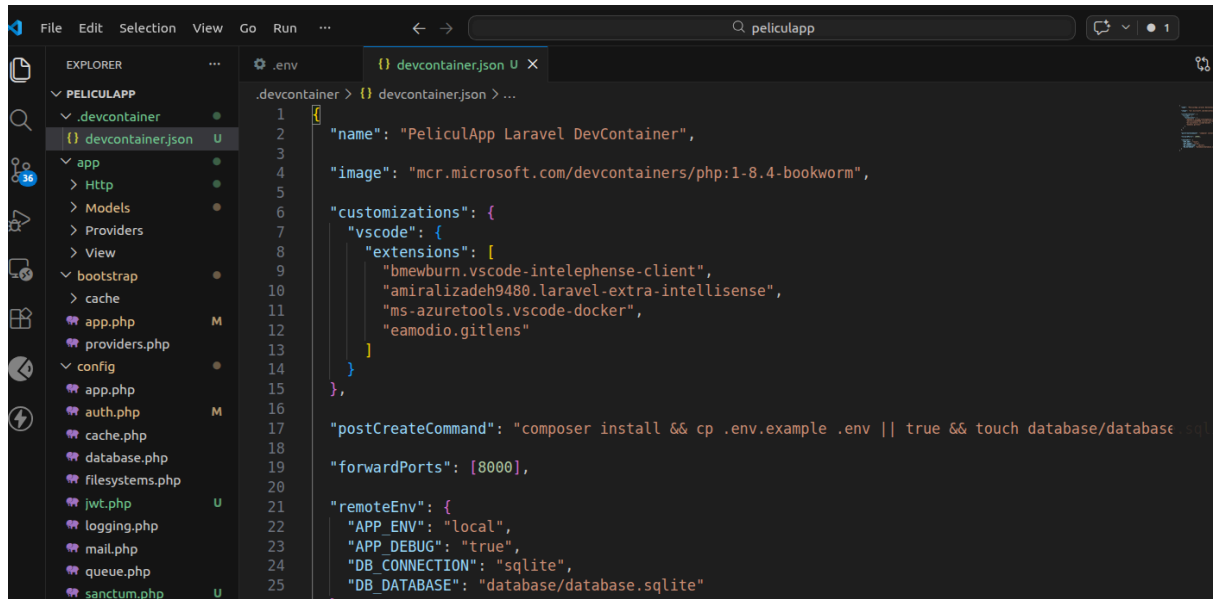
Creamos el directorio `.devcontainer/` en la raíz del repositorio.

```
usuario@MV-Ubuntu:~$ cd peliculapp
usuario@MV-Ubuntu:~/peliculapp$ mkdir -p .devcontainer/
usuario@MV-Ubuntu:~/peliculapp$ cd .devcontainer/
bash: cd: ./devcontainer: No existe el archivo o el directorio
usuario@MV-Ubuntu:~/peliculapp$ ls
app      bootstrap  composer.lock  database      package.json  phpunit.xml    public  resources  storage  tests  vite.config.js
artisan  composer.json  config         node_modules  package-lock.json  postcss.config.js  README.md  routes  tailwind.config.js  vendor
usuario@MV-Ubuntu:~/peliculapp$ code .
```

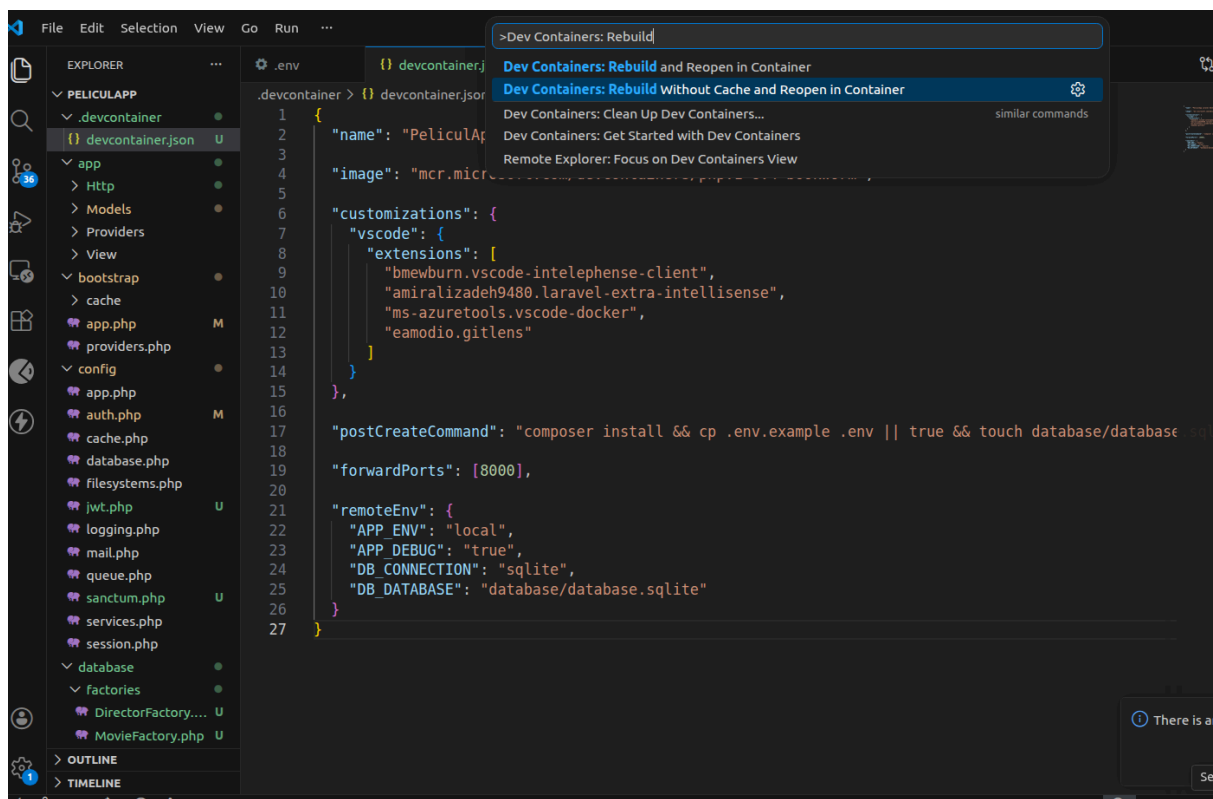


Creamos el fichero `devcontainer.json` con:

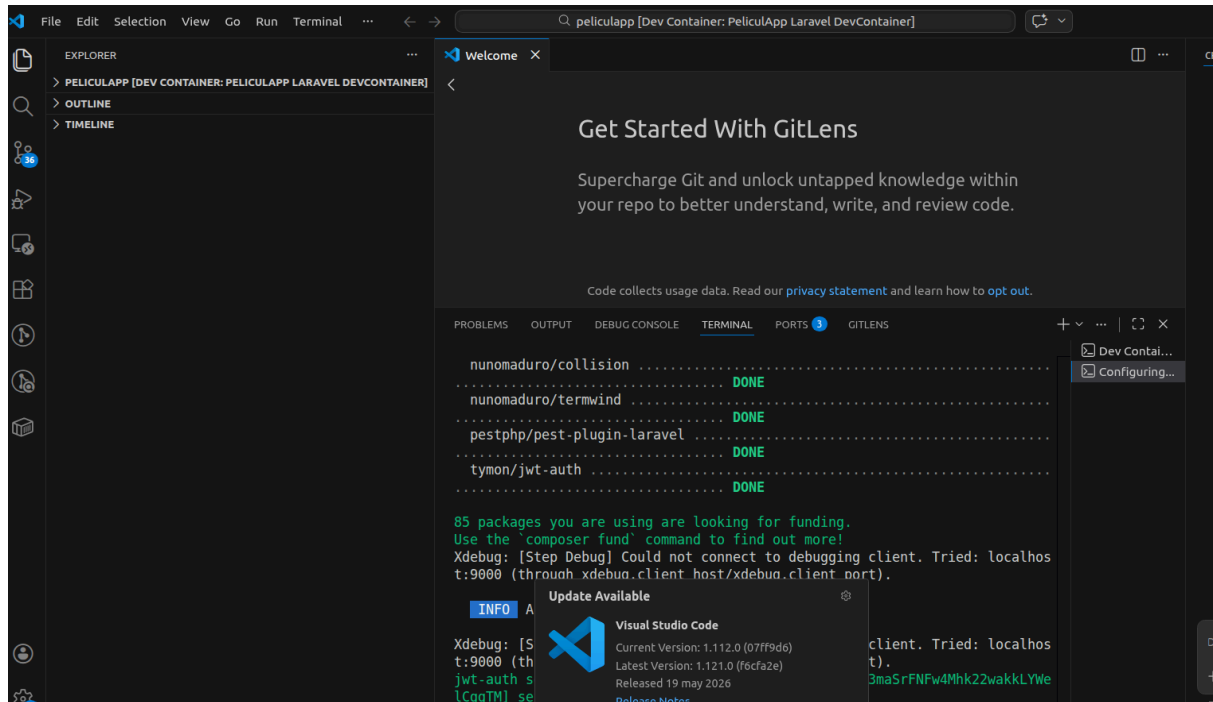
- Imagen base de PHP + Composer apropiada (p. ej. mcr.microsoft.com/devcontainers/php).
- Extensiones de VS Code recomendadas: PHP Intelephense, Laravel Extra Intellisense, Docker, GitLens.
- `postCreateCommand` que instale dependencias: `composer install && php artisan key:generate`.
- Variables de entorno necesarias para la conexión a base de datos (usando `.env` o `devcontainer secrets`).



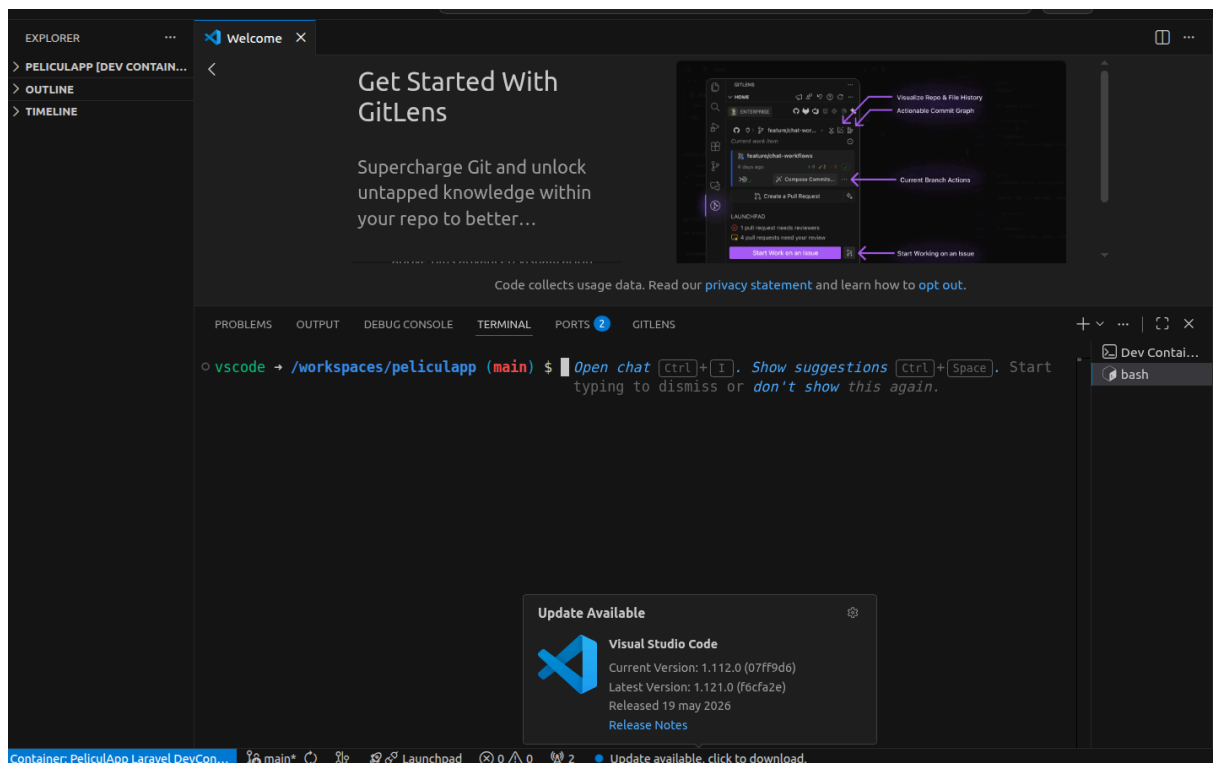
Verificamos que el entorno arranca correctamente en VS Code local con la extensión Dev Containers.



Tras el proceso, la aplicación se queda dentro del contenedor:



Abrimos un terminal:



Verificamos la version de php:


```
vscode → /workspaces/peliculapp (main) $ php -v
PHP 8.4.10 (cli) (built: Jul  3 2025 22:57:42) (NTS)
Copyright (c) The PHP Group
Built by https://github.com/docker-library/php
Zend Engine v4.4.10, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.10, Copyright (c), by Zend Technologies
    with Xdebug v3.4.4, Copyright (c) 2002-2025, by Derick Rethans
vscode → /workspaces/peliculapp (main) $
```

Por último realizamos la prueba: con php artisan test

```
With Xdebug v3.4.4, Copyright (c) 2002-2025, by Derick Rethans
• vscode → /workspaces/peliculapp (main) $ php artisan test
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9000 (through xdebug.client_host/xdebug.client_port).
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9000 (through xdebug.client_host/xdebug.client_port).

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\Auth\AuthTest
✓ user can login with valid credentials 2.06s
✓ user cannot login with invalid credentials 0.01s
✓ login requires email 0.01s
✓ login requires password 0.01s
✓ authenticated user can access me endpoint 0.01s
✓ me endpoint requires token 0.01s
✓ user can refresh token 0.01s
✓ logout invalidates token 0.01s

PASS Tests\Feature\Director\DirectorTest
✓ authenticated user can list directors 0.02s
✓ authenticated user can create director 0.01s
✓ create director requires valid data 0.01s
✓ authenticated user can view director 0.01s
✓ authenticated user can update director 0.01s
✓ authenticated user can delete director 0.01s
✓ directors endpoints require authentication 0.01s

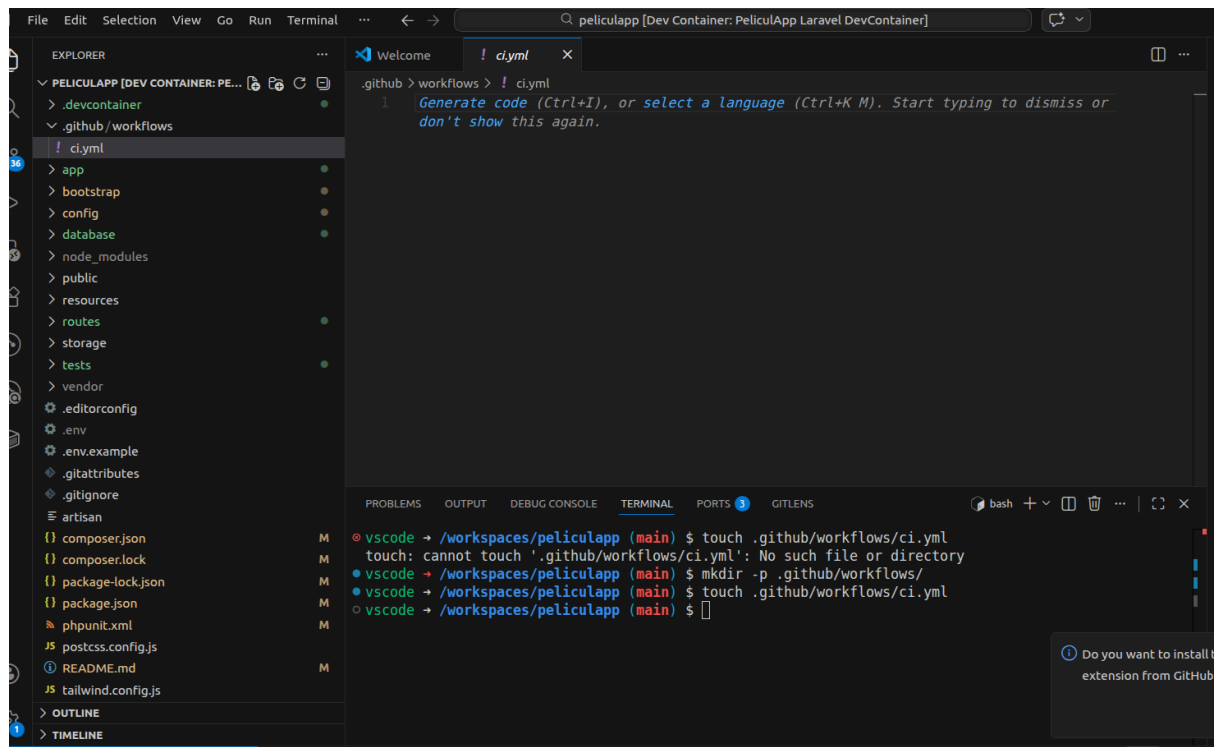
PASS Tests\Feature\Movie\MovieTest
✓ authenticated user can list movies 0.02s
✓ authenticated user can create movie 0.01s
✓ create movie requires valid data 0.01s
✓ authenticated user can view movie 0.01s
✓ authenticated user can update movie 0.01s
✓ authenticated user can delete movie 0.01s
✓ movies endpoints require authentication 0.01s

PASS Tests\Feature\Security\SecurityTest
```

Las pruebas en el dev container funcionan.

3.2.- Pipeline de Integración Continua (CI)

Creamos el directorio .github/workflows en la raíz del repositorio. Creamos un ci.yml:



En el archivo ci.yml ponemos:

name: CI

on:

push:

branches:

- main
- develop

pull_request:

branches:

- main
- develop

permissions:

contents: read

jobs:

lint:

name: Lint (Pint PSR-12)

runs-on: ubuntu-latest

strategy:

matrix:
 php-version: [8.4]

steps:

- **name:** Checkout repository
 uses: actions/checkout@v4

- **name:** Setup PHP
 uses: shivammathur/setup-php@v2
 with:
 php-version: \${ matrix.php-version }
 extensions: mbstring, pdo, sqlite, pdo_sqlite
 coverage: none

- **name:** Install dependencies
 run: composer install --no-interaction --prefer-dist

- **name:** Run Pint
 run: ./vendor/bin/pint --test

test:
 name: PHPUnit Tests
 runs-on: ubuntu-latest
 needs: lint

strategy:
 matrix:
 php-version: [8.4]

env:
 APP_ENV: testing
 APP_DEBUG: false
 APP_KEY: \${ secrets.APP_KEY }

DB_CONNECTION: sqlite
 DB_DATABASE: ":memory:"

CACHE_DRIVER: array
 SESSION_DRIVER: array
 QUEUE_CONNECTION: sync

JWT_SECRET: \${ secrets.JWT_SECRET }

steps:

- name: Checkout repository
uses: actions/checkout@v4
- name: Setup PHP
uses: shivammathur/setup-php@v2
with:
 php-version: \${{ matrix.php-version }}
 extensions: mbstring, pdo, sqlite, pdo_sqlite
 coverage: none
- name: Copy .env
run: cp .env.example .env
- name: Install dependencies
run: composer install --no-interaction --prefer-dist
- name: Generate application key
run: php artisan key:generate
- name: Run migrations
run: php artisan migrate --force
- name: Run tests
run: php artisan test

build:

name: Docker Build
runs-on: ubuntu-latest
needs: test

steps:

- name: Checkout repository
uses: actions/checkout@v4
- name: Build Docker image
run: docker build -t peliculapp .
- name: Run container
run: docker run -d -p 8000:8000 --name peliculapp_test peliculapp
- name: Verify container is running
run: docker ps

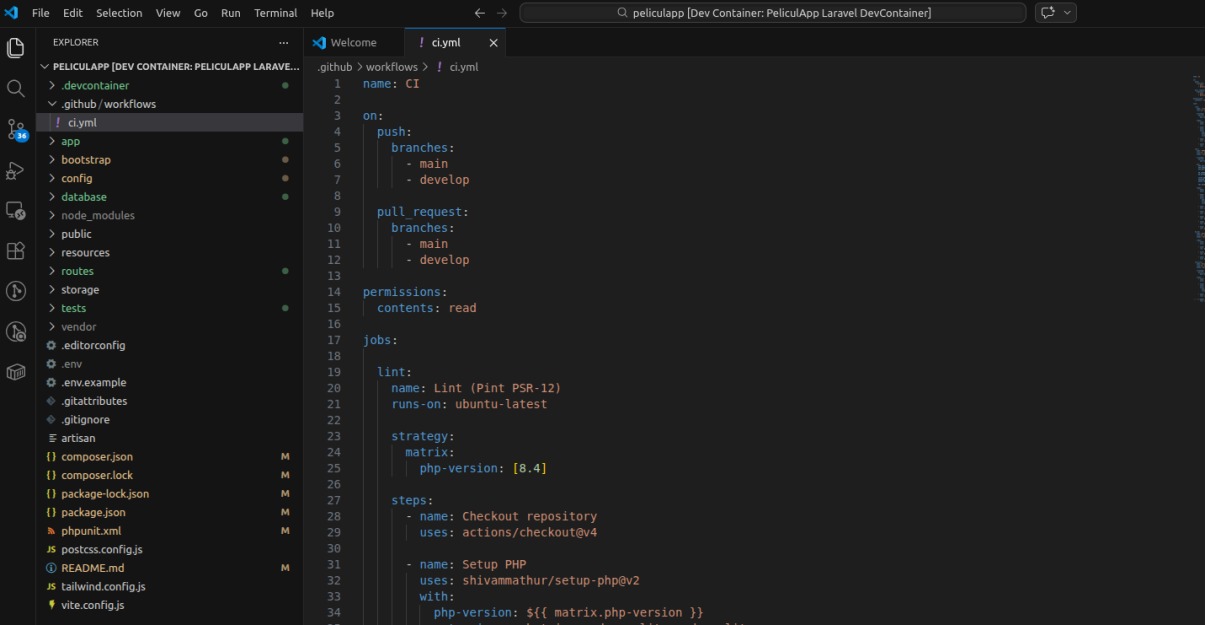
security:

name: Security Audit
runs-on: ubuntu-latest
needs: build

strategy:
matrix:
 php-version: [8.4]

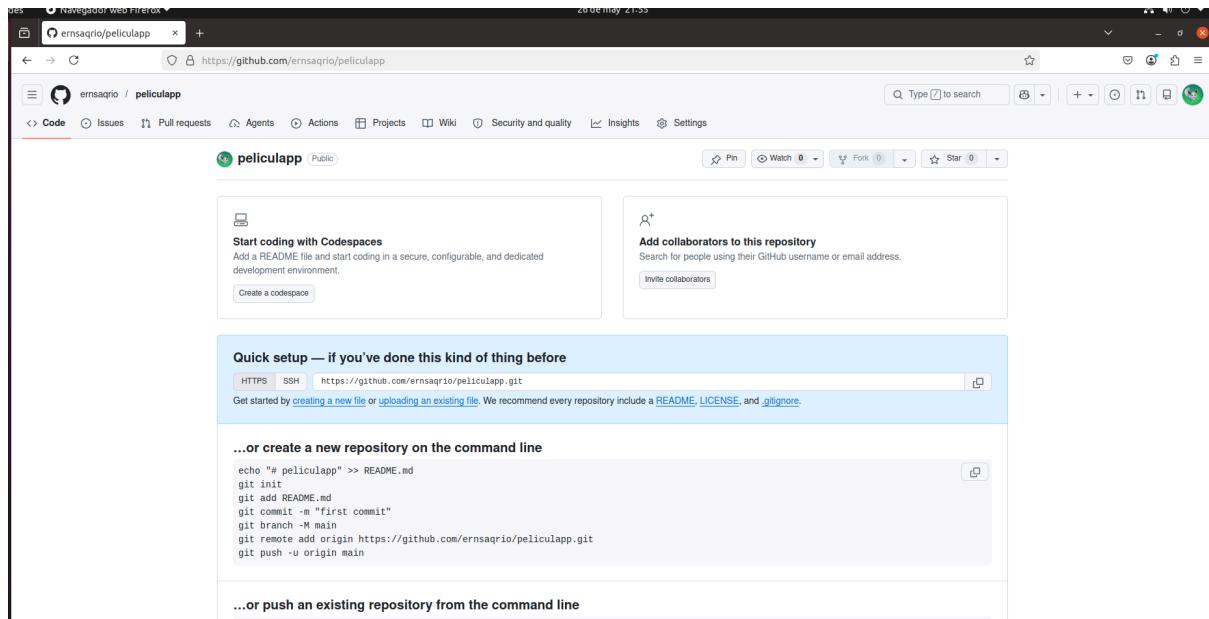
- steps:**
- **name: Checkout repository**
 uses: actions/checkout@v4
 - **name: Setup PHP**
 uses: shivammathur/setup-php@v2
 with:
 php-version: \${ matrix.php-version }
 coverage: none
 - **name: Install dependencies**
 run: composer install --no-interaction --prefer-dist
 - **name: Run composer audit**
 run: composer audit

El archivo quedaría:



```
1 name: CI
2
3 on:
4   push:
5     branches:
6       - main
7       - develop
8
9   pull_request:
10    branches:
11      - main
12      - develop
13
14 permissions:
15   contents: read
16
17 jobs:
18
19   lint:
20     name: Lint (Pint PSR-12)
21     runs-on: ubuntu-latest
22
23     strategy:
24       matrix:
25         php-version: [8.4]
26
27     steps:
28       - name: Checkout repository
29         uses: actions/checkout@v4
30
31       - name: Setup PHP
32         uses: shivammathur/setup-php@v2
33         with:
34           php-version: ${ matrix.php-version }
```

Creamos un repositorio en github llamado peliculapp



Usamos el comando `git remote -v` para verificar el repositorio remoto del que he clonado el proyecto.

```
usuario@MV-Ubuntu:~/peliculapp$ git remote -v
origin https://github.com/jlfernandez83mm/peliculapp.git (fetch)
origin https://github.com/jlfernandez83mm/peliculapp.git (push)
usuario@MV-Ubuntu:~/peliculapp$
```

Borramos ese repositorio remoto: `git remote remove origin`

```
usuario@MV-Ubuntu:~/peliculapp$ git remote remove origin
usuario@MV-Ubuntu:~/peliculapp$ git remote -v
usuario@MV-Ubuntu:~/peliculapp$
```

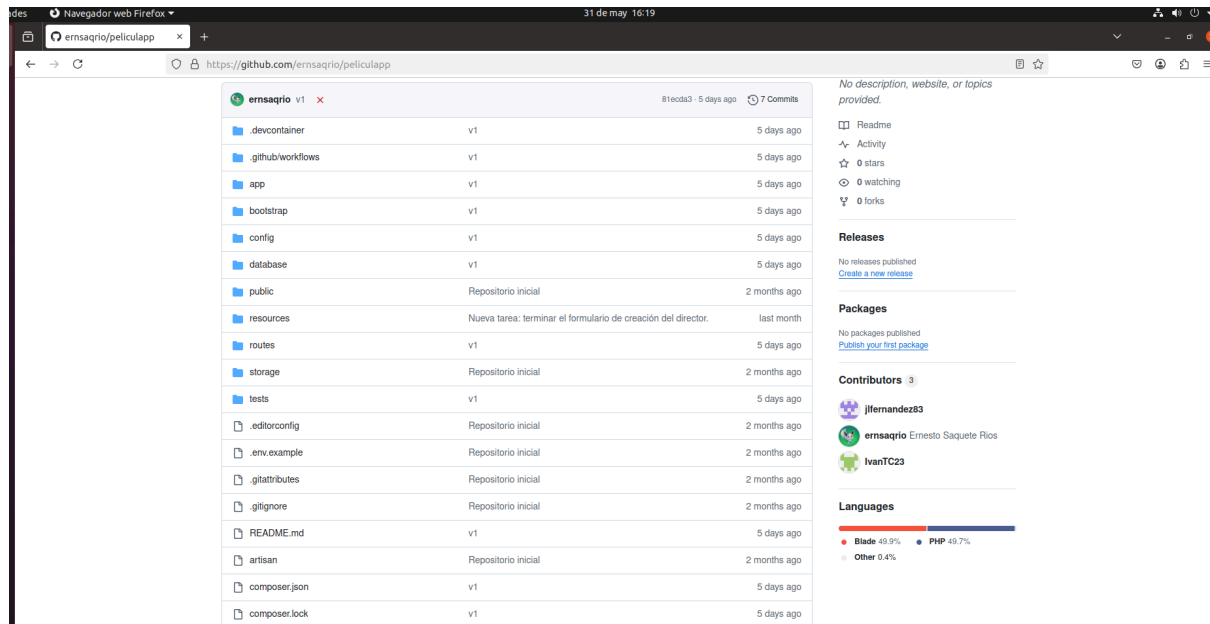
Ya no tiene repositorio remoto. Añadimos ahora el repo a nuestro repo: `git remote add origin https://github.com/ernsaqrio/peliculapp.git`

```
usuario@MV-Ubuntu:~/peliculapp$ git remote add origin https://github.com/ernsaqrio/peliculapp.git
usuario@MV-Ubuntu:~/peliculapp$ git remote -v
origin https://github.com/ernsaqrio/peliculapp.git (fetch)
origin https://github.com/ernsaqrio/peliculapp.git (push)
usuario@MV-Ubuntu:~/peliculapp$
```

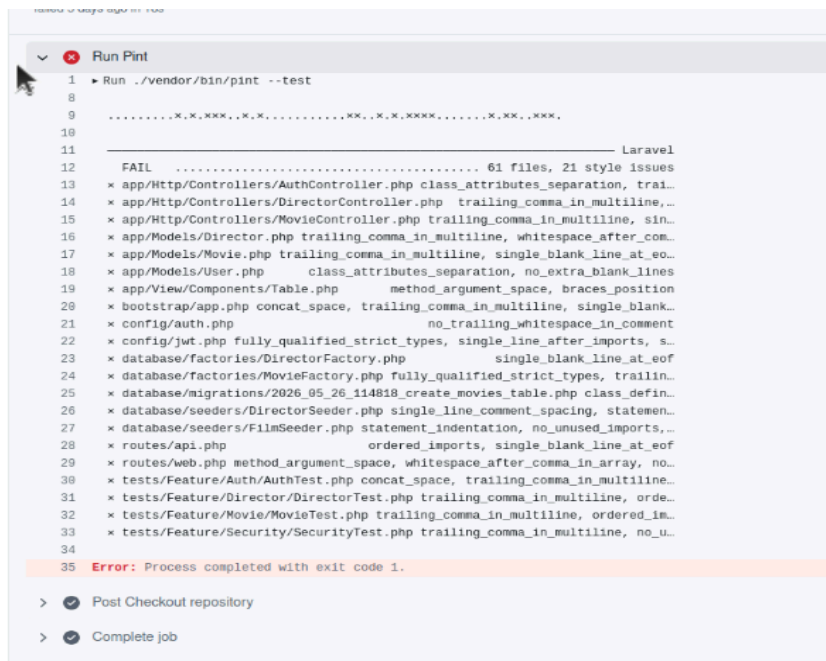
Despues se ejecuta los comandos:

- `git add .`
- `git status`
- `git commit -m "el mensaje que quieras"`
- `git push origin main`

El proyecto queda guardado en nuestro repositorio de github. También se instaló la extensión github actions en vscode. En la captura se muestra el proyecto en github (la x indica que github actions funciona pero el proyecto no cumple con PSR-12, esto es normal ya que se trata de un commit inicial y hay que solventar varios aspectos).



Otra evidencia de que no cumple con PSR-12.



Para solventar el problema usamos el comando ./vendor/bin/pint. Pint modificará automáticamente los archivos que aparecen en la lista.

```
usuario@MV-Ubuntu:~/peliculapp$ ./vendor/bin/pint

.....
Laravel
61 files

usuario@MV-Ubuntu:~/peliculapp$
```

Usando git status vemos los cambios que se han producido:

```
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
modificados:   app/Http/Controllers/AuthController.php
modificados:   app/Http/Controllers/DirectorController.php
modificados:   app/Http/Controllers/MovieController.php
modificados:   app/Models/Director.php
modificados:   app/Models/Movie.php
modificados:   app/Models/User.php
modificados:   app/View/Components/Table.php
modificados:   bootstrap/app.php
modificados:   config/auth.php
modificados:   config/jwt.php
modificados:   database/factories/DirectorFactory.php
modificados:   database/factories/MovieFactory.php
modificados:   database/migrations/2026_05_26_114818_create_movies_table.php
modificados:   database/seeder/DirectorSeeder.php
modificados:   database/seeder/FilmSeeder.php
modificados:   routes/api.php
modificados:   routes/web.php
modificados:   tests/Feature/Auth/AuthTest.php
modificados:   tests/Feature/Director/DirectorTest.php
modificados:   tests/Feature/Movie/MovieTest.php
modificados:   tests/Feature/Security/SecurityTest.php

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
usuario@MV-Ubuntu:~/peliculapp$
```

Después realizamos el commit en el repositorio local.

```
sin cambios agregados al commit (usa "git add" y/o "git commit -a")
usuario@MV-Ubuntu:~/peliculapp$ git add .
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
(usa "git restore --staged <archivo>..." para sacar del área de stage)
modificados:   app/Http/Controllers/AuthController.php
modificados:   app/Http/Controllers/DirectorController.php
modificados:   app/Http/Controllers/MovieController.php
modificados:   app/Models/Director.php
modificados:   app/Models/Movie.php
modificados:   app/Models/User.php
modificados:   app/View/Components/Table.php
modificados:   bootstrap/app.php
modificados:   config/auth.php
modificados:   config/jwt.php
modificados:   database/factories/DirectorFactory.php
modificados:   database/factories/MovieFactory.php
modificados:   database/migrations/2026_05_26_114818_create_movies_table.php
modificados:   database/seeder/DirectorSeeder.php
modificados:   database/seeder/FilmSeeder.php
modificados:   routes/api.php
modificados:   routes/web.php
modificados:   tests/Feature/Auth/AuthTest.php
modificados:   tests/Feature/Director/DirectorTest.php
modificados:   tests/Feature/Movie/MovieTest.php
modificados:   tests/Feature/Security/SecurityTest.php

usuario@MV-Ubuntu:~/peliculapp$ git commit -m "Apply Pint formatting"
[main 0789b17] Apply Pint formatting
21 files changed, 144 insertions(+), 154 deletions(-)
rewrite database/seeder/DirectorSeeder.php (76%)
usuario@MV-Ubuntu:~/peliculapp$
```

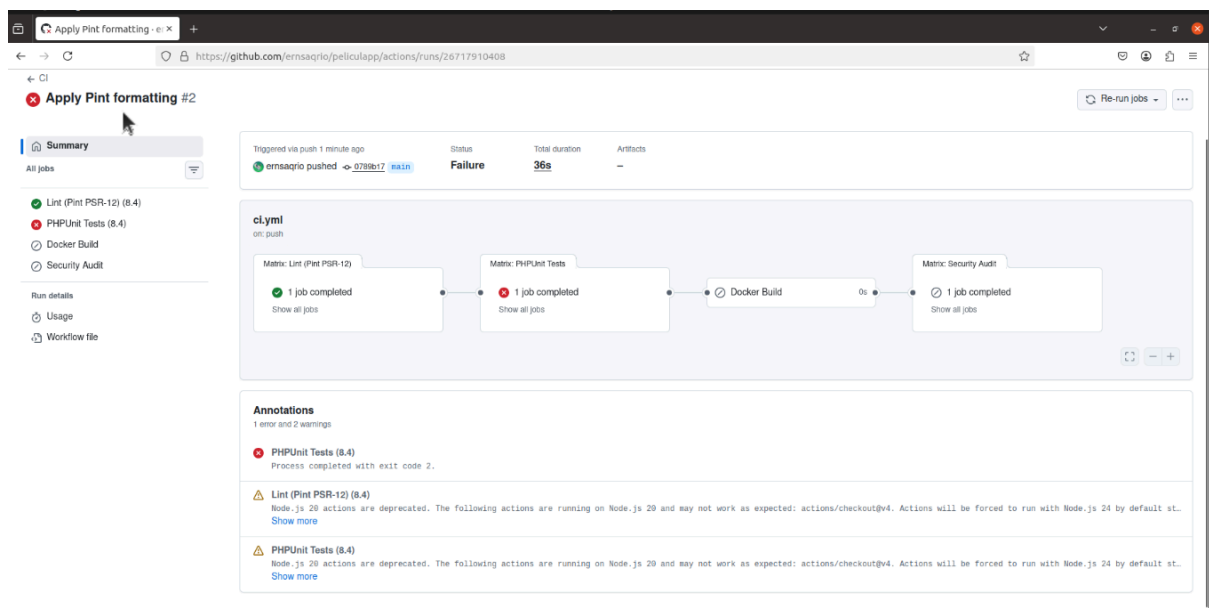

Ejecutamos `./vendor/bin/pint --test` y verificamos que los 61 archivos pasan:

```
usuario@MV-Ubuntu:~/peliculapp$ ./vendor/bin/pint --test
.....
Laravel
61 files
usuario@MV-Ubuntu:~/peliculapp$
```

Publicamos en remoto:

```
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsaqrio
Password for 'https://ernsaqrio@github.com':
Enumerando objetos: 82, listo.
Contando objetos: 100% (82/82), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (37/37), listo.
Escribiendo objetos: 100% (42/42), 4.20 KiB | 4.20 MiB/s, listo.
Total 42 (delta 25), reusado 0 (delta 0)
remote: Resolving deltas: 100% (25/25), completed with 25 local objects.
To https://github.com/ernsaqrio/peliculapp.git
 81ecda3..0789b17  main -> main
usuario@MV-Ubuntu:~/peliculapp$
```

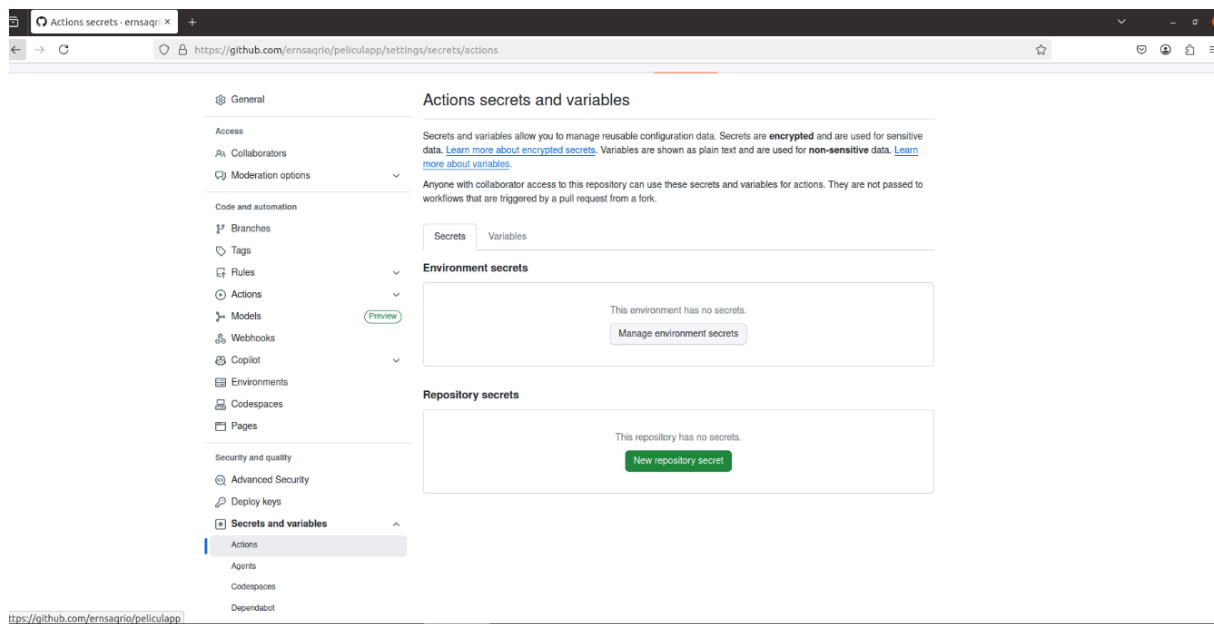
Se aprecia que el lint está correcto pero no pasan los test:



```
Run tests
1 ▶ Run php artisan test
17
18 PASS Tests\Unit\ExampleTest
19 ✓ that true is true
20
21 FAIL Tests\Feature\Auth\AuthTest
22 ✕ user can login with valid credentials 0.17s
23 ✓ user cannot login with invalid credentials 0.01s
24 ✓ login requires email 0.01s
25 ✓ login requires password 0.01s
26 ✕ authenticated user can access me endpoint 0.01s
27 ✓ me endpoint requires token 0.01s
28 ✕ user can refresh token 0.01s
29 ✕ logout invalidates token 0.01s
30
31 FAIL Tests\Feature\Director\DirectorTest
32 ✕ authenticated user can list directors 0.01s
33 ✕ authenticated user can create director 0.01s
34 ✕ create director requires valid data 0.01s
35 ✕ authenticated user can view director 0.01s
36 ✕ authenticated user can update director 0.01s
37 ✕ authenticated user can delete director 0.01s
38 ✓ directors endpoints require authentication 0.01s
39
40 FAIL Tests\Feature\Movie\MovieTest
41 ✕ authenticated user can list movies 0.01s
42 ✕ authenticated user can create movie 0.01s
43 ✕ create movie requires valid data 0.01s
44 ✕ authenticated user can view movie 0.01s
45 ✕ authenticated user can update movie 0.01s
46 ✕ authenticated user can delete movie 0.01s
47 ✓ movies endpoints require authentication 0.01s
48
49 FAIL Tests\Feature\Security\SecurityTest
```

Esto se debe a que no se ha establecido ninguna clave secreta en el ci.yml

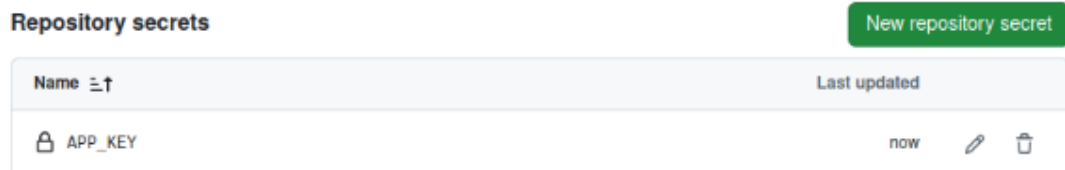
Verificamos los secrets y vemos que no hay:



Obtenemos primero el app key:

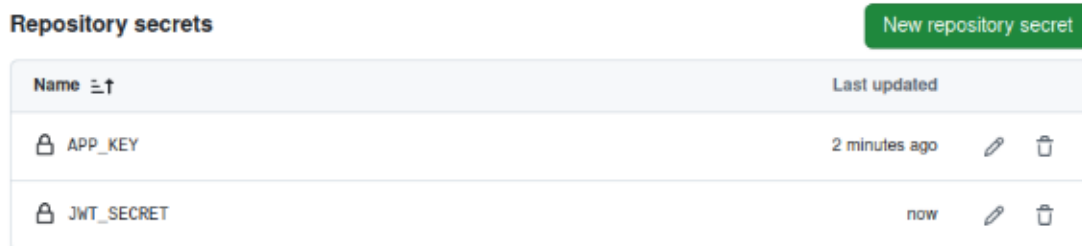
```
usuario@MV-Ubuntu:~/peliculapp$ cat .env | grep APP_KEY
APP_KEY=base64:fWNS8h++8U+sHcp1RnSDwUCrrmt+Je5VbqSzt7IozM4=
usuario@MV-Ubuntu:~/peliculapp$
```

Copiamos solo el valor y lo incluimos en github.



Hacemos lo mismo con el JWT_secret:

```
usuario@MV-Ubuntu:~/peliculapp$ cat .env | grep APP_KEY
APP_KEY=base64:fWNS8h++8U+sHcp1RnSDwUCrrmt+Je5Vbg5zt7IozM4=
usuario@MV-Ubuntu:~/peliculapp$ cat .env | grep JWT_SECRET
JWT_SECRET=aGMqviPtJj4w72dm1XnmWY2M9CMcCgBcmXh3maSrFNFw4Mhk22wakkLYWelCggTM
usuario@MV-Ubuntu:~/peliculapp$
```



Después una vez que todo está arreglado, hacemos commit y push:

```

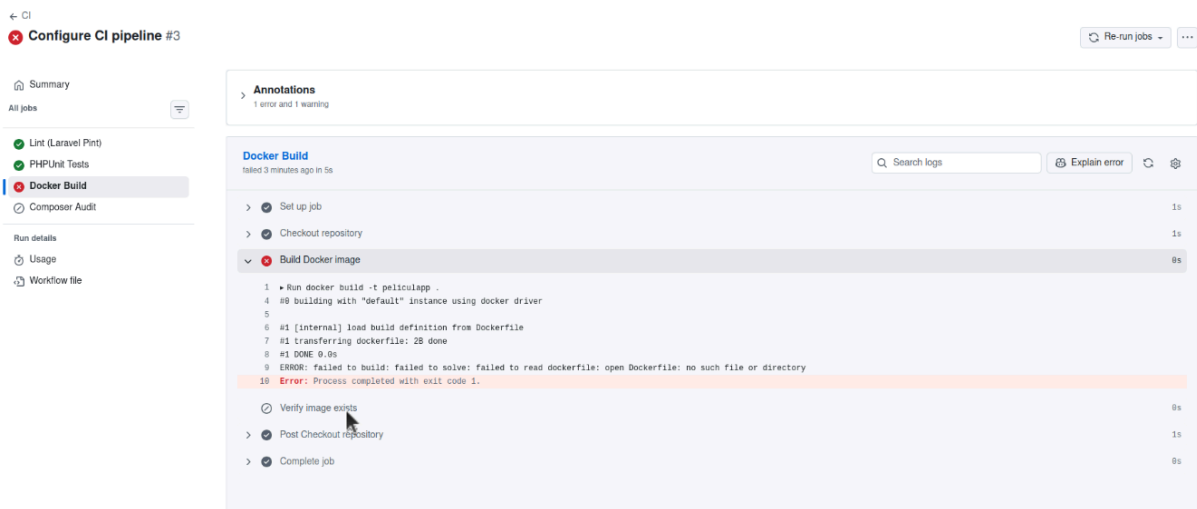
usuario@MV-Ubuntu:~/peliculapp$ git add .
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificados:      .github/workflows/ci.yml

usuario@MV-Ubuntu:~/peliculapp$ git commit -m "Configure CI pipeline"
[main 25dacde] Configure CI pipeline
 1 file changed, 24 insertions(+), 44 deletions(-)
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsaqrio
Password for 'https://ernsaqrio@github.com':
Enumerando objetos: 9, listo.
Contando objetos: 100% (9/9), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (5/5), 735 bytes | 735.00 KiB/s, listo.
Total 5 (delta 2), reusado 0 (delta 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/ernsaqrio/peliculapp.git
   0789b17..25dacde  main -> main
usuario@MV-Ubuntu:~/peliculapp$

```

Vemos que ha pasado los test pero falla docker build posiblemente no exista ningún archivo Dockerfile y este se tiene que crear:



← CI
 ✖ Configure CI pipeline #3

Summary
 All jobs

Lint (Laravel Pint)
 PHPUnit Tests
 ✖ Docker Build
 Composer Audit

Run details
 Usage
 Workflow file

Annotations
 1 error and 1 warning

Docker Build
 failed 3 minutes ago in 5s

Search logs Explain error

- Set up job 1s
- Checkout repository 1s
- Build Docker image 8s
 - 1 • Run docker build -t peliculapp .
 - 4 ## building with "default" instance using docker driver
 - 5
 - 6 #1 [internal] load build definition from Dockerfile
 - 7 #1 transferring dockerfile: 2B done
 - 8 #1 DONE 0.0s
 - 9 ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
 - 10 Error: Process completed with exit code 1.
- Verify image exists 0s
- Post Checkout repository 1s
- Complete job 0s

Haciendo un `ls -la` en el directorio del proyecto, vemos que no hay un Dockerfile por lo que hay que crearlo:

```

usuario@MV-Ubuntu:~/peliculapp$ ls -la
total 652
-rwxrwxr-x 16 usuario usuario 4096 may 26 21:40 .
-rwxr-xr-x 24 usuario usuario 4096 may 26 22:08 ..
-rwxrwxr-x 6 usuario usuario 4096 may 4 16:23 app
-rwxrwxr-x 1 usuario usuario 425 may 4 16:23 artisan
-rwxrwxr-x 3 usuario usuario 4096 may 4 16:23 bootstrap
-rw-rw-r-- 1 usuario usuario 2978 may 12 20:59 composer.json
-rw-rw-r-- 1 usuario usuario 345912 may 12 20:59 composer.lock
-rwxrwxr-x 2 usuario usuario 4096 may 12 21:00 config
-rwxrwxr-x 5 usuario usuario 4096 may 26 21:44 database
-rwxrwxr-x 2 usuario usuario 4096 may 26 21:23 .devcontainer
-rw-rw-r-- 1 usuario usuario 252 may 4 16:23 .editorconfig
-rw-rw-r-- 1 usuario usuario 1208 may 26 21:29 .env
-rw-rw-r-- 1 usuario usuario 1080 may 4 16:23 .env.example
-rwxrwxr-x 9 usuario usuario 4096 may 31 18:52 .git
-rw-rw-r-- 1 usuario usuario 186 may 4 16:23 .gitattributes
-rwxr-xr-x 3 usuario usuario 4096 may 26 21:40 .github
-rw-rw-r-- 1 usuario usuario 283 may 4 16:23 .gitignore
-rwxrwxr-x 181 usuario usuario 4096 may 4 16:29 node_modules
-rw-rw-r-- 1 usuario usuario 656 may 4 16:26 package.json
-rw-rw-r-- 1 usuario usuario 194251 may 11 16:12 package-lock.json
-rw-rw-r-- 1 usuario usuario 1474 may 26 17:20 phpunit.xml
-rw-rw-r-- 1 usuario usuario 93 may 4 16:23 postcss.config.js
-rwxrwxr-x 2 usuario usuario 4096 may 5 21:33 public
-rw-rw-r-- 1 usuario usuario 4570 may 19 21:19 README.md
-rwxrwxr-x 5 usuario usuario 4096 may 4 16:23 resources
-rwxrwxr-x 2 usuario usuario 4096 may 12 21:00 routes
-rwxrwxr-x 6 usuario usuario 4096 may 4 16:23 storage
-rw-rw-r-- 1 usuario usuario 541 may 4 16:23 tailwind.config.js
-rwxrwxr-x 4 usuario usuario 4096 may 4 16:23 tests
-rwxrwxr-x 49 usuario usuario 4096 may 12 20:59 vendor
-rw-rw-r-- 1 usuario usuario 263 may 4 16:23 vite.config.js
usuario@MV-Ubuntu:~/peliculapp$

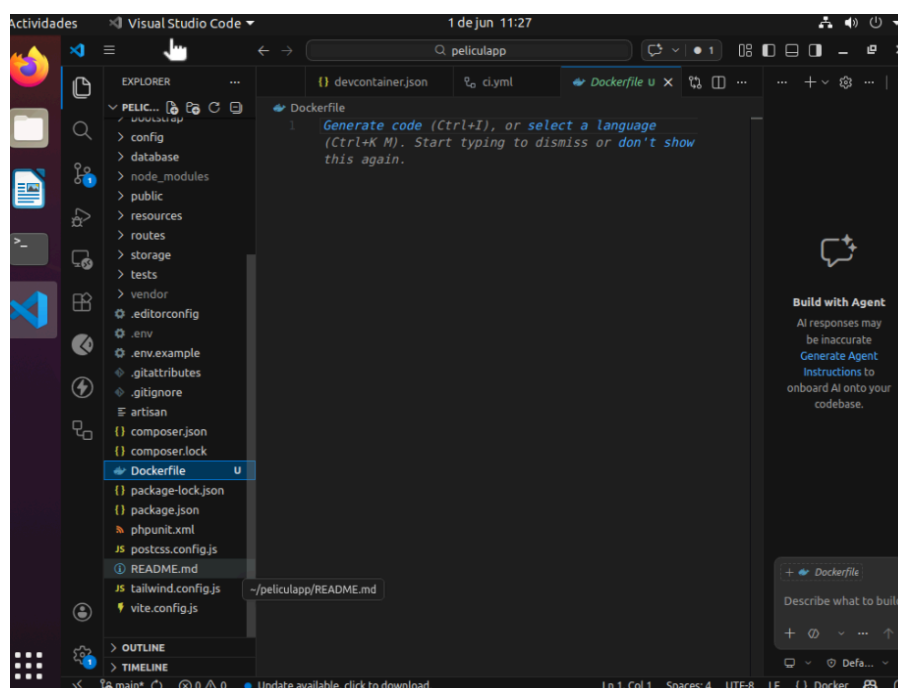
```

Creamos un Dockerfile vacío:

```

usuario@MV-Ubuntu:~/peliculapp$ ls
app                composer.lock      node_modules       postcss.config.js  routes             vendor
artisan            config             package.json        public              storage            vite.config.js
bootstrap           database            package-lock.json  README.md           tailwind.config.js
composer.json       Dockerfile          phpunit.xml         resources           tests
usuario@MV-Ubuntu:~/peliculapp$

```



Después en ese fichero ponemos:

FROM php:8.4-cli

**RUN apt-get update && apt-get install -y **

**git **

**unzip **

**libsqlite3-dev **

**sqlite3 **

&& docker-php-ext-install pdo pdo_sqlite

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

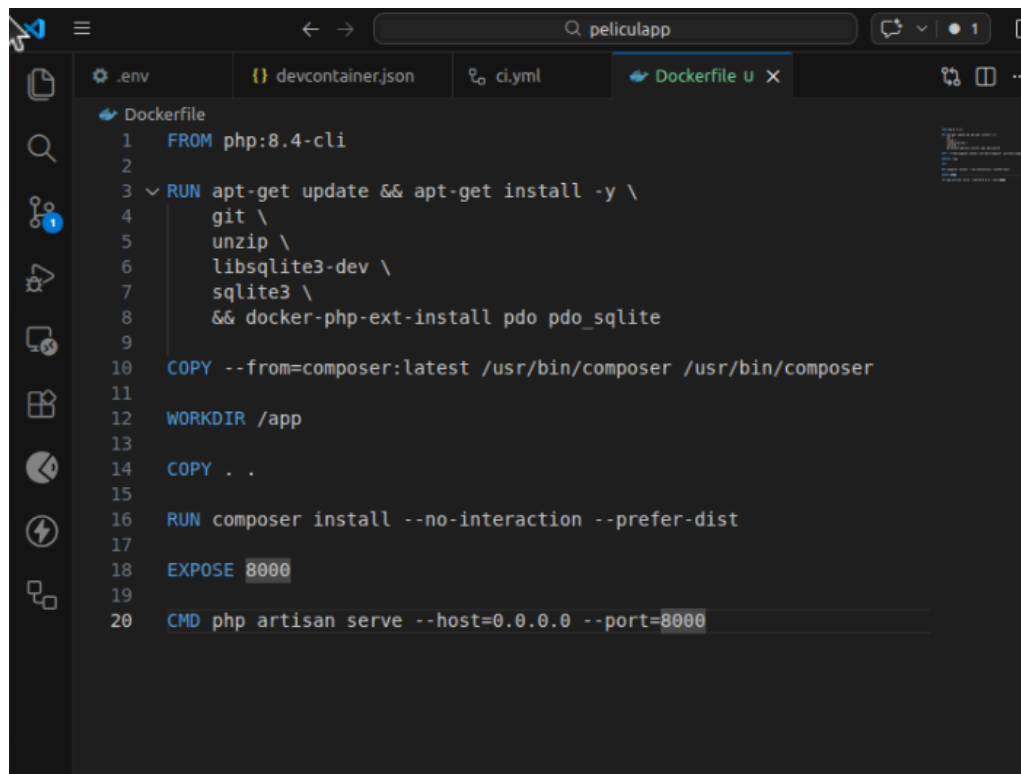
WORKDIR /app

COPY . .

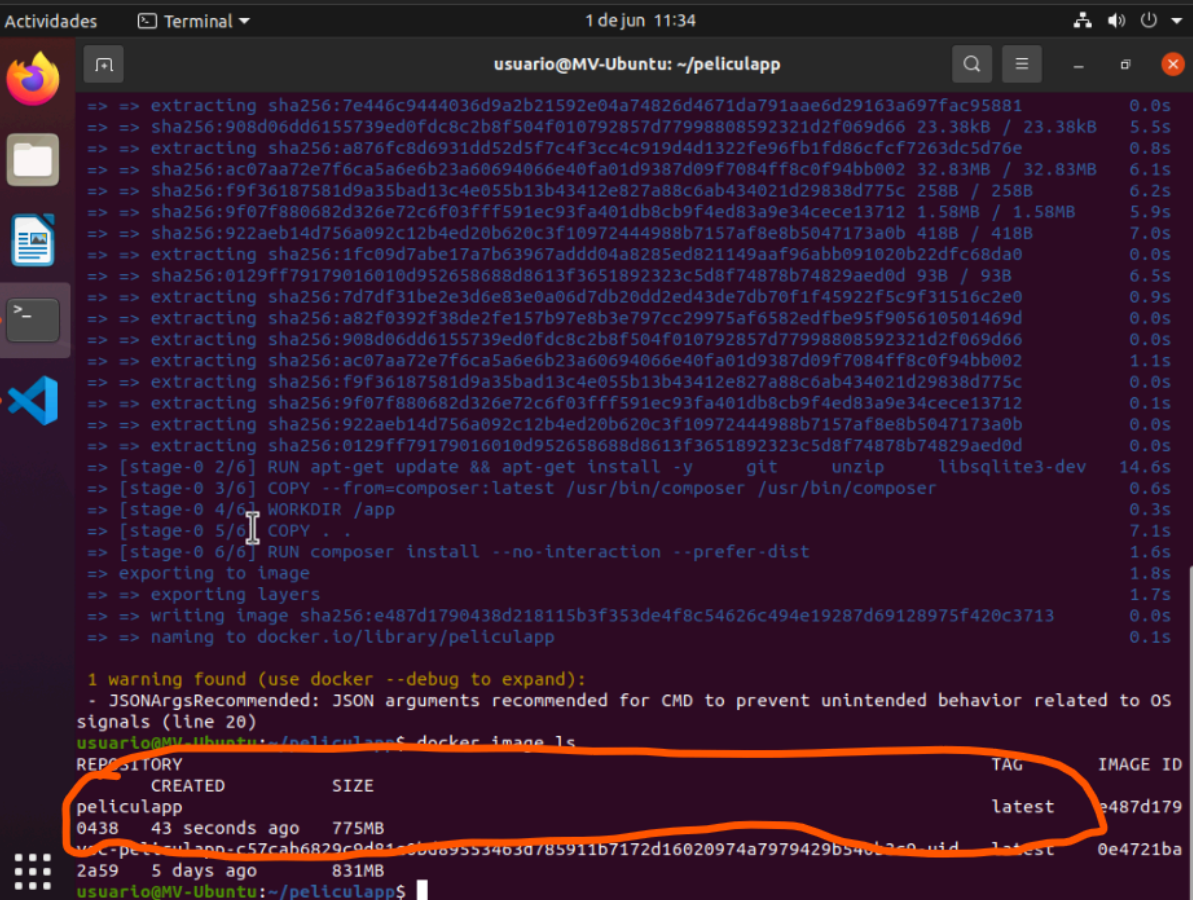
RUN composer install --no-interaction --prefer-dist

EXPOSE 8000

CMD php artisan serve --host=0.0.0.0 --port=8000



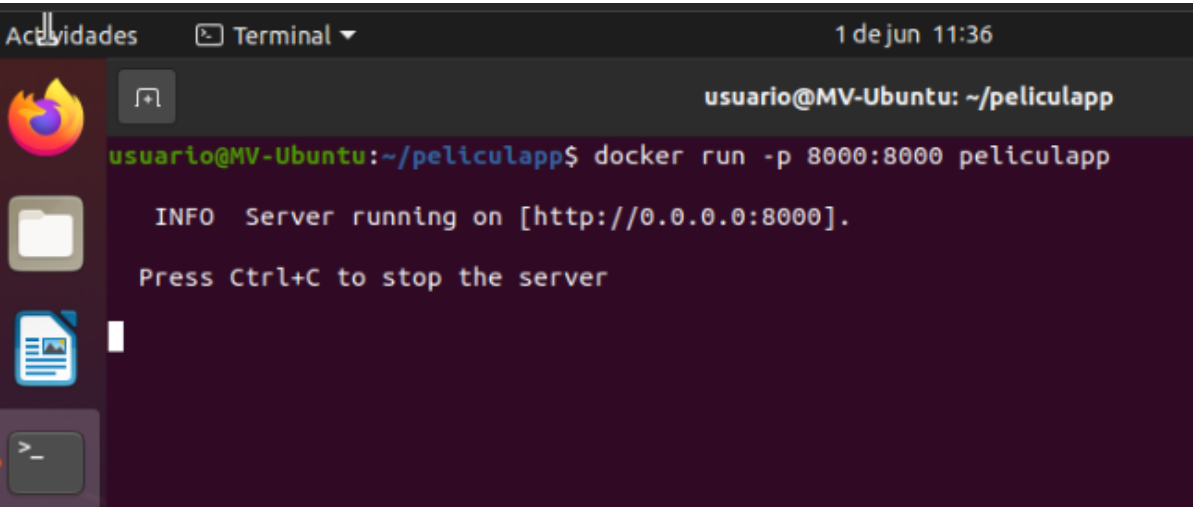
Tras crear el Dockerfile, lo probamos localmente con el comando **docker build -t peliculapp**.



```
=> extracting sha256:7e446c9444036d9a2b21592e04a74826d4671da791aae6d29163a697fac95881 0.0s
=> sha256:908d06dd6155739ed0fdc8c2b8f504f010792857d77998808592321d2f069d66 23.38kB / 23.38kB 5.5s
=> extracting sha256:a876fc8d6931dd52d5f7c4f3cc4c919d4d1322fe96fb1fd86cfcf7263dc5d76e 0.8s
=> sha256:ac07aa72e7f6ca5a6e6b23a60694066e40fa01d9387d09f7084ff8c0f94bb002 32.83MB / 32.83MB 6.1s
=> sha256:f9f36187581d9a35bad13c4e055b13b43412e827a88c6ab434021d29838d775c 258B / 258B 6.2s
=> sha256:9f07f880682d326e72c6f03fff591ec93fa401db8cb9f4ed83a9e34cece13712 1.58MB / 1.58MB 5.9s
=> sha256:922aeb14d756a092c12b4ed20b620c3f10972444988b7157af8e8b5047173a0b 418B / 418B 7.0s
=> extracting sha256:1fc09d7abe17a7b63967add04a8285ed821149aaf96abb091020b22dfc68da0 0.0s
=> sha256:0129ff79179016010d952658688d8613f3651892323c5d8f74878b74829aed0d 93B / 93B 6.5s
=> extracting sha256:7d7df31be2e3d6e83e0a06d7db20dd2ed43de7db70f1f45922f5c9f31516c2e0 0.9s
=> extracting sha256:a82f0392f38de2fe157b97e8b3e797cc29975af6582edf95f905610501469d 0.0s
=> sha256:908d06dd6155739ed0fdc8c2b8f504f010792857d77998808592321d2f069d66 0.0s
=> extracting sha256:ac07aa72e7f6ca5a6e6b23a60694066e40fa01d9387d09f7084ff8c0f94bb002 1.1s
=> extracting sha256:f9f36187581d9a35bad13c4e055b13b43412e827a88c6ab434021d29838d775c 0.0s
=> sha256:9f07f880682d326e72c6f03fff591ec93fa401db8cb9f4ed83a9e34cece13712 0.1s
=> extracting sha256:922aeb14d756a092c12b4ed20b620c3f10972444988b7157af8e8b5047173a0b 0.0s
=> extracting sha256:0129ff79179016010d952658688d8613f3651892323c5d8f74878b74829aed0d 0.0s
=> [stage-0 2/6] RUN apt-get update && apt-get install -y git unzip libsqlite3-dev 14.6s
=> [stage-0 3/6] COPY --from=composer:latest /usr/bin/composer /usr/bin/composer 0.6s
=> [stage-0 4/6] WORKDIR /app 0.3s
=> [stage-0 5/6] COPY . . 7.1s
=> [stage-0 6/6] RUN composer install --no-interaction --prefer-dist 1.6s
=> exporting to image 1.8s
=> exporting layers 1.7s
=> writing image sha256:e487d1790438d218115b3f353de4f8c54626c494e19287d69128975f420c3713 0.0s
=> naming to docker.io/library/peliculapp 0.1s

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 20)
usuario@MV-Ubuntu:~/peliculapp$ docker image ls
REPOSITORY          CREATED          SIZE          TAG          IMAGE ID
peliculapp          43 seconds ago  775MB         latest       e487d179
mic-peliculapp-c57cah6829c0d91c0b089553403d785911b7172d16020974a7979429b54013c0-uid:latest 0e4721ba
2a59 5 days ago  831MB
```

Corremos el contenedor con **docker run -p 8000:8000 peliculapp**



```
usuario@MV-Ubuntu:~/peliculapp$ docker run -p 8000:8000 peliculapp
INFO Server running on [http://0.0.0.0:8000].
Press Ctrl+C to stop the server
```

El contenedor ya funciona, por lo que el Dockerfile funciona.

Con todo esto, podemos ya hacer un commit y verificar que el error del Dockerfile en github se ha solucionado.

```
usuario@MV-Ubuntu:~/peliculapp$ git commit -m "Add dockerfile for CI"
[main 83d78c3] Add dockerfile for CI
1 file changed, 20 insertions(+)
create mode 100644 Dockerfile
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsaqrio
Password for 'https://ernsaqrio@github.com':
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 519 bytes | 519.00 KiB/s, listo.
Total 3 (delta 1), reusado 0 (delta 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ernsaqrio/peliculapp.git
25dacde..83d78c3 main -> main
usuario@MV-Ubuntu:~/peliculapp$
```

Ha pasado el build pero la auditoría ha fallado:

The screenshot shows a web browser window displaying a GitHub Actions workflow run. The URL is <https://github.com/ernsaqrio/peliculapp/actions/runs/26748066689>. The workflow is named "Add dockerfile for CI #4" and is in a "Failure" state. The summary shows the workflow was triggered via push 1 minute ago, pushed by ernsaqrio to the 83d78c3 commit on the main branch. The total duration is 1m 48s. The workflow steps are: Lint (Laravel Pint) (10s), PHPUnit Tests (22s), Docker Build (33s), and Composer Audit (34s). The Composer Audit step failed. The annotations section indicates 1 error and 4 warnings.

Summary

All Jobs

Lint (Laravel Pint) ✓

PHPUnit Tests ✓

Docker Build ✓

Composer Audit ✗

Run details

Usage

Workflow file

ci.yml

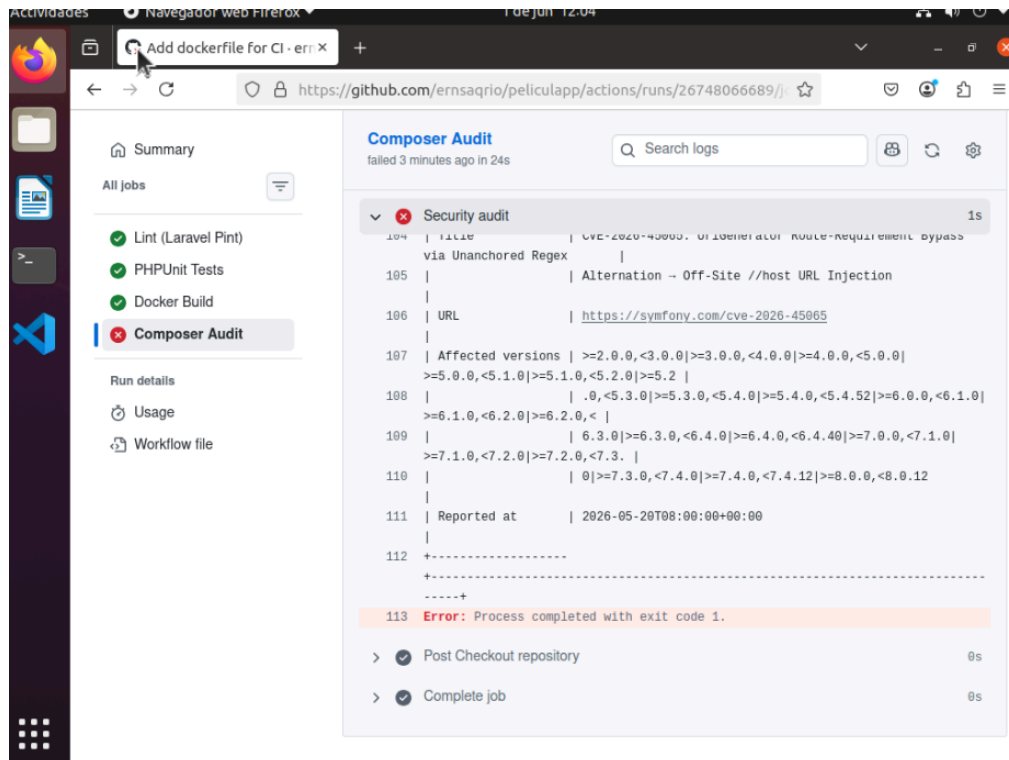
on: push

Lint (Laravel Pint) 10s → PHPUnit Tests 22s → Docker Build 33s → Composer Audit 34s

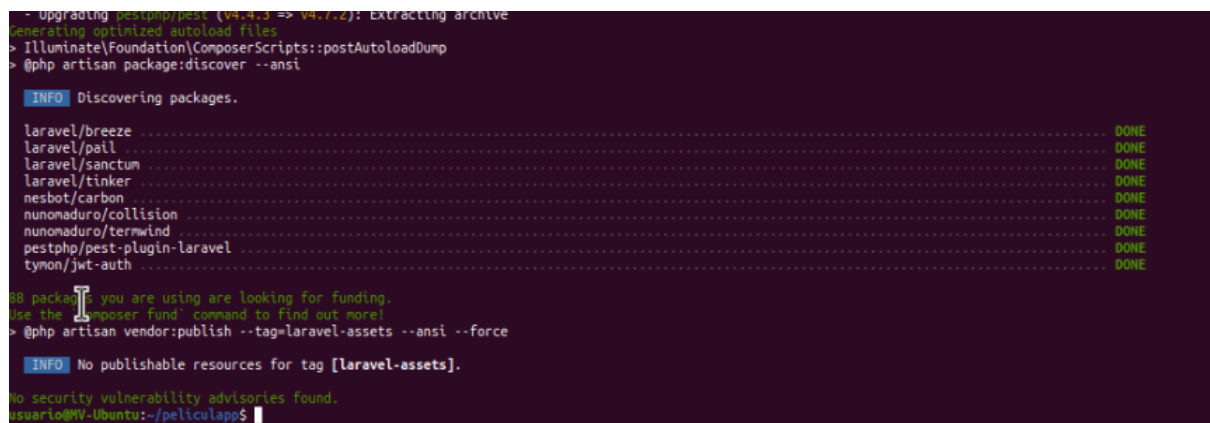
Annotations

1 error and 4 warnings

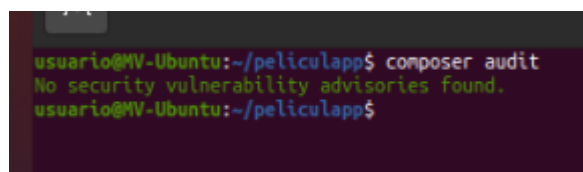
La auditoría falla ya que se han encontrado vulnerabilidades, que era algo de esperarse:



Actualizamos las dependencias con composer update:



Después usamos composer audit y vemos que no se muestran vulnerabilidades:



Verificamos los test y todo debe estar en orden:

```
Tests: 26 passed (49 assertions)
Duration: 0.29s

usuario@MV-Ubuntu:~/peliculapp$ php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\Auth\AuthTest
✓ user can login with valid credentials 0.10s
✓ user cannot login with invalid credentials 0.01s
✓ login requires email
✓ login requires password
✓ authenticated user can access me endpoint 0.01s
✓ me endpoint requires token
✓ user can refresh token 0.01s
✓ logout invalidates token 0.01s

PASS Tests\Feature\Director\DirectorTest
✓ authenticated user can list directors 0.01s
✓ authenticated user can create director 0.01s
✓ create director requires valid data 0.01s
✓ authenticated user can view director 0.01s
✓ authenticated user can update director 0.01s
✓ authenticated user can delete director 0.01s
✓ directors endpoints require authentication 0.01s

PASS Tests\Feature\Movie\MovieTest
✓ authenticated user can list movies 0.01s
✓ authenticated user can create movie 0.01s
✓ create movie requires valid data 0.01s
✓ authenticated user can view movie 0.01s
✓ authenticated user can update movie 0.01s
✓ authenticated user can delete movie 0.01s
✓ movies endpoints require authentication

PASS Tests\Feature\Security\SecurityTest
✓ token expirado devuelve 401 0.01s
✓ respuestas de error no exponen stack trace
✓ password no aparece en respuesta me 0.01s

Tests: 26 passed (49 assertions)
Duration: 0.28s

usuario@MV-Ubuntu:~/peliculapp$ ./vendor/bin/pint --test

.....
Laravel
61 files

usuario@MV-Ubuntu:~/peliculapp$
```

lo mismo ocurre con docker build

```
usuario@MV-Ubuntu:~/peliculapp$ docker build -t peliculapp .
[+] Building 12.2s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 406B                                0.0s
=> [internal] load metadata for docker.io/library/composer:latest 0.9s
=> [internal] load metadata for docker.io/library/php:8.4-cli      0.9s
=> [internal] load .dockerignore                                    0.1s
=> => transferring context: 2B                                       0.0s
=> [stage-0 1/6] FROM docker.io/library/php:8.4-cli@sha256:e82af972270274f1f6edb644155c9a132ac0c7ef568d91bb093fdd71f82e2853 0.0s
=> FROM docker.io/library/composer:latest@sha256:1b73755de4f19775ba6087fd5313664493e06fab72b6fc27dc2044e87bb7c4c3 0.0s
=> [internal] load build context                                    1.5s
=> => transferring context: 47.08MB                                   1.3s
=> CACHED [stage-0 2/6] RUN apt-get update && apt-get install -y git unzip libsqlite3-dev sqlite3 && docker-php-ext-install pdo 0.0s
=> CACHED [stage-0 3/6] COPY --from=composer:latest /usr/bin/composer /usr/bin/composer 0.0s
=> CACHED [stage-0 4/6] WORKDIR /app                               0.0s
=> [stage-0 5/6] COPY . .                                           5.3s
=> [stage-0 6/6] RUN composer install --no-interaction --prefer-dist 2.0s
=> exporting to image                                              1.8s
=> => exporting layers                                              1.6s
=> => writing image sha256:8b0635ea92dff65c9b849948e9378285f5fa57106ab01e1f5d05dbd2c1989ac6 0.0s
=> => naming to docker.io/library/peliculapp                        0.1s

usuario@MV-Ubuntu:~/peliculapp$
```

Tras comprobar las verificaciones y que ha habido éxito, procedemos a hacer el commit y verificar que el pipeline está en verde:

```
Actividades Terminal 1 de jun 13:29
usuario@MV-Ubuntu: ~/peliculapp$ git add .
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
(usa "git restore --staged <archivo>..." para sacar del área de stage)
modificados: composer.lock

usuario@MV-Ubuntu:~/peliculapp$ git commit -m "fix: update dependencies and security audit"
[main a8d9df7] fix: update dependencies and security audit
1 file changed, 693 insertions(+), 437 deletions(-)
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsaqrio
Password for 'https://ernsaqrio@github.com':
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 7.81 KiB | 3.90 MiB/s, listo.
Total 3 (delta 2), reusado 0 (delta 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/ernsaqrio/peliculapp.git
83d78c3..a8d9df7 main -> main
usuario@MV-Ubuntu:~/peliculapp$
```

Se puede apreciar el pipeline de integración continua indica que se ha pasado todos los trabajos y esté pipeline está en verde. Esto indica que ci se ha completado con éxito:

Actividades Navegador web Firefox 1 de jun 13:31

fix: update dependencies x

https://github.com/ernsaqrio/peliculapp/actions/runs/26752163069

ernsaqrio / peliculapp

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

← CI

fix: update dependencies and security audit #5

Re-run all jobs

Summary

All Jobs

Lint (Laravel Pint) ✓

PHPUnit Tests ✓

Docker Build ✓

Composer Audit ✓

Run details

Usage

Workflow file

Triggered via push 1 minute ago

ernsaqrio pushed → a8d9df7 main

Status Success

Total duration 1m 39s

Artifacts -

ci.yml

on: push

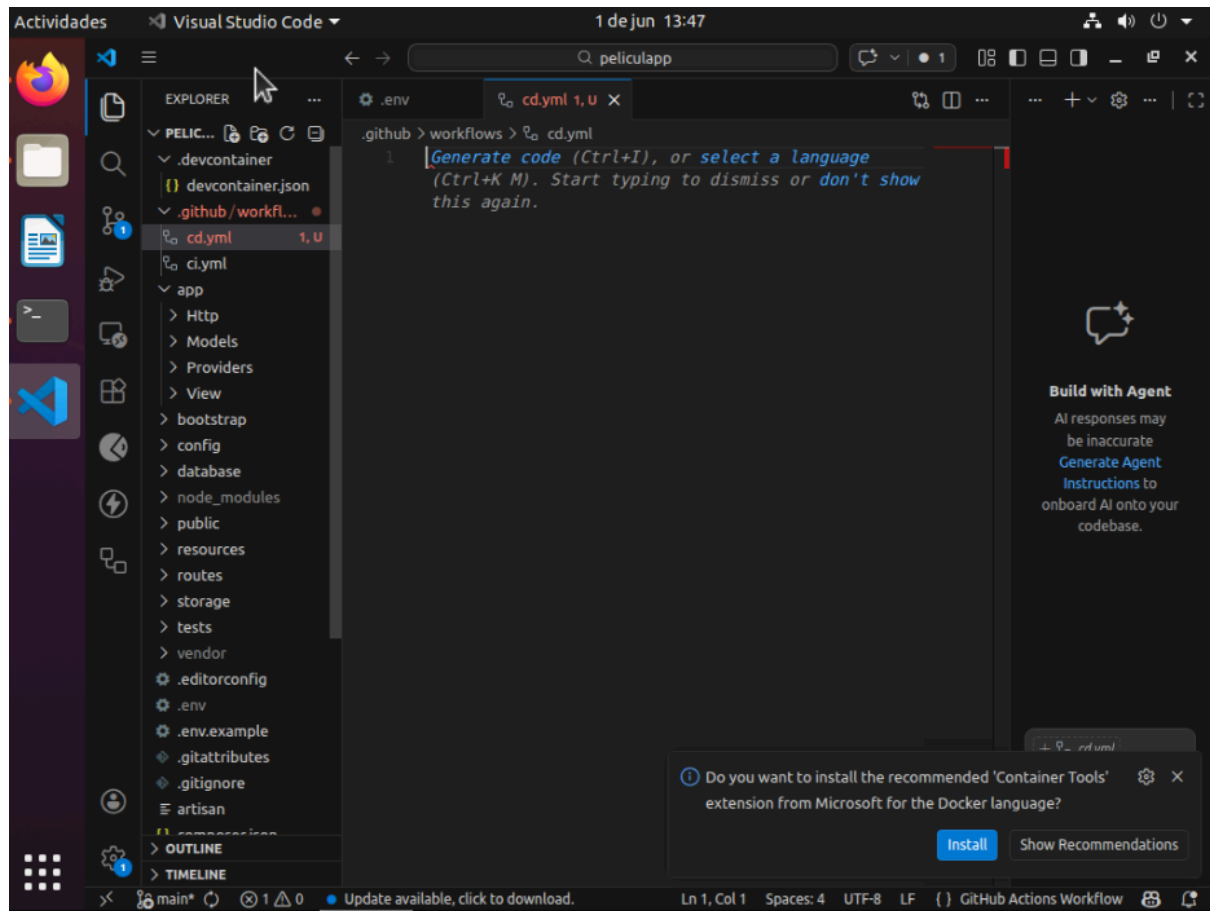
Lint (Laravel Pint) 17s → PHPUnit Tests 17s → Docker Build 32s → Composer Audit 21s

Annotations

4 warnings

3.3.- Pipeline de Despliegue Continuo (CD)

En el directorio `.github/workflows` creamos un archivo `cd.yml`:



En el archivo `cd.yml` ponemos:

name: CD

on:

push:

branches: [main]

permissions:

contents: read

packages: write

jobs:

docker-publish:

name: Build, Test and Publish Docker Image

runs-on: ubuntu-latest

env:

IMAGE_NAME: ghcr.io/\${{ github.repository }}

steps:

- uses: actions/checkout@v4

- name: Login to GHCR

uses: docker/login-action@v3

with:

registry: ghcr.io

username: \${{ github.actor }}

password: \${{ secrets.GITHUB_TOKEN }}

- name: Build Docker image

run: |

**docker build **

**-t \$IMAGE_NAME:\${{ github.sha }} **

**-t \$IMAGE_NAME:latest **

.

- name: Remove previous container (if exists)

run: docker rm -f peliculapp-test || true

- name: Run container

run: |

**docker run -d **

**--name peliculapp-test **

**-p 8000:8000 **

**-e APP_ENV=testing **

**-e APP_DEBUG=true **

**-e APP_KEY="\${{ secrets.APP_KEY }}" **

**-e JWT_SECRET="\${{ secrets.JWT_SECRET }}" **

\$IMAGE_NAME:\${{ github.sha }}

- name: Wait for app (robust healthcheck)

run: |

for i in {1..15}; do

if curl -s http://localhost:8000 > /dev/null; then

echo "App is up"

exit 0

fi

echo "Waiting..."

```
sleep 2
done
```

```
echo "App failed to start"
docker logs peliculapp-test || true
exit 1
```

- name: Smoke Test

run: |

```
STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8000)
```

```
echo "Status: $STATUS"
```

```
if [ "$STATUS" != "200" ] && [ "$STATUS" != "302" ]; then
```

```
docker logs peliculapp-test || true
```

```
exit 1
```

```
fi
```

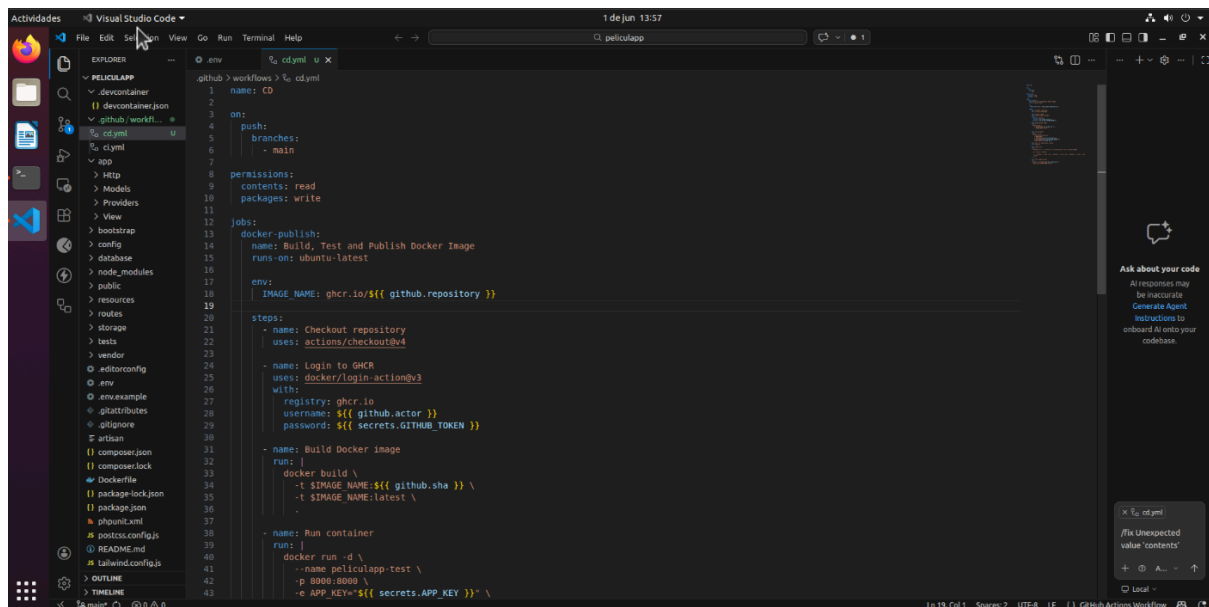
- name: Push image to GHCR

run: |

```
docker push $IMAGE_NAME:${{ github.sha }}
```

```
docker push $IMAGE_NAME:latest
```

El archivo quedaría:



Despues se ejecuta los comandos:

- `git add .`

- git status
- git commit git push origin main

```

usuario@MV-Ubuntu:~$ cd peliculapp
usuario@MV-Ubuntu:~/peliculapp$ code .
usuario@MV-Ubuntu:~/peliculapp$ git add .
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
nuevos archivos: .github/workflows/cd.yml

usuario@MV-Ubuntu:~/peliculapp$ git commit -m "add CD pipeline with docker publish"
[main 164d3c1] add CD pipeline with docker publish
1 file changed, 63 insertions(+)
create mode 100644 .github/workflows/cd.yml
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsaqrio
Password for 'https://ernsaqrio@github.com':
Enumerando objetos: 8, listo.
Contando objetos: 100% (8/8), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (4/4), listo.
Escribiendo objetos: 100% (5/5), 1010 bytes | 1010.00 KiB/s, listo.
Total 5 (delta 1), reusado 0 (delta 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ernsaqrio/peliculapp.git
   a8d9df7..164d3c1  main -> main
usuario@MV-Ubuntu:~/peliculapp$

```

Vemos que el CI funciona pero el CD no por el smoke test.

Annotations
1 error and 1 warning

Build, Test and Publish Docker Image

failed 3 minutes ago in 1m 4s

Search logs
Explain error

Set up job
Checkout repository
Login to GHCR
Build Docker image
Run container
Wait for application startup
Smoke Test
Push image to GHCR
Post Login to GHCR
Post Checkout repository
Complete job

1s
1s
1s
36s
0s
29s
0s
15s
15s
15s
15s

```

1 ▶ Run STATUS=$(curl -s -o /dev/null -w "%(http_code)" http://localhost:8000)
12 Status: 500
13 Error: Process completed with exit code 1.

```

Tras varios arreglos volvemos a hacer el commit y el push:

```

usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
modificados:   .github/workflows/cd.yml
modificados:   .github/workflows/ci.yml

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
usuario@MV-Ubuntu:~/peliculapp$ git add .github/workflows/ci.yml .github/workflows/cd.yml
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
(usa "git restore --staged <archivo>..." para sacar del área de stage)
modificados:   .github/workflows/cd.yml
modificados:   .github/workflows/ci.yml

usuario@MV-Ubuntu:~/peliculapp$ git commit -m "ci/cd: improve pipeline debugging and health checks"
[main 11f1e35] ci/cd: improve pipeline debugging and health checks
 2 files changed, 43 insertions(+), 50 deletions(-)
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsaqrio
Password for 'https://ernsaqrio@github.com':
Enumerando objetos: 11, listo.
Contando objetos: 100% (11/11), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (5/5), listo.
Escribiendo objetos: 100% (6/6), 1014 bytes | 1014.00 KiB/s, listo.
Total 6 (delta 3), reusado 0 (delta 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/ernsaqrio/peliculapp.git
 164d3c1..11f1e35  main -> main
usuario@MV-Ubuntu:~/peliculapp$

```

Sin embargo vuelve a fallar. Se han modificado routes/api.php y el cd.yml de tal forma que pasará la prueba de smoke.

Se ha introducido

- Ruta /api/health
- Smoke test contra una ruta de salud real
- Logs automáticos si falla
- Espera activa en lugar de sleep 20
- Limpieza del contenedor antes de arrancar

Tras realizar las modificaciones necesarias, se hacen unas pruebas en local para asegurar el funcionamiento del smoke test:


```
usuario@MV-Ubuntu:~/peliculapp$ docker build -t peliculapp .
[+] Building 13.8s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 406B
=> [internal] load metadata for docker.io/library/composer:latest
=> [internal] load metadata for docker.io/library/php:8.4-cli
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [stage-0 1/6] FROM docker.io/library/php:8.4-cli@sha256:e82af972270274f1f6edb644155c9a132ac0c7ef568d91bb093fdd71f82e2853
=> [internal] load build context
=> => transferring context: 1.78MB
=> FROM docker.io/library/composer:latest@sha256:1b73755de4f19775ba6087fd5313664493e06fab72b6fc27dc2044e87bb7c4c3
=> CACHED [stage-0 2/6] RUN apt-get update && apt-get install -y git unzip libsqli3-dev sqli3 && docker-php-ext-install pdo pdo_sqli3
=> CACHED [stage-0 3/6] COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
=> CACHED [stage-0 4/6] WORKDIR /app
=> [stage-0 5/6] COPY . .
=> [stage-0 6/6] RUN composer install --no-interaction --prefer-dist
=> exporting to image
=> => exporting layers
=> => writing image sha256:0a67522fa3e7d7f7c6777ac39ec133f801afc8de724968679fc915fd889cd68
=> => naming to docker.io/library/peliculapp

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 20)
usuario@MV-Ubuntu:~/peliculapp$ docker run -d --name peliculapp-test -p 8000:8000 peliculapp
b93ca6aa3b0660e50d157fc7cc9b3146c5af7763382b51123ced9787a31dd5
usuario@MV-Ubuntu:~/peliculapp$ curl http://localhost:8000/api/health
{"status":"ok"}usuario@MV-Ubuntu:~/peliculapp$
```

Posteriormente se hace el commit y el push y se comprueba que el pipeline CI y el pipeline CD funcionen:

```
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
    modificados:   .github/workflows/cd.yml
    modificados:   routes/api.php

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
usuario@MV-Ubuntu:~/peliculapp$ git add .
usuario@MV-Ubuntu:~/peliculapp$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificados:   .github/workflows/cd.yml
    modificados:   routes/api.php

usuario@MV-Ubuntu:~/peliculapp$ git commit -m "cd: add health endpoint and improve smoke test"
[main 05c2902] cd: add health endpoint and improve smoke test
 2 files changed, 38 insertions(+), 14 deletions(-)
usuario@MV-Ubuntu:~/peliculapp$ git push origin main
Username for 'https://github.com': ernsagrio
Password for 'https://ernsagrio@github.com':
Enumerando objetos: 13, listo.
Contando objetos: 100% (13/13), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (6/6), listo.
Escribiendo objetos: 100% (7/7), 1.08 KiB | 1.08 MiB/s, listo.
Total 7 (delta 3), reusado 0 (delta 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/ernsagrio/peliculapp.git
   15ba691..05c2902  main -> main
usuario@MV-Ubuntu:~/peliculapp$
```

Tanto el CI como el CD funcionan:

 Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback 

15 workflow runs	Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
 style: fix Pint formatting CD #5: Commit 4a4ac08 pushed by ernsagrio	main		 2 minutes ago  1m 10s		...
 style: fix Pint formatting CI #10: Commit 4a4ac08 pushed by ernsagrio	main		 2 minutes ago  2m 3s		...
 cd: add health endpoint and improve smoke test CD #4: Commit 05c2902 pushed by ernsagrio	main		 5 minutes ago  1m 22s		...

← → ↺

https://github.com/ernsaqrio/peliculapp/actions/runs/26763886525

☆

🔒 🔔 📧 ⋮

ernsaqrio / peliculapp

🔍 Type to search

📦 + 🔁 📄 🌐

<> Code 🔍 Issues 📄 Pull requests 🏠 Agents 🔄 Actions 📁 Projects 📖 Wiki 🔒 Security and quality 📈 Insights ⚙️ Settings

← CI

🟢 style: fix Pint formatting #10

Re-run all jobs ⋮

Summary

All jobs

Lint (Laravel Pint)

PHPUnit Tests

Docker Build

Composer Audit

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

Status

Total duration

Artifacts

ernsaqrio pushed

4a4ac08

#317

Success

2m 3s

—

cl.yml

on: push

Lint (Laravel Pint)

20s

PHPUnit Tests

29s

Docker Build

37s

Composer Audit

18s

Annotations

4 warnings

Lint (Laravel Pint)

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4. Actions will be forced to run with Node.js 24 by default st...

Show more

PHPUnit Tests

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4. Actions will be forced to run with Node.js 24 by default st...

Show more

← → ↺

https://github.com/ernsaqrio/peliculapp/actions/runs/26763887618

☆

🔒 🔔 📧 ⋮

ernsaqrio / peliculapp

🔍 Type to search

📦 + 🔁 📄 🌐

<> Code 🔍 Issues 📄 Pull requests 🏠 Agents 🔄 Actions 📁 Projects 📖 Wiki 🔒 Security and quality 📈 Insights ⚙️ Settings

← CD

🟢 style: fix Pint formatting #5

Re-run all jobs ⋮

Summary

All jobs

Build, Test and Publish Docker Image

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

Status

Total duration

Artifacts

ernsaqrio pushed

4a4ac08

#317

Success

1m 10s

—

cd.yml

on: push

Build, Test and Publish D...

1m 5s

Annotations

1 warning

Build, Test and Publish Docker Image

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, docker/login-action@3. Actions will be forced to run with ...

Show more